CFD LAB

LABORATORY RECORD

Experiments 1–11

Name of Course: Computational Fluid Dynamics and Heat Transfer Lab

Course Code: 20ME406P

Semester: Seven (7th)

Branch: Mechanical

Name of Student: Somin Kachhi

Roll Number: 23BME031

Academic Year: 2026-2027

Contents

EXPERIMENT 1 – Flow over a NACA 0012 airfoil 3
EXPERIMENT 2 – Counter-flow heat exchanger 10
EXPERIMENT 3 – Flow past an immersed circular cylinder 18
EXPERIMENT 4 – Radial rotating annular passage: pressure and heat transfer 22
EXPERIMENT 5 – Couette flow 29
EXPERIMENT 6 – Two-dimensional steady heat conduction 32
EXPERIMENT 7 – Shallow-water wave propagation 35
EXPERIMENT 8 – Rayleigh–Taylor instability using a VOF-type model 45
EXPERIMENT 9 – Lid-driven cavity flow 57
EXPERIMENT 10 – Turbulent pipe-flow model comparison 67
EXPERIMENT 11 – Convection–diffusion transport 78

EXPERIMENT 1 – Flow over a NACA 0012 airfoil

Aim

To numerically simulate two-dimensional incompressible flow over a NACA 0012 airfoil at a 5° angle of attack and examine the velocity, vorticity and continuity behaviour.

Objective

- Generate the NACA 0012 airfoil geometry and embed it in a rectangular computational domain.
- Solve the incompressible momentum equations using finite-difference approximations and a pressure-Poisson correction.
- Apply no-slip conditions on the airfoil and free-stream conditions at the far-field boundaries.
- Visualise velocity magnitude, vorticity and the continuity residual history.

Governing Equation

Continuity: $\partial u/\partial x + \partial v/\partial y = 0$

Momentum (dimensionless):

$\partial u/\partial t + u\partial u/\partial x + v\partial u/\partial y = -\partial p/\partial x + (1/Re)\nabla^2 u$

$\partial v/\partial t + u\partial v/\partial x + v\partial v/\partial y = -\partial p/\partial y + (1/Re)\nabla^2 v$

The code uses an explicit predictor step, a Jacobi pressure-Poisson solve and velocity correction. Convective derivatives are upwinded; diffusion and pressure gradients are finite-differenced.

Boundary Condition

- Inlet: $u = U_\infty = 1$, $v = 0$.
- Outlet: zero-gradient velocity; pressure fixed to $p = 0$.
- Top and bottom far-field: $u = U_\infty$, $v = 0$.
- Airfoil surface: no-slip velocity enforced approximately with ghost-node values; pressure uses a zero-gradient treatment at the solid boundary.

Code

Main inputs: NX=120, NY=60, Re=100, angle of attack=5°, 300 outer iterations, 80 pressure-Poisson Jacobi sweeps, domain 5.0 × 2.5, U_∞=1.0 and chord=1.0.

```
import csv
import math
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path

# Grid and flow settings
NX, NY = 120, 60
RE = 100
ALPHA_DEG = 5
ITERATIONS = 300
POISSON_ITERATIONS = 80

LX, LY = 5.0, 2.5
U_INF = 1.0
CHORD = 1.0
X_LE = 1.6

OUTPUT_DIR = Path("airfoil_result_5deg")


def naca0012_half_thickness(x):
    # NACA 0012 half-thickness
    xc = np.clip(x, 0.0, 1.0)

    yt = 5.0 * 0.12 * (
        0.2969 * np.sqrt(xc)
        - 0.1260 * xc
```

```
        - 0.3516 * xc**2
        + 0.2843 * xc**3
        - 0.1036 * xc**4
    )

    return np.where((x >= 0.0) & (x <= 1.0), yt, -1.0)


def airfoil_outline(alpha_deg, n=401):
    # Create the airfoil outline
    beta = np.linspace(0.0, math.pi, n)
    xc = 0.5 * (1.0 - np.cos(beta))
    yt = naca0012_half_thickness(xc)

    xb = np.concatenate((xc[::-1], xc[1:])) * CHORD
    yb = np.concatenate((-yt[::-1], yt[1:])) * CHORD

    angle = math.radians(alpha_deg)
    xq = X_LE + 0.25 * CHORD

    xr = xb - 0.25 * CHORD

    x = xq + math.cos(angle) * xr - math.sin(angle) * yb
    y = math.sin(angle) * xr + math.cos(angle) * yb

    return x, y


def airfoil_mask(x, y, alpha_deg):
    # True means inside the airfoil
    angle = math.radians(alpha_deg)
    xq = X_LE + 0.25 * CHORD

    xr = (
        math.cos(angle) * (x - xq)
        + math.sin(angle) * y
        + 0.25 * CHORD
    )

    yr = (
        -math.sin(angle) * (x - xq)
        + math.cos(angle) * y
    )

    yt = naca0012_half_thickness(xr / CHORD) * CHORD

    return (yt >= 0.0) & (np.abs(yr) <= yt)
def find_ghost_nodes(solid):
    # Ghost nodes are solid nodes next to the fluid
    ghost = np.zeros_like(solid, dtype=bool)

    ghost[1:, :] |= solid[1:, :] & ~solid[:-1, :]
    ghost[:-1, :] |= solid[:-1, :] & ~solid[1:, :]
    ghost[:, 1:] |= solid[:, 1:] & ~solid[:, :-1]
    ghost[:, :-1] |= solid[:, :-1] & ~solid[:, 1:]

    return ghost


def apply_ghost_values(field, solid, ghost, wall_value=0.0,
                       no_slip=False):
    # Use the closest fluid node for each ghost node
    old = field.copy()
    ny, nx = field.shape

    for j in range(1, ny - 1):
        for i in range(1, nx - 1):

            if not ghost[j, i]:
                continue

            neighbors = [
                (j, i - 1),
                (j, i + 1),
                (j - 1, i),
                (j + 1, i)
            ]

            for jj, ii in neighbors:
                if not solid[jj, ii]:
                    if no_slip:
                        # Approximate a stationary wall halfway
                        field[j, i] = 2.0 * wall_value - old[jj, ii]
                    else:
                        # Zero-gradient condition
                        field[j, i] = old[jj, ii]
                    break

    # Keep deeper solid nodes at the wall value
    inside = solid & ~ghost
    field[inside] = wall_value

    return field


def ddx_central(f, dx):
    # Central difference in x
    out = np.zeros_like(f)
    out[1:-1, 1:-1] = (
```

```
        f[1:-1, 2:] - f[1:-1, :-2]
    ) / (2.0 * dx)

    return out


def ddy_central(f, dy):
    # Central difference in y
    out = np.zeros_like(f)
    out[1:-1, 1:-1] = (
        f[2:, 1:-1] - f[:-2, 1:-1]
    ) / (2.0 * dy)

    return out


def laplacian(f, dx, dy):
    # Second derivatives in x and y
    out = np.zeros_like(f)

    out[1:-1, 1:-1] = (
        (f[1:-1, 2:] - 2.0 * f[1:-1, 1:-1]
         + f[1:-1, :-2]) / dx**2
        +
        (f[2:, 1:-1] - 2.0 * f[1:-1, 1:-1]
         + f[:-2, 1:-1]) / dy**2
    )

    return out


def ddx_upwind(f, u, dx):
    # Upwind difference follows the flow direction
    out = np.zeros_like(f)

    out[1:-1, 1:-1] = np.where(
        u[1:-1, 1:-1] >= 0.0,
        (f[1:-1, 1:-1] - f[1:-1, :-2]) / dx,
        (f[1:-1, 2:] - f[1:-1, 1:-1]) / dx
    )

    return out


def ddy_upwind(f, v, dy):
    # Upwind difference follows the flow direction
    out = np.zeros_like(f)

    out[1:-1, 1:-1] = np.where(
        v[1:-1, 1:-1] >= 0.0,
        (f[1:-1, 1:-1] - f[:-2, 1:-1]) / dy,
        (f[2:, 1:-1] - f[1:-1, 1:-1]) / dy
    )

    return out
def apply_bc(u, v, p, solid, ghost):
    # Inlet
    u[:, 0] = U_INF
    v[:, 0] = 0.0

    # Outlet: zero-gradient velocity
    u[:, -1] = u[:, -2]
    v[:, -1] = v[:, -2]

    # Top and bottom: free-stream flow
    u[0, :] = U_INF
    u[-1, :] = U_INF
    v[0, :] = 0.0
    v[-1, :] = 0.0

    # Pressure boundaries
    p[:, 0] = p[:, 1]
    p[:, -1] = 0.0
    p[0, :] = p[1, :]
    p[-1, :] = p[-2, :]

    # Apply wall ghost values to velocity and pressure
    apply_ghost_values(u, solid, ghost, no_slip=True)
    apply_ghost_values(v, solid, ghost, no_slip=True)
    apply_ghost_values(p, solid, ghost, no_slip=False)


def poisson_jacobi(rhs, dx, dy, sweeps, solid, ghost):
    # Solve Laplacian(phi) = rhs
    phi = np.zeros_like(rhs)

    inv_dx2 = 1.0 / dx**2
    inv_dy2 = 1.0 / dy**2
    denom = 2.0 * (inv_dx2 + inv_dy2)

    fluid = ~solid

    for _ in range(sweeps):
        old = phi.copy()

        new_phi = (
            (old[1:-1, 2:] + old[1:-1, :-2]) * inv_dx2
            + (old[2:, 1:-1] + old[:-2, 1:-1]) * inv_dy2
            - rhs[1:-1, 1:-1]
```

```
        ) / denom

        # Update only fluid nodes
        region = fluid[1:-1, 1:-1]
        phi[1:-1, 1:-1][region] = new_phi[region]

        # Apply pressure boundary conditions
        phi[:, 0] = phi[:, 1]
        phi[:, -1] = 0.0
        phi[0, :] = phi[1, :]
        phi[-1, :] = phi[-2, :]

        apply_ghost_values(phi, solid, ghost, no_slip=False)

    return phi
def main():
    OUTPUT_DIR.mkdir(exist_ok=True)

    # Build the grid
    x1 = np.linspace(0.0, LX, NX)
    y1 = np.linspace(-0.5 * LY, 0.5 * LY, NY)
    x, y = np.meshgrid(x1, y1)

    dx = x1[1] - x1[0]
    dy = y1[1] - y1[0]

    nu = U_INF * CHORD / RE

    # Airfoil and ghost nodes
    solid = airfoil_mask(x, y, ALPHA_DEG)
    ghost = find_ghost_nodes(solid)

    # Initial conditions
    u = np.full((NY, NX), U_INF, dtype=float)
    v = np.zeros_like(u)
    p = np.zeros_like(u)

    apply_bc(u, v, p, solid, ghost)

    # Stable time step
    dt = min(
        0.20 * min(dx, dy) / U_INF,
        0.15 * min(dx**2, dy**2) / nu
    )

    history = []

    print("-" * 60)
    print(f"Grid: {NX} x {NY}")
    print(f"Reynolds number: {RE}")
    print(f"Angle of attack: {ALPHA_DEG} degrees")
    print(f"Time step: {dt:.5e}")
    print("-" * 60)

    for step in range(1, ITERATIONS + 1):

        # Step 1: Predictor velocity
        u_old = u.copy()
        v_old = v.copy()

        du = (
            -u_old * ddx_upwind(u_old, u_old, dx)
            -v_old * ddy_upwind(u_old, v_old, dy)
            +nu * laplacian(u_old, dx, dy)
        )

        dv = (
            -u_old * ddx_upwind(v_old, u_old, dx)
            -v_old * ddy_upwind(v_old, v_old, dy)
            +nu * laplacian(v_old, dx, dy)
        )

        us = u_old + dt * du
        vs = v_old + dt * dv

        apply_bc(us, vs, p, solid, ghost)

        # Step 2: Pressure correction
        rhs = (
            ddx_central(us, dx)
            + ddy_central(vs, dy)
        ) / dt

        rhs[solid] = 0.0

        phi = poisson_jacobi(
            rhs, dx, dy, POISSON_ITERATIONS, solid, ghost
        )

        # Step 3: Correct velocity
        u = us - dt * ddx_central(phi, dx)
        v = vs - dt * ddy_central(phi, dy)

        apply_bc(u, v, p, solid, ghost)

        # Update pressure
        p += phi
        p[solid & ~ghost] = 0.0
```

```
    # Reapply boundary conditions
    apply_bc(u, v, p, solid, ghost)

    # Measure continuity in the fluid
    div = ddx_central(u, dx) + ddy_central(v, dy)
    fluid = ~solid

    continuity = float(
        np.sqrt(np.mean(div[fluid] ** 2))
    )

    speed = np.sqrt(u**2 + v**2)
    max_speed = float(np.max(speed[fluid]))

    history.append([step, step * dt, continuity, max_speed])

    # Stop if the solution becomes invalid
    if not (
        np.all(np.isfinite(u))
        and np.all(np.isfinite(v))
        and np.all(np.isfinite(p))
    ):
        raise FloatingPointError(
            f"Invalid value detected at iteration {step}"
        )

    if max_speed > 25.0:
        raise FloatingPointError(
            f"Solution diverged at iteration {step}"
        )

    if (
        step == 1
        or step == ITERATIONS
        or step % max(1, ITERATIONS // 5) == 0
    ):
        print(
            f"Iteration {step:4d} | "
            f"Continuity = {continuity:.4e} | "
            f"Max speed = {max_speed:.4f}"
        )

# Final flow fields
speed = np.sqrt(u**2 + v**2)
vorticity = ddx_central(v, dx) - ddy_central(u, dy)

# Save iteration history
with open(
    OUTPUT_DIR / "history.csv",
    "w",
    newline="",
    encoding="utf-8"
) as file:
    writer = csv.writer(file)
    writer.writerow([
        "iteration", "pseudo_time", "continuity", "max_speed"
    ])
    writer.writerows(history)

# Save numerical fields
np.savez(
    OUTPUT_DIR / "airfoil_fields.npz",
    x=x, y=y, u=u, v=v, p=p,
    speed=speed, vorticity=vorticity,
    solid=solid, ghost=ghost
)

# Airfoil outline
wall_x, wall_y = airfoil_outline(ALPHA_DEG)

# Plot results
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

axes[0].semilogy(
    np.array(history)[:, 0],
    np.maximum(np.array(history)[:, 2], 1e-14)
)
axes[0].set_title("Continuity")
axes[0].set_xlabel("Iteration")
axes[0].set_ylabel("L2 divergence")
axes[0].grid(True)

plot = axes[1].contourf(
    x, y, speed, levels=30
)
axes[1].plot(wall_x, wall_y, "k-", linewidth=1.2)
fig.colorbar(plot, ax=axes[1], label="Speed")
axes[1].set_title("Velocity magnitude")

plot = axes[2].contourf(
    x, y, vorticity, levels=30, cmap="RdBu_r"
)
axes[2].plot(wall_x, wall_y, "k-", linewidth=1.2)
fig.colorbar(plot, ax=axes[2], label="Vorticity")
axes[2].set_title("Vorticity")

for ax in axes[1:]:
    ax.set_xlabel("x")
    ax.set_ylabel("y")
```

```
        ax.set_aspect("equal", adjustable="box")
        ax.set_xlim(0.5, 4.0)
        ax.set_ylim(-1.0, 1.0)

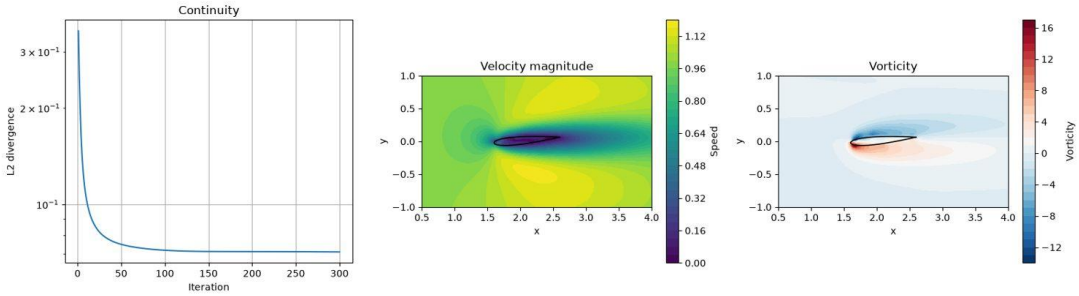
    fig.tight_layout()
    fig.savefig(OUTPUT_DIR / "results.png", dpi=150)
    plt.show()

    print("-" * 60)
    print(f"Final continuity: {history[-1][2]:.4e}")
    print(f"Results saved in: {OUTPUT_DIR.resolve()}")


if __name__ == "__main__":
    main()
```

Output

The following figures are reproduced from the output folder supplied for this experiment.



Experiment 1 output: Figure_1

```
Time step: 8.40336e-03
------------------------------------------------------------
Iteration   1 | Continuity = 3.4824e-01 | Max speed = 1.1890
Iteration  60 | Continuity = 7.3907e-02 | Max speed = 1.1721
Iteration 120 | Continuity = 7.1479e-02 | Max speed = 1.1743
Iteration 180 | Continuity = 7.1234e-02 | Max speed = 1.1748
Iteration 240 | Continuity = 7.1187e-02 | Max speed = 1.1756
Iteration 300 | Continuity = 7.1024e-02 | Max speed = 1.1747
------------------------------------------------------------
Final continuity: 7.1024e-02
```

Experiment 1 output: Figure_2

Results and Discussion

The saved field figure shows the airfoil in the flow domain, a velocity-magnitude disturbance around the body and a wake extending downstream. The vorticity plot shows opposite-signed vorticity regions near the airfoil and in its wake. The continuity-history curve decreases rapidly at first but levels off. The saved console image reports a final continuity measure of approximately 7.10×10^{-2} after 300 iterations; therefore, the displayed run should be described as a partially reduced residual, not as a fully converged solution.

Conclusion

The code produces a physically recognisable airfoil flow field and wake-vorticity pattern. The continuity residual decreases substantially but remains finite in the saved run, so additional iterations, grid refinement and a more stringent convergence study would be needed before treating the solution as quantitatively converged.

EXPERIMENT 2 – Counter-flow heat exchanger

Aim

To model transient heat transfer between hot and cold brine streams separated by a copper wall and compare the resulting temperature field and bulk-temperature distributions.

Objective

- Construct a two-stream heat-exchanger geometry with a conducting copper partition.
- Prescribe counter-flow or parallel-flow velocity profiles for the two brine channels.
- Solve the advection–diffusion energy equation using a finite-difference time-marching scheme.
- Evaluate temperature contours and bulk temperatures along the exchanger.

Governing Equation

Energy equation represented in the code:

$\partial T/\partial t + u\partial T/\partial x = \nabla\cdot(D\nabla T)$

Here D is a piecewise thermal-diffusion coefficient: D_cold=1/(Re_cold Pr), D_hot=1/(Re_hot Pr), and D_wall is scaled by the specified copper-to-brine conductivity ratio. The velocity field is prescribed rather than solved from Navier–Stokes equations.

Boundary Condition

- Hot brine inlet at the left: T = 2.
- Hot brine outlet at the right: zero streamwise temperature gradient.
- For counter-flow, cold brine enters from the right: T = −8; its left outlet uses zero-gradient treatment.
- Copper-wall ends: zero-gradient temperature treatment.
- Outer top and bottom surfaces: insulated, implemented by copying the adjacent interior temperature.

Code

Main inputs: Lx=8, Ly=4, nx=160, 20 cold-channel rows, 6 copper-wall rows, 20 hot-channel rows, Re_hot=500, Re_cold=250, Pr=7, conductivity ratio=25, counter_flow=True, U_hot=5, U_cold=1.2, maximum 15,000 iterations and tolerance 10^{-8}.

```
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# 1. Grid
# ------------------------------------------------------------

Lx, Ly = 8.0, 4.0

nx = 160          # nodes along the length
ny_cold = 20      # nodes in cold brine channel
ny_hot = 20       # nodes in hot brine channel
ny_wall = 6       # nodes in copper wall

ny = ny_cold + ny_wall + ny_hot

# Materials
cold_fluid = "Brine"
hot_fluid = "Brine"
wall = "Copper"

if nx < 10 or min(ny_cold, ny_hot) < 3 or ny_wall < 2:
    raise ValueError(
        "Grid is too coarse. Use nx >= 10, "
        "ny_cold/ny_hot >= 3 and ny_wall >= 2."
    )

x = np.linspace(0, Lx, nx)
y = np.linspace(0, Ly, ny)

X, Y = np.meshgrid(x, y)

dx = x[1] - x[0]
dy = y[1] - y[0]
```

```
# 0 = cold brine, 1 = copper wall, 2 = hot brine
zone = np.zeros((ny, nx), dtype=int)
zone[ny_cold:ny_cold + ny_wall, :] = 1
zone[ny_cold + ny_wall:, :] = 2

print(f"Grid = {nx} x {ny} = {nx * ny} nodes")
print(f"dx = {dx:.4f}, dy = {dy:.4f}")
print(f"Cold fluid = {cold_fluid}")
print(f"Hot fluid = {hot_fluid}")
print(f"Wall = {wall}")


# ------------------------------------------------------------
# 2. Flow and heat transfer settings
# ------------------------------------------------------------

Re_hot = 500
Re_cold = 250

Pr = 7.0

# Copper conducts heat much better than brine
k_ratio = 25

# True means hot and cold brine flow in opposite directions
counter_flow = True

T_hot_in = 2
T_cold_in = -8.0

U_hot = 5.0
U_cold = 1.2

max_iter = 15000
tol = 1e-8
dt_factor = 0.4


# ------------------------------------------------------------
# 3. Velocity profile
# ------------------------------------------------------------

def velocity_profile(ny_cold, ny_wall, ny_hot,
                     nx, U_cold, U_hot, counter_flow):

    ny = ny_cold + ny_wall + ny_hot

    u = np.zeros((ny, nx))

    # Position across each channel
    eta_cold = np.linspace(0, 1, ny_cold)
    eta_hot = np.linspace(0, 1, ny_hot)

    # Simple parabolic velocity profile
    profile_cold = 6 * eta_cold * (1 - eta_cold)
    profile_hot = 6 * eta_hot * (1 - eta_hot)

    # Cold brine flows opposite to hot brine
    cold_sign = -1.0 if counter_flow else 1.0

    u[:ny_cold, :] = (
        cold_sign * U_cold * profile_cold[:, None]
    )

    # Copper wall has zero velocity
    u[ny_cold + ny_wall:, :] = (
        U_hot * profile_hot[:, None]
    )

    return u


u = velocity_profile(
    ny_cold,
    ny_wall,
    ny_hot,
    nx,
    U_cold,
    U_hot,
    counter_flow
)


# ------------------------------------------------------------
# 4. Thermal diffusivity
# ------------------------------------------------------------

D_cold = 1 / (Re_cold * Pr)
D_hot = 1 / (Re_hot * Pr)

# Give the copper wall higher thermal conductivity
D_wall = k_ratio * 0.5 * (D_hot + D_cold)

D = np.where(
    zone == 0,
    D_cold,
    np.where(zone == 1, D_wall, D_hot)
)
```

```
print(f"D_cold = {D_cold:.5f}")
print(f"D_hot  = {D_hot:.5f}")
print(f"D_wall = {D_wall:.5f}")


# ------------------------------------------------------------
# 5. Check velocity and geometry
# ------------------------------------------------------------

fig, ax = plt.subplots(1, 2, figsize=(11, 3.2))

ax[0].plot(u[:, 0], y)
ax[0].axhline(y[ny_cold], color='k', lw=0.8)
ax[0].axhline(y[ny_cold + ny_wall], color='k', lw=0.8)

ax[0].set_xlabel("Velocity")
ax[0].set_ylabel("y")
ax[0].set_title("Velocity profile")
ax[0].grid(True)

ax[1].contourf(
    X, Y, zone,
    levels=[-0.5, 0.5, 1.5, 2.5],
    colors=['#5da5da', '#b0b0b0', '#f17cb0']
)

ax[1].set_xlabel("x")
ax[1].set_ylabel("y")
ax[1].set_title("Cold brine / Copper / Hot brine")
ax[1].set_aspect("equal")

plt.tight_layout()
plt.show()


# ------------------------------------------------------------
# 6. Boundary conditions
# ------------------------------------------------------------

def apply_bc(T):

    # Hot brine enters from the left
    T[ny_cold + ny_wall:, 0] = T_hot_in

    # Hot brine outlet
    T[ny_cold + ny_wall:, -1] = \
        T[ny_cold + ny_wall:, -2]

    if counter_flow:

        # Cold brine enters from the right
        T[:ny_cold, -1] = T_cold_in

        # Cold brine leaves from the left
        T[:ny_cold, 0] = T[:ny_cold, 1]

    else:

        # Parallel flow
        T[:ny_cold, 0] = T_cold_in
        T[:ny_cold, -1] = T[:ny_cold, -2]

    # No heat loss from the ends of the copper wall
    T[ny_cold:ny_cold + ny_wall, 0] = \
        T[ny_cold:ny_cold + ny_wall, 1]

    T[ny_cold:ny_cold + ny_wall, -1] = \
        T[ny_cold:ny_cold + ny_wall, -2]

    # Outer surfaces are insulated
    T[0, :] = T[1, :]
    T[-1, :] = T[-2, :]

    return T


# ------------------------------------------------------------
# 7. One FDM step
# ------------------------------------------------------------

def fdm_step(T, u, D, dx, dy, dt):

    Tc = T[1:-1, 1:-1]

    Te = T[1:-1, 2:]
    Tw = T[1:-1, :-2]

    Tn = T[2:, 1:-1]
    Ts = T[:-2, 1:-1]

    Dc = D[1:-1, 1:-1]

    # Harmonic mean at material boundaries
    De = 2 * Dc * D[1:-1, 2:] / (
        Dc + D[1:-1, 2:]
    )

    Dw = 2 * Dc * D[1:-1, :-2] / (
        Dc + D[1:-1, :-2]
```

```
    )

    Dn = 2 * Dc * D[2:, 1:-1] / (
        Dc + D[2:, 1:-1]
    )

    Ds = 2 * Dc * D[:-2, 1:-1] / (
        Dc + D[:-2, 1:-1]
    )

    # Heat diffusion
    diffusion_x = (
        De * (Te - Tc) -
        Dw * (Tc - Tw)
    ) / dx**2

    diffusion_y = (
        Dn * (Tn - Tc) -
        Ds * (Tc - Ts)
    ) / dy**2

    # Heat convection
    uc = u[1:-1, 1:-1]

    dTdx = np.where(
        uc >= 0,
        (Tc - Tw) / dx,
        (Te - Tc) / dx
    )

    T_new = T.copy()

    T_new[1:-1, 1:-1] = (
        Tc
        + dt * (
            diffusion_x
            + diffusion_y
            - uc * dTdx
        )
    )

    return T_new


# ------------------------------------------------------------
# 8. Stable time step
# ------------------------------------------------------------

def get_dt(D, u, dx, dy):

    # Diffusion limit
    dt_diff = 1 / (
        2 * D.max() *
        (1 / dx**2 + 1 / dy**2)
    )

    # CFL limit
    u_max = np.abs(u).max()

    if u_max > 0:
        dt_cfl = dx / u_max
    else:
        dt_cfl = np.inf

    return dt_factor * min(dt_diff, dt_cfl)


# ------------------------------------------------------------
# 9. Initial temperature
# ------------------------------------------------------------

T = np.full(
    (ny, nx),
    0.5 * (T_hot_in + T_cold_in)
)

# Cold brine starts cold
T[:ny_cold, :] = T_cold_in

# Hot brine starts hot
T[ny_cold + ny_wall:, :] = T_hot_in

# Initial temperature through copper wall
T[ny_cold:ny_cold + ny_wall, :] = np.linspace(
    T_cold_in,
    T_hot_in,
    ny_wall
)[:, None]

T = apply_bc(T)


# ------------------------------------------------------------
# 10. Solve
# ------------------------------------------------------------

dt = get_dt(D, u, dx, dy)

residuals = []
```

```
for it in range(1, max_iter + 1):

    T_old = T.copy()

    T = fdm_step(
        T, u, D, dx, dy, dt
    )

    T = apply_bc(T)

    residual = np.max(
        np.abs(T - T_old)
    )

    residuals.append(residual)

    if not np.all(np.isfinite(T)):
        raise FloatingPointError(
            f"Solution diverged at iteration {it}"
        )

    if it == 1 or it % 2000 == 0:
        print(
            f"iteration {it:6d}   "
            f"residual = {residual:.3e}"
        )

    if residual < tol and it > 50:
        break

print(
    f"\nStopped after {it} iterations"
)
print(
    f"dt = {dt:.5f}"
)
print(
    f"Final residual = {residual:.3e}"
)


# ------------------------------------------------------------
# 11. Bulk temperature
# ------------------------------------------------------------

def bulk_temp(T_slice, u_slice):

    # Velocity-weighted average temperature
    weights = np.abs(u_slice) + 1e-12

    return np.average(
        T_slice,
        axis=0,
        weights=weights
    )


hot_rows = slice(
    ny_cold + ny_wall,
    ny
)

cold_in_col, cold_out_col = (
    (-1, 0)
    if counter_flow
    else (0, -1)
)

hot_bulk = bulk_temp(
    T[hot_rows, :],
    u[hot_rows, :]
)

cold_bulk = bulk_temp(
    T[:ny_cold, :],
    u[:ny_cold, :]
)


# ------------------------------------------------------------
# 12. Temperature plots
# ------------------------------------------------------------

fig, ax = plt.subplots(
    2, 1,
    figsize=(10, 6),
    height_ratios=[2, 1.3]
)

cf = ax[0].contourf(
    X, Y, T,
    levels=30,
    cmap='inferno'
)

ax[0].contour(
    X, Y, zone,
    levels=[0.5, 1.5],
```

```
    colors='cyan',
    linewidths=0.8
)

plt.colorbar(
    cf,
    ax=ax[0],
    label='Temperature'
)

ax[0].set_xlabel("x")
ax[0].set_ylabel("y")
ax[0].set_title(
    "Temperature field"
)

ax[0].set_aspect("equal")


ax[1].plot(
    x,
    hot_bulk,
    label="Hot brine"
)

ax[1].plot(
    x,
    cold_bulk,
    label="Cold brine"
)

ax[1].set_xlabel("x")
ax[1].set_ylabel("Temperature")

ax[1].set_title(
    "Bulk temperature along the heat exchanger"
)

ax[1].legend()
ax[1].grid(True, alpha=0.4)

plt.tight_layout()
plt.show()


# ------------------------------------------------------------
# 13. Heat transfer calculation
# ------------------------------------------------------------

hot_in_T = float(
    bulk_temp(
        T[hot_rows, 0],
        u[hot_rows, 0]
    )
)

hot_out_T = float(
    bulk_temp(
        T[hot_rows, -1],
        u[hot_rows, -1]
    )
)

cold_in_T = float(
    bulk_temp(
        T[:ny_cold, cold_in_col],
        u[:ny_cold, cold_in_col]
    )
)

cold_out_T = float(
    bulk_temp(
        T[:ny_cold, cold_out_col],
        u[:ny_cold, cold_out_col]
    )
)


# Simple trapezoidal integration
def integrate(values, coordinates):

    values = np.asarray(values, dtype=float)
    coordinates = np.asarray(coordinates, dtype=float)

    return float(
        np.sum(
            0.5 *
            (values[1:] + values[:-1]) *
            np.diff(coordinates)
        )
    )


mdot_hot = integrate(
    np.maximum(u[hot_rows, 0], 0),
    y[hot_rows]
)

mdot_cold = integrate(
```

```
    np.abs(u[:ny_cold, cold_in_col]),
    y[:ny_cold]
)


# Heat lost by hot brine
q_hot = mdot_hot * (
    hot_in_T - hot_out_T
)

# Heat gained by cold brine
q_cold = mdot_cold * (
    cold_out_T - cold_in_T
)

imbalance = abs(
    q_hot - q_cold
) / max(
    abs(q_hot),
    abs(q_cold),
    1e-30
)


# Maximum possible heat transfer
q_max = min(
    mdot_hot,
    mdot_cold
) * (
    T_hot_in - T_cold_in
)

effectiveness = (
    q_cold / max(q_max, 1e-30)
)


print("\n--- Heat exchanger results ---")

print(
    f"Hot brine   : {hot_in_T:.4f} -> {hot_out_T:.4f}"
)

print(
    f"Cold brine  : {cold_in_T:.4f} -> {cold_out_T:.4f}"
)

print(
    f"Heat lost by hot brine  = {q_hot:.5f}"
)

print(
    f"Heat gained by cold brine = {q_cold:.5f}"
)

print(
    f"Energy imbalance = {imbalance:.2e}"
)

print(
    f"Effectiveness = {effectiveness:.4f}"
)


# ------------------------------------------------------------
# 14. Save result for ParaView
# ------------------------------------------------------------

def write_vtk(path, x, y, T, zone):
    nx = len(x)
    ny = len(y)

    with open(path, 'w') as f:
        f.write("# vtk DataFile Version 3.0\n")
        f.write("Brine-Copper heat exchanger\nASCII\n")
        f.write("DATASET STRUCTURED_GRID\n")
        f.write(f"DIMENSIONS {nx} {ny} 1\n")
        f.write(f"POINTS {nx * ny} float\n")

        for j in range(ny):
            for i in range(nx):
                f.write(f"{x[i]:.6e} {y[j]:.6e} 0\n")

        f.write(f"POINT_DATA {nx * ny}\n")
        f.write("SCALARS temperature float 1\nLOOKUP_TABLE default\n")

        for j in range(ny):
            for i in range(nx):
                f.write(f"{T[j, i]:.6e}\n")

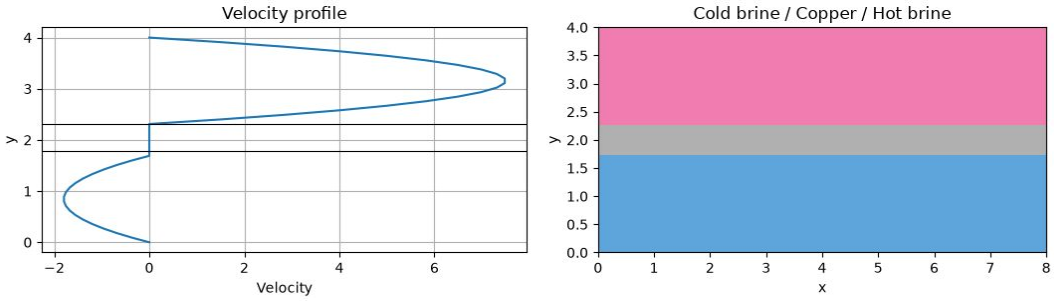
        f.write("SCALARS zone float 1\nLOOKUP_TABLE default\n")

        for j in range(ny):
            for i in range(nx):
                f.write(f"{zone[j, i]:.1f}\n")

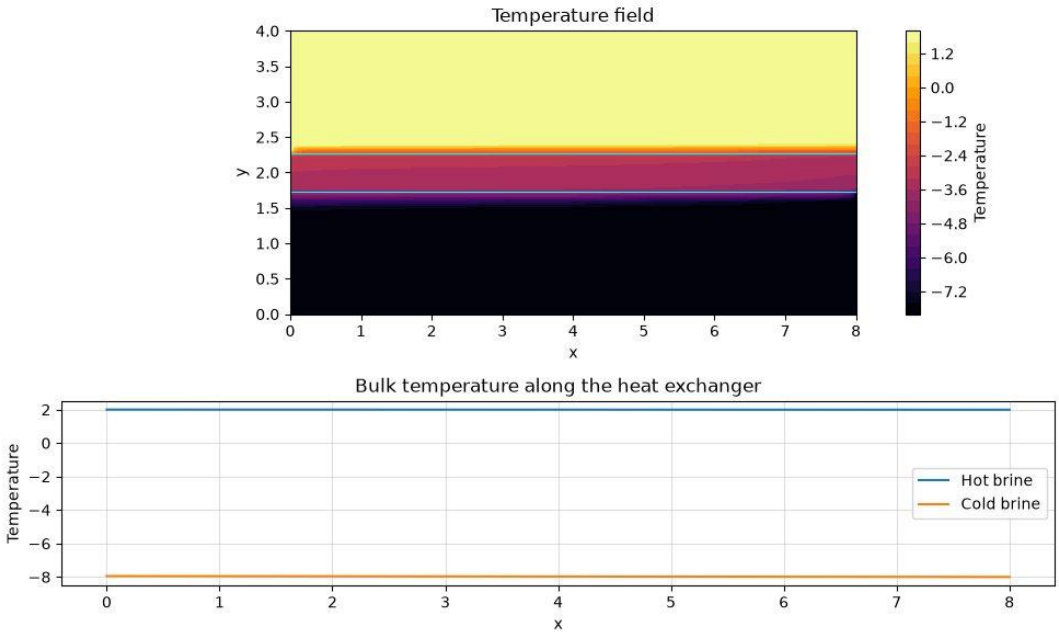

write_vtk("double_pipe_cht.vtk", x, y, T, zone)
print("Saved: double_pipe_cht.vtk")
```

Output

The following figures are reproduced from the output folder supplied for this experiment.



Experiment 2 output: Figure_1



Experiment 2 output: Figure_2

Results and Discussion

The saved geometry plot clearly distinguishes the cold-brine, copper and hot-brine regions. The temperature field shows a strong temperature transition across the separating wall. In the saved bulk-temperature plot, the hot and cold bulk-temperature curves remain almost horizontal near their inlet values. This indicates that the saved output does not show a substantial bulk-temperature change over the exchanger length. The result should therefore be reported as the computed output, with this limitation explicitly noted rather than claiming high heat-exchanger effectiveness.

Conclusion

The model demonstrates the coupled layout of two flowing streams and a conducting partition, and generates a temperature field. The saved bulk-temperature curves show little streamwise temperature change; the heat-transfer prediction should be checked further by reviewing the time history, interface coupling and energy balance.

EXPERIMENT 3 – Flow past an immersed circular cylinder

Aim

To simulate two-dimensional incompressible flow around a circular cylinder and examine the time variation of lift coefficient and the downstream vortex street.

Objective

- Create a circular solid mask inside a rectangular flow domain.
- Solve the momentum equations using upwind convection, viscous diffusion and pressure correction.
- Represent the cylinder boundary using a penalty method.
- Calculate lift coefficient, vorticity and the shedding frequency/Strouhal number when periodic lift oscillations are present.

Governing Equation

Incompressible Navier–Stokes equations (dimensionless):

$\partial u/\partial x + \partial v/\partial y = 0$

$\partial u/\partial t + u\partial u/\partial x + v\partial u/\partial y = -\partial p/\partial x + (1/Re)\nabla^2 u + penalty_u$

$\partial v/\partial t + u\partial v/\partial x + v\partial v/\partial y = -\partial p/\partial y + (1/Re)\nabla^2 v + penalty_v$

A pressure-correction equation is formed from the divergence of the predicted velocity: $\nabla^2 p' = (\nabla \cdot u^*)/\Delta t$.

Boundary Condition

- Inlet: u=1, v=0.
- Outlet: zero-gradient u and v; pressure reference p=0.
- Top and bottom: symmetry/free-slip-type conditions, with v=0 and streamwise velocity copied from the adjacent row.
- Cylinder: immersed solid mask with penalty forcing that damps velocity inside the body.

Code

Inputs: D=1, domain 25D × 5D, grid 500×100, cylinder radius 0.5D, Re=200, inlet velocity=1, Courant number=0.1, 25,000 time steps, 50 pressure-correction sweeps and pressure relaxation αp=0.3.

```
import numpy as np
from matplotlib import pyplot as plt

D = 1                   # length scale = cylinder diameter
Lx = 25
Ly = 5
nx = 500
ny = 100
dx = Lx/nx              # 0.05 D
dy = Ly/ny
Cx = Lx/3
Cy = Ly/2
Cr = 0.5*D              # radius = D/2
nit = 25000
Co = 0.1
Re = 200
u_inlet = 1             # U, nondimensional
dt = Co * min(dx, dy) / u_inlet     # 0.0125 (in units of D/U)
eta = max(0.5 * dt, 1e-5)           # nondimensional penalty parameter
alpha_p = 0.3

def apply_boundary(u, v, p):
    # left
    u[:,0] = 1
    v[:,0]=0
    p[:,0] = p[:,1]
    # right
    u[:,-1] = u[:,-2]
    v[:,-1] = v[:,-2]
    p[:,-1] = 0
    # top symmetry
    u[0,:] = u[1,:]
    v[0,:] = 0
    p[0,:] = p[1,:]
    # bottom symmetry
    u[-1,:] = u[-2,:]
```

```
    v[-1,:] = 0
    p[-1,:] = p[-2,:]

alpha = np.zeros((ny, nx))
# draw cylinder
# alpha is 1 inside cylinder, 0 outside it in fluid region
x = np.linspace(0, (nx - 1) * dx, nx)
y = np.linspace(0, (ny - 1) * dy, ny)
X, Y = np.meshgrid(x, y)
mask = (X - Cx)**2 + (Y - Cy)**2 < Cr**2
alpha[mask] = 1
# mask plot
# plt.figure(figsize=(10, 2))
# plt.imshow(alpha, extent=[0, Lx, 0, Ly], origin='lower', cmap='gray')
# plt.title('Cylinder immersed in fluid')
# plt.xlabel('x')
# plt.ylabel('y')
# plt.show()

# initial conditions
u = np.zeros((ny, nx))
v = np.zeros((ny, nx))
p = np.zeros((ny, nx))
pc = np.zeros((ny, nx))
u_old = u.copy()
v_old = v.copy()
p_old = p.copy()

Cl = np.zeros(nit)

for n in range(nit):
    apply_boundary(u, v, p)
    u_old = u.copy()
    v_old = v.copy()
    p_old = p.copy()

    # 1. u*: upwind convection + (1/Re) diffusion - grad p, penalty implicit
    u[1:-1,1:-1] = (u_old[1:-1,1:-1] + dt*(
        - np.maximum(u_old[1:-1,1:-1],0)*(u_old[1:-1,1:-1] - u_old[1:-1,:-2])/dx
        + np.minimum(u_old[1:-1,1:-1],0)*(u_old[1:-1,1:-1] - u_old[1:-1,2:])/dx
        - np.maximum(v_old[1:-1,1:-1],0)*(u_old[1:-1,1:-1] - u_old[:-2,1:-1])/dy
        + np.minimum(v_old[1:-1,1:-1],0)*(u_old[1:-1,1:-1] - u_old[2:,1:-1])/dy
        + (1/Re)*((u_old[1:-1,2:] - 2*u_old[1:-1,1:-1] + u_old[1:-1,:-2])/dx**2
                + (u_old[2:,1:-1] - 2*u_old[1:-1,1:-1] + u_old[:-2,1:-1])/dy**2)
        - (p[1:-1,2:] - p[1:-1,:-2])/(2*dx)
    )) / (1 + dt*alpha[1:-1,1:-1]/eta)
    # 2. v*
    v[1:-1,1:-1] = (v_old[1:-1,1:-1] + dt*(
        - np.maximum(u_old[1:-1,1:-1],0)*(v_old[1:-1,1:-1] - v_old[1:-1,:-2])/dx
        + np.minimum(u_old[1:-1,1:-1],0)*(v_old[1:-1,1:-1] - v_old[1:-1,2:])/dx
        - np.maximum(v_old[1:-1,1:-1],0)*(v_old[1:-1,1:-1] - v_old[:-2,1:-1])/dy
        + np.minimum(v_old[1:-1,1:-1],0)*(v_old[1:-1,1:-1] - v_old[2:,1:-1])/dy
        + (1/Re)*((v_old[1:-1,2:] - 2*v_old[1:-1,1:-1] + v_old[1:-1,:-2])/dx**2
                + (v_old[2:,1:-1] - 2*v_old[1:-1,1:-1] + v_old[:-2,1:-1])/dy**2)
        - (p[2:,1:-1] - p[:-2,1:-1])/(2*dy)
    )) / (1 + dt*alpha[1:-1,1:-1]/eta)
    # 3. pressure correction: laplacian(p') = div(u*)/dt
    div = np.zeros((ny, nx))
    div[1:-1,1:-1] = (u[1:-1,2:] - u[1:-1,:-2])/(2*dx) + (v[2:,1:-1] - v[:-2,1:-1])/(2*dy)
    pc[:,:] = 0
    for k in range(50):
        pc[:,0] = pc[:,1]; pc[0,:] = pc[1,:]; pc[-1,:] = pc[-2,:]; pc[:,-1] = 0
        pc[1:-1,1:-1] = ((pc[1:-1,2:] + pc[1:-1,:-2])*dy**2 + (pc[2:,1:-1] + pc[:-2,1:-1])*dx**2
                         - dx**2*dy**2*div[1:-1,1:-1]/dt) / (2*(dx**2 + dy**2))
    # 4. correct velocity and pressure
    u[1:-1,1:-1] = u[1:-1,1:-1] - dt*(pc[1:-1,2:] - pc[1:-1,:-2])/(2*dx)/(1 + dt*alpha[1:-1,1:-1]/eta)
    v[1:-1,1:-1] = v[1:-1,1:-1] - dt*(pc[2:,1:-1] - pc[:-2,1:-1])/(2*dy)/(1 + dt*alpha[1:-1,1:-1]/eta)
    p = p_old + alpha_p*pc

    Fy = np.sum(alpha*v)/eta*dx*dy
    Cl[n] = 2*Fy/(u_inlet**2*D) # force on the fluid by the penalty term, sign flipped for the body below

    if n%500 == 0:
        print(f"Still solving n={n}")

plt.figure(figsize=(10, 2))
plt.plot(np.linspace(dt, dt*nit, nit), Cl)
plt.ylabel('Cl')
plt.title('Cylinder immersed in fluid')
plt.xlabel('nt')
plt.show()

# ---------- vorticity ----------
w = np.gradient(v, dx, axis=1) - np.gradient(u, dy, axis=0) # dv/dx - du/dy
w[mask] = np.nan # hide cylinder interior
plt.figure(figsize=(12, 3))
plt.imshow(w, origin='lower', extent=[0, Lx, 0, Ly], cmap='seismic', vmin=-2, vmax=2)
plt.colorbar(label='vorticity')
plt.title(f'Vorticity at t = {nit*dt:.1f}')
plt.xlabel('x / D')
plt.ylabel('y / D')
plt.show()

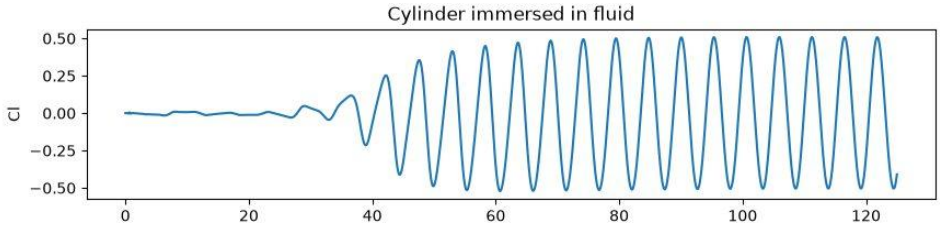
# ---------- Strouhal number from Cl ----------
# St = f*D/U ; time is already scaled by D/U, so St = f of the Cl oscillation
start = nit//2                              # discard start-up transient
cl = Cl[start:] - np.mean(Cl[start:])
print("Cl amplitude (last half):", np.max(np.abs(cl)))
```

```
if np.max(np.abs(cl)) > 1e-3:               # only if Cl actually oscillates
    npad = 16*len(cl)                       # zero-padding -> finer frequency grid
    freq = np.fft.rfftfreq(npad, d=dt)
    amp = np.abs(np.fft.rfft(cl, n=npad))
    f = freq[np.argmax(amp[1:]) + 1]        # skip the zero-frequency bin
    St = f*D/u_inlet
    print(f"Shedding frequency f = {f:.4f}, St = {St:.4f}  (literature ~0.165 at Re = 200)")

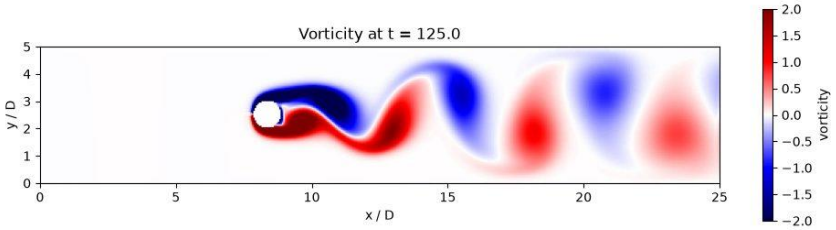
    plt.figure(figsize=(6, 3))
    plt.plot(freq, amp)
    plt.xlim(0, 1)
    plt.xlabel('frequency (U/D)')
    plt.ylabel('|FFT(Cl)|')
    plt.title('Cl spectrum, peak = St')
    plt.show()
else:
    print("Cl is flat: no shedding developed, so St cannot be computed.")
```

Output

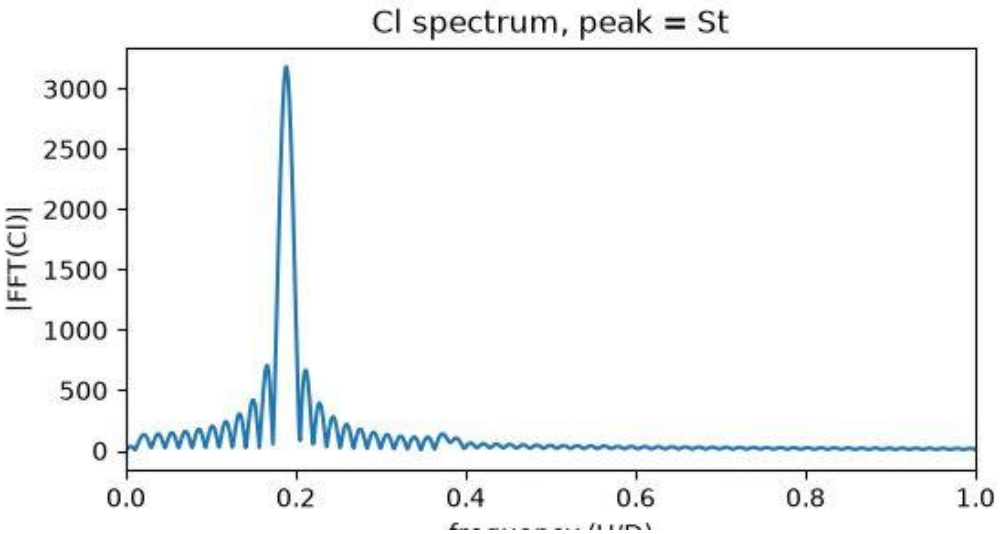
The following figures are reproduced from the output folder supplied for this experiment.



Experiment 3 output: Figure_1



Experiment 3 output: Figure_2



Experiment 3 output: Figure_3

Results and Discussion

The lift-coefficient history begins with a small transient and develops into a sustained oscillation. The vorticity field shows alternating positive and negative vortical structures downstream of the cylinder, consistent with a von Kármán-type wake. The third saved figure contains the lift-spectrum/Strouhal analysis when the lift signal satisfies the oscillation threshold.

Conclusion

The simulation captures unsteady vortex shedding behind a circular cylinder at Re=200. The alternating wake and periodic lift response are the principal observed features. The computed Strouhal value should be quoted from the program output if required; it is not inferred from the figure alone.

EXPERIMENT 4 – Radial rotating annular passage: pressure and heat transfer

Aim

To calculate pressure, relative velocity and temperature distributions in a rotating annular passage using a radial finite-difference model.

Objective

- Define the annular computational domain in radial and circumferential coordinates.
- Prescribe radial and tangential velocity components associated with rotation.
- Calculate radial pressure rise using a finite-difference integration and compare it with the analytical expression included in the code.
- Solve the steady temperature field using an upwind convection–diffusion finite-difference matrix.

Governing Equation

Steady energy model represented by the matrix coefficients:
$\rho c_p(u_r \partial T/\partial r + u_{\theta,rel}/r \partial T/\partial \theta) = k[\partial^2 T/\partial r^2 + (1/r^2)\partial^2 T/\partial \theta^2] +$ wall/source terms

The code uses $\alpha = k/(\rho c_p)$, a wall heat-transfer coefficient term $\beta = 2h/(\rho c_p b)$, and an additional loss source. The implemented radial diffusion stencil contains the second radial derivative and angular diffusion term; it does not explicitly include the cylindrical $(1/r)\partial T/\partial r$ contribution. This is therefore a simplified radial-annular teaching discretization.

The code uses $\alpha = k/(\rho c_p)$, a wall heat-transfer coefficient term $\beta = 2h/(\rho c_p b)$, and an additional loss source. Pressure is integrated from the radial balance term $\rho u_\theta^2/r - 12\mu u_r/b^2$.

Boundary Condition

- Inlet radial boundary: prescribed temperature Tin=350 K.
- Outlet radial boundary: zero radial temperature gradient, implemented with a second-order one-sided relation.
- Circumferential direction: periodic indexing for angular neighbours.
- Wall heating is represented through the source/diagonal coefficient involving Twall=420 K and wall_htc=150; this is not a separate explicit wall boundary row.

Code

Inputs: nr=100, ntheta=180, rpm=3000, Re=1200, Tin=350 K, Twall=420 K, swirl=0.70, wall_htc=150, loss=0.05, r1=0.30 m, r2=0.120 m, b=0.010 m and the listed brine-like properties.

```
import numpy as np
import matplotlib.pyplot as plt
import math

try:
    from scipy.sparse import lil_matrix  # type: ignore[import-not-found]
    from scipy.sparse.linalg import spsolve  # type: ignore[import-not-found]
except ImportError as exc:
    raise ImportError(
        "SciPy is required for this script. Install it with: pip install scipy"
    ) from exc


# ============================================================
# 1. INPUTS
# ============================================================

nr = 100
ntheta = 180

rpm = 3000
Re = 1200
```

```
Tin = 350.0
Twall = 420

swirl = 0.70
wall_htc = 150.0
loss = 0.05

r1 = 0.30
r2 = 0.120
b = 0.010

rho = 997.0
mu = 0.0010
cp = 4180.0
k = 0.60
g = 9.81


omega = rpm * 2 * math.pi / 60

Dh = 2 * b
u_in = Re * mu / (rho * Dh)

r = np.linspace(r1, r2, nr)
theta = np.linspace(0, 2*np.pi, ntheta, endpoint=False)

dr = r[1] - r[0]
dtheta = theta[1] - theta[0]

R, TH = np.meshgrid(r, theta, indexing="ij")

print(f"Grid = {nr} x {ntheta}")
print(f"dr = {dr:.6e} m")
print(f"dtheta = {dtheta:.6e} rad")


# ============================================================
# 4. VELOCITY
# ============================================================

# Radial velocity decreases as radius increases
u_r = u_in * r1 / r

# Tangential velocity due to rotation
u_theta = swirl * omega * r

# Relative tangential velocity
w_theta = u_theta - omega * r

ur = u_r[:, None] * np.ones((1, ntheta))
ut = u_theta[:, None] * np.ones((1, ntheta))
wr = w_theta[:, None] * np.ones((1, ntheta))


# ============================================================
# 5. PRESSURE FDM
# ============================================================

P = np.zeros(nr)

pressure_term = (
    rho * u_theta**2 / r
    - 12 * mu * u_r / b**2
)

P[1:] = np.cumsum(dr * pressure_term[:-1])


# Exact solution used only to check the FDM result
P_exact = (
    0.5 * rho * swirl**2 * omega**2 * (r**2 - r1**2)
    - 12 * mu * u_in * r1 * np.log(r / r1) / b**2
)


# ============================================================
# 6. THERMAL PROPERTIES
# ============================================================

alpha = k / (rho * cp)

# Heat transfer from the wall
beta = 2 * wall_htc / (rho * cp * b)

# Small heat source due to losses
source = loss * np.abs(u_r) * (omega * r)**2 / (b * cp)


# ============================================================
# 7. CREATE FDM MATRIX
# ============================================================

N = nr * ntheta

A = lil_matrix((N, N))
B = np.zeros(N)


def node(i, j):
```

```
    return i * ntheta + j


# ============================================================
# 8. BOUNDARY CONDITIONS
# ============================================================

for j in range(ntheta):

    # Inlet temperature
    row = node(0, j)

    A[row, row] = 1
    B[row] = Tin


    # Outlet: dT/dr = 0
    row = node(nr - 1, j)

    A[row, node(nr - 1, j)] = 3
    A[row, node(nr - 2, j)] = -4
    A[row, node(nr - 3, j)] = 1


# ============================================================
# 9. FDM EQUATION INSIDE THE DOMAIN
# ============================================================

for i in range(1, nr - 1):

    ri = r[i]

    # Diffusion terms
    ar = alpha / dr**2
    at = alpha / (ri**2 * dtheta**2)

    # Local velocities
    vr = u_r[i]
    vt = w_theta[i] / ri


    # Upwind scheme for convection
    aw = ar + max(vr, 0) / dr
    ae = ar + max(-vr, 0) / dr

    ass = at + max(vt, 0) / dtheta
    an = at + max(-vt, 0) / dtheta

    ap = aw + ae + ass + an + beta


    for j in range(ntheta):

        row = node(i, j)

        # Centre node
        A[row, row] = ap

        # Radial neighbours
        A[row, node(i - 1, j)] = -aw
        A[row, node(i + 1, j)] = -ae

        # Angular neighbours
        # % ntheta makes the solution periodic
        A[row, node(i, (j - 1) % ntheta)] = -ass
        A[row, node(i, (j + 1) % ntheta)] = -an

        # Right hand side
        B[row] = beta * Twall + source[i]


# ============================================================
# 10. SOLVE TEMPERATURE FIELD
# ============================================================

A = A.tocsr()

T = spsolve(A, B).reshape(nr, ntheta)

if not np.all(np.isfinite(T)):
    raise ValueError("Temperature solution is not valid")

print("\nFDM solution completed")
print(f"Minimum temperature = {T.min():.4f} K")
print(f"Maximum temperature = {T.max():.4f} K")
print(f"Outlet temperature  = {T[-1, :].mean():.4f} K")


# ============================================================
# 11. VELOCITY IN X-Y DIRECTION
# ============================================================

x = R * np.cos(TH)
y = R * np.sin(TH)

u_abs_x = ur * np.cos(TH) - ut * np.sin(TH)
u_abs_y = ur * np.sin(TH) + ut * np.cos(TH)
```

```
u_rel_x = ur * np.cos(TH) - wr * np.sin(TH)
u_rel_y = ur * np.sin(TH) + wr * np.cos(TH)

speed = np.sqrt(u_rel_x**2 + u_rel_y**2)


# ============================================================
# 12. PLOTS
# ============================================================

fig, ax = plt.subplots(1, 3, figsize=(16, 5))


# Temperature
p = ax[0].pcolormesh(
    x, y, T,
    shading="auto",
    cmap="inferno"
)

ax[0].set_title("Temperature [K]")
ax[0].set_xlabel("x [m]")
ax[0].set_ylabel("y [m]")
ax[0].set_aspect("equal")
fig.colorbar(p, ax=ax[0])


# Pressure
ax[1].plot(r, P, "o-", label="FDM")
ax[1].plot(r, P_exact, "--", label="Exact")

ax[1].set_xlabel("Radius [m]")
ax[1].set_ylabel("Pressure [Pa]")
ax[1].set_title("Radial Pressure Rise")
ax[1].grid(True)
ax[1].legend()


# Relative velocity
p = ax[2].pcolormesh(
    x, y, speed,
    shading="auto"
)

ax[2].set_title("Relative Velocity")
ax[2].set_xlabel("x [m]")
ax[2].set_ylabel("y [m]")
ax[2].set_aspect("equal")
fig.colorbar(p, ax=ax[2])


plt.tight_layout()
plt.show()


# ============================================================
# 13. VELOCITY VECTORS
# ============================================================

skip_r = max(1, nr // 10)
skip_t = max(1, ntheta // 16)

plt.figure(figsize=(7, 6))

plt.pcolormesh(
    x, y, speed,
    shading="auto"
)

plt.quiver(
    x[::skip_r, ::skip_t],
    y[::skip_r, ::skip_t],
    u_rel_x[::skip_r, ::skip_t],
    u_rel_y[::skip_r, ::skip_t],
    color="white",
    scale=20
)

plt.xlabel("x [m]")
plt.ylabel("y [m]")
plt.title("Relative Velocity Vectors")
plt.axis("equal")
plt.colorbar(label="Velocity [m/s]")
plt.show()


# ============================================================
# 14. CHECK THE FDM RESULT
# ============================================================

# Inlet temperature error
inlet_error = np.max(
    np.abs(T[0, :] - Tin)
)


# Outlet dT/dr should be approximately zero
outlet_gradient = np.max(
    np.abs(
```

```
        (3*T[-1, :] - 4*T[-2, :] + T[-3, :])
        / (2 * dr)
    )
)


# Temperature should be periodic in theta
periodic_error = np.max(
    np.abs(T[:, 0] - T[:, -1])
)


# Difference between FDM and exact pressure
pressure_error = np.max(
    np.abs(P - P_exact)
)


# ============================================================
# 15. FINAL RESULTS
# ============================================================

mass_flow = rho * u_in * (2 * math.pi * r1 * b)

pump_head = (
    P[-1] - P[0]
) / (rho * g)

heat_gain = (
    mass_flow * cp *
    (T[-1, :].mean() - Tin)
)


print("\n========== FDM CHECK ==========")

print(f"Inlet temperature error       : {inlet_error:.6e} K")
print(f"Outlet Neumann error          : {outlet_gradient:.6e} K/m")
print(f"Angular periodicity error     : {periodic_error:.6e} K")
print(f"Pressure FDM error             : {pressure_error:.6e} Pa")

print("\n========== PUMP RESULTS ==========")

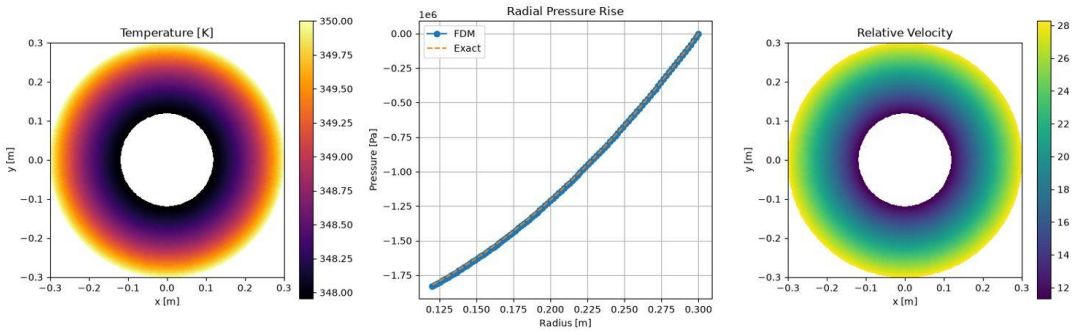
print(f"Pressure rise                 : {P[-1] - P[0]:.4f} Pa")
print(f"Pump head                     : {pump_head:.4f} m")
print(f"Mass flow rate                : {mass_flow:.6f} kg/s")
print(f"Heat gained by fluid          : {heat_gain:.4f} W")

print("\n========== TEMPERATURE ==========")

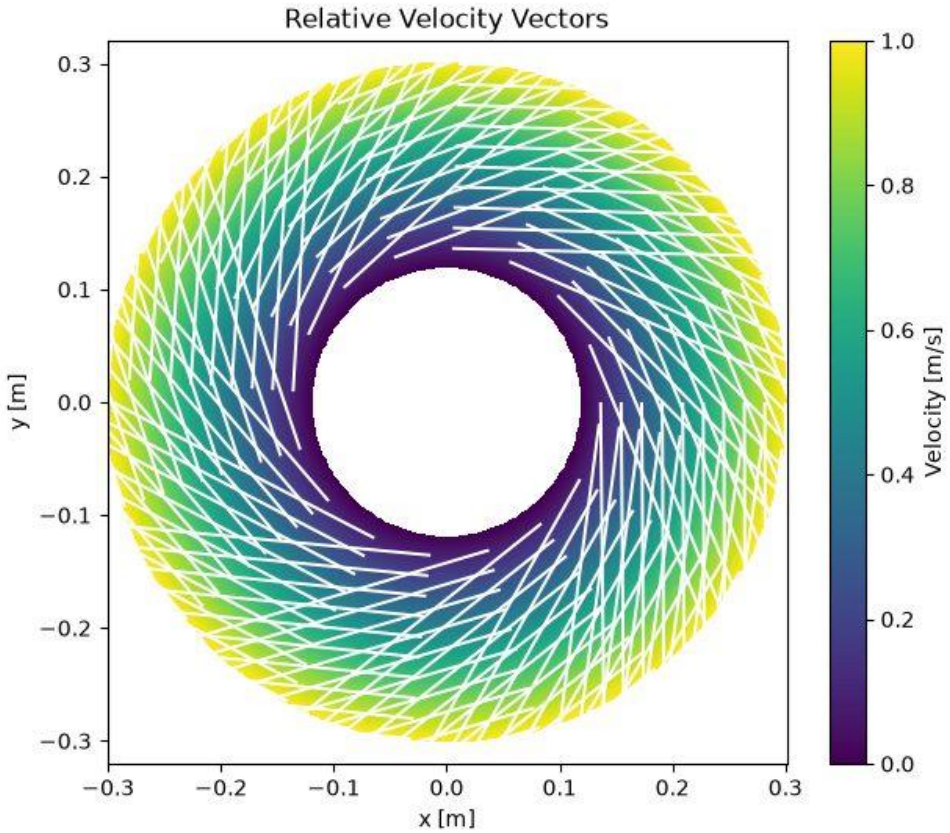
print(f"Inlet temperature             : {Tin:.2f} K")
print(f"Wall temperature              : {Twall:.2f} K")
print(f"Outlet temperature            : {T[-1, :].mean():.2f} K")
print(f"Temperature range             : {T.min():.2f} - {T.max():.2f} K")
```

Output

The following figures are reproduced from the output folder supplied for this experiment.



Experiment 4 output: Figure_1



Experiment 4 output: Figure_2

Results and Discussion

The saved temperature contour shows a circumferentially symmetric annular temperature distribution, with warmer values toward the outer radius. The radial pressure curve from FDM closely follows the analytical reference in the saved plot. The relative-velocity contour shows a radial variation across the annular gap. The model is a prescribed-flow educational approximation; it does not solve a fully coupled rotating-fluid momentum and energy system.

Conclusion

The numerical pressure integration agrees closely with the analytical pressure curve in the supplied figure, and the temperature/relative-velocity fields are successfully generated. The conclusions are limited to the prescribed kinematic and thermal assumptions of the model.

EXPERIMENT 5 – Couette flow

Aim

To obtain the steady velocity distribution between two parallel plates, one stationary and the other moving, using explicit and implicit finite-difference methods.

Objective

- Model one-dimensional momentum diffusion between parallel plates.
- Apply stationary and moving-wall velocity boundary conditions.
- Compute the steady profile using explicit and implicit time-marching schemes.
- Compare the convergence behaviour and final velocity profiles.

Governing Equation

For fully developed, unidirectional Couette flow with no pressure gradient:

$\partial u/\partial t = \nu \, \partial^2 u/\partial y^2$

At steady state: $d^2u/dy^2=0$, giving a linear velocity profile. The code advances the diffusion equation using explicit and implicit finite-difference updates.

Boundary Condition

- Lower plate: u(0)=0.
- Upper plate: u(H)=U, with U=1.5.
- Initial interior velocity: zero.
- No pressure-gradient or streamwise-convection term is included.

Code

Inputs: ny=100, H=1, U=1.5, T=1, Re=200, F=0.3, explicit time step computed from F and grid spacing; implicit time step=5; convergence threshold=10^{-6}.

```
import numpy as np
from matplotlib import pyplot as plt

ny = 100
H = 1
U = 1.5
T = 1
Re = 200
dy = H / ny
nit = 10000000
F = 0.3
dt = F * dy**2 * Re / (T*U)

def apply_boundary(u):
    u[0] = 0
    u[-1] = U

# explicit solution
# initialization
u = np.zeros(ny)
u_old = u.copy()

def convergence_criteria(u, u_old):
    return np.max(np.abs(u - u_old))

for n in range(nit):
    apply_boundary(u)

    d2u_dy2 = (u_old[2:] + u_old[0:-2] - 2*u_old[1:-1])/(dy**2)
    non_dimensional = (1/Re)*(T*U/H)

    u[1:-1] = u_old[1:-1] + dt * (non_dimensional * d2u_dy2)

    if convergence_criteria(u, u_old) < 1e-6:
        print(f'Explicit converged after {n} iterations, dt = {dt}')
        exp_conv = n
        exp_dt = dt
        break

    u_old = u.copy()
```

```
u_explicit = u.copy()

# implicit solution
# initialization
u = np.zeros(ny)
u_old = u.copy()
dt = 5
for n in range(nit):
    apply_boundary(u)

    non_dimensional = (1/Re)*(T*U/H)
    u[1:-1] = ((non_dimensional/dy)*(u[2:]+u[0:-2]) + (dy/dt)*u_old[1:-1])/((2*non_dimensional/dy) + (dy/dt)) # Patankar

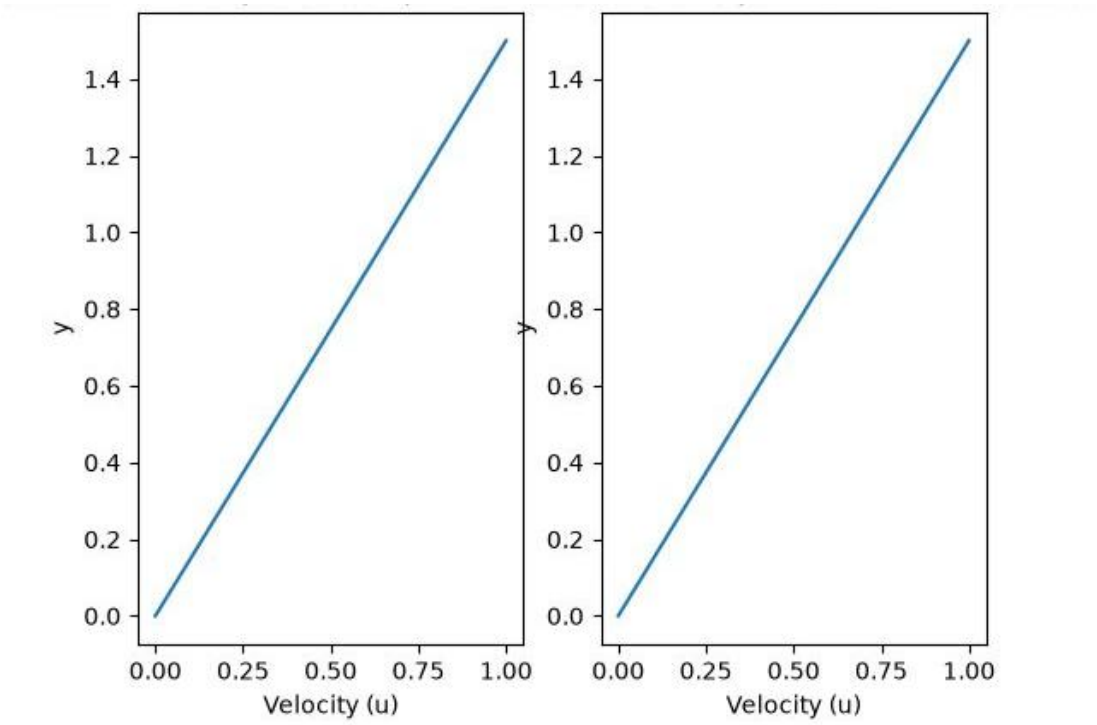
    if convergence_criteria(u, u_old) < 1e-6:
        print(f'Implicit converged after {n} iterations, dt = {dt}')
        imp_conv = n
        imp_dt = dt
        break

    u_old = u.copy()
u_implicit = u.copy()

# plot results
fig, ax = plt.subplots(1, 2)
ax[0].plot(np.linspace(0, H, ny), u_explicit, label='Explicit Solution')
ax[0].set_xlabel('Velocity (u)')
ax[0].set_ylabel('y')
ax[0].set_title(f'Explicit Solution - Converged after {exp_conv} iterations, dt = {exp_dt}')
ax[1].plot(np.linspace(0, H, ny), u_implicit, label='Implicit Solution')
ax[1].set_xlabel('Velocity (u)')
ax[1].set_ylabel('y')
ax[1].set_title(f'Implicit Solution - Converged after {imp_conv} iterations, dt = {imp_dt}')
plt.show()
```

Output

The following figures are reproduced from the output folder supplied for this experiment.



Experiment 5 output: Figure_1

Results and Discussion

The saved figure shows nearly linear velocity distributions for both explicit and implicit methods, which is the expected steady Couette profile. The figure titles report convergence after approximately 18,370 explicit iterations and 12,277 implicit iterations. The plotted axes/labels are slightly crowded in the saved image, but both curves connect the stationary and moving wall velocities.

Conclusion

Both explicit and implicit finite-difference schemes approach the linear steady Couette velocity profile. The implicit method reaches the stated update tolerance in fewer iterations in the saved run, although its time step differs from the explicit scheme.

EXPERIMENT 6 – Two-dimensional steady heat conduction

Aim

To solve the two-dimensional heat-conduction equation in a square domain and compare explicit and implicit finite-difference solutions.

Objective

- Discretise a square plate using a uniform Cartesian grid.
- Apply a prescribed hot boundary, cold side boundaries and an insulated lower boundary.
- Iterate the two-dimensional diffusion equation to steady state using explicit and implicit schemes.
- Compare the temperature contours and convergence iteration counts.

Governing Equation

Transient conduction equation (constant properties, no source):

$\partial T/\partial t = \alpha(\partial^2 T/\partial x^2 + \partial^2 T/\partial y^2)$

At steady state: $\nabla^2 T=0$. The code uses a Fourier-number coefficient Fo=0.2 in the finite-difference update.

Boundary Condition

- Top boundary: T=100.
- Left and right boundaries: T=0.
- Bottom boundary: zero normal temperature gradient, implemented as T[0,:]=T[1,:].
- Initial temperature field: zero.

Code

Inputs: L=1, nx=150, Fo=0.2, maximum iterations=1,000,000 and convergence threshold=10^{-6}.

```
import numpy as np
from matplotlib import cm, pyplot as plt

L = 1
nx = 150
dx = L/nx
Fo = 0.2
nit = 1000000

def apply_boundary(T):
    T[-1, :] = 100
    T[:, 0] = 0
    T[:, -1] = 0
    T[0, :] = T[1, :]

def convergence_criteria(T, T_old):
    return np.max(np.abs(T - T_old))

def explicit_soln():
    T = np.zeros((nx, nx))
    T_old = T.copy()
    for n in range(nit):
        apply_boundary(T)

        T[1:-1,1:-1] = T_old[1:-1,1:-1] + Fo*(T_old[0:-2,1:-1] + T_old[2:,1:-1] + T_old[1:-1,0:-2] + T_old[1:-1,2:] - 4*T_old[1:-1,1:-1])

        if convergence_criteria(T, T_old) < 1e-6:
            print(f"Explicit converged at n={n}")
            break

        T_old = T.copy()
    return T, n

def implicit_soln():
    T = np.zeros((nx, nx))
    T_old = T.copy()
    for n in range(nit):
        apply_boundary(T)

        T[1:-1,1:-1] = (T_old[1:-1,1:-1] + Fo*(T[0:-2,1:-1] + T[2:,1:-1] + T[1:-1,0:-2] + T[1:-1,2:]))/(1 + 4*Fo)

        if convergence_criteria(T, T_old) < 1e-6:
            print(f"Implicit converged at n={n}")
            break
```

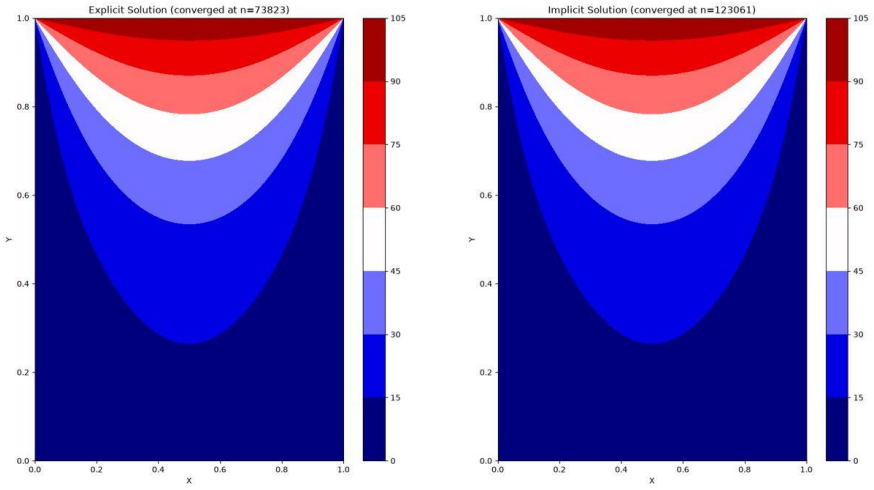
```
        T_old = T.copy()
    return T, n

exp_T, exp_n = explicit_soln()
imp_T, imp_n = implicit_soln()

x = np.linspace(0, L, nx)
y = np.linspace(0, L, nx)
X,Y = np.meshgrid(x, y)
plt.subplots(1, 2)
plt.subplot(1, 2, 1)
plt.contourf(X, Y, exp_T, cmap=cm.seismic)
plt.colorbar()
plt.title(f'Explicit Solution (converged at n={exp_n})')
plt.xlabel('X')
plt.ylabel('Y')
plt.subplot(1, 2, 2)
plt.contourf(X, Y, imp_T, cmap=cm.seismic)
plt.colorbar()
plt.title(f'Implicit Solution (converged at n={imp_n})')
plt.xlabel('X')
plt.ylabel('Y')
plt.show()
```

Output

The following figures are reproduced from the output folder supplied for this experiment.



Experiment 6 output: Figure_1

Results and Discussion

The saved contour plots for explicit and implicit schemes show the same broad steady-state pattern: temperature is highest along the heated top boundary and decreases toward the cold side boundaries and insulated bottom region. The saved figure reports convergence at n=73,823 for the explicit scheme and n=123,061 for the implicit scheme. The visual agreement is good, while the iteration counts differ.

Conclusion

The explicit and implicit methods both produce a smooth two-dimensional steady temperature field satisfying the implemented boundary conditions. In this run, the explicit scheme reaches the update tolerance in fewer iterations; iteration count alone should not be interpreted as a general stability or efficiency ranking because the update formulations differ.

EXPERIMENT 7 – Shallow-water wave propagation

Aim

To compare numerical schemes for one-dimensional shallow-water wave propagation and study the influence of the Courant–Friedrichs–Lewy (CFL) number.

Objective

- Solve the linearised shallow-water wave model using characteristic variables.
- Compare first-order Upwind and second-order Lax–Wendroff schemes with an analytical reference profile.
- Run the prescribed CFL values 0.4, 0.8 and 1.1.
- Compare stability, amplitude loss, phase error and mass conservation.

Governing Equation

Linear shallow-water equations about a constant depth H:

$\partial\eta/\partial t + H\partial u/\partial x = 0$

$\partial u/\partial t + g\partial\eta/\partial x = 0$

The characteristic wave speed is $c=\sqrt{(gH)}$. The numerical scheme advances the two characteristic variables using Upwind and Lax–Wendroff updates. The notebook clips the requested CFL to a bounded interval in its generated solver; the report should therefore distinguish requested CFL from any internally applied clipping.

Boundary Condition

- The characteristic updates use periodic array rolls at the ends of the one-dimensional domain.
- Initial free-surface disturbance is a Gaussian pulse centred at the specified location, with specified amplitude and width.
- The analytical reference is constructed from two waves travelling at $\pm\sqrt{(gH)}$.

Code

Custom parameters in the updated notebook: nx=401, depth=1.0, amplitude=0.02, x0=0.5 in the notebook parameter cell, length=10.0, t_end=2.0, output_every=25, gravity=9.81, n_snapshots=180, space_time_rows=61. The compulsory comparison is CFL=0.4, 0.8 and 1.1.

```
# ===== NOTEBOOK CODE CELL 0 =====
from pathlib import Path
import shutil
import subprocess
import sys
import zipfile
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Markdown, display

IN_COLAB = False
try:
    from google.colab import files as colab_files
    IN_COLAB = True
except ImportError:
    colab_files = None


def run_solver(script_path, arguments, output_dir, timeout=1800):
    output_dir = Path(output_dir)
    if output_dir.exists():
        shutil.rmtree(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    command = [sys.executable, str(script_path), *[str(value) for value in arguments]]
    print("Running:", " ".join(command))
    completed = subprocess.run(command, cwd=output_dir, capture_output=True, text=True, timeout=timeout)
    (output_dir / "solver_stdout.log").write_text(completed.stdout, encoding="utf-8")
    (output_dir / "solver_stderr.log").write_text(completed.stderr, encoding="utf-8")
    if completed.stdout.strip():
        print(completed.stdout.strip())
    if completed.returncode != 0:
        raise RuntimeError(completed.stderr.strip() or f"Solver exited with code {completed.returncode}")
    if completed.stderr.strip():
```

```
        print(completed.stderr.strip())
    return output_dir


def read_csv(path):
    data = np.genfromtxt(path, delimiter=",", names=True, dtype=None, encoding="utf-8")
    return np.atleast_1d(data)


def show_summary(path):
    text = Path(path).read_text(encoding="utf-8").strip()
    display(Markdown(f"### Solver summary\n```text\n{text}\n```"))
    return text


def list_outputs(output_dir):
    rows = []
    for path in sorted(Path(output_dir).iterdir()):
        if path.is_file():
            size = path.stat().st_size
            if size < 1024:
                size_text = f"{size} B"
            elif size < 1024**2:
                size_text = f"{size / 1024:.1f} KB"
            else:
                size_text = f"{size / 1024**2:.1f} MB"
            rows.append((path.name, size_text))
    table = "\n".join(f"| `{name}` | {size} |" for name, size in rows)
    display(Markdown(f"### Generated files\n| File | Size |\n|---|---:|\n{table}"))
    return rows


def read_structured_vtk(path):
    lines = Path(path).read_text(encoding="utf-8").splitlines()
    dims_index = next(i for i, line in enumerate(lines) if line.startswith("DIMENSIONS "))
    _, nx, ny, nz = lines[dims_index].split()
    nx, ny, nz = int(nx), int(ny), int(nz)
    if nz != 1:
        raise ValueError("Only 2D structured VTK files are supported by this notebook preview")
    points_index = next(i for i, line in enumerate(lines) if line.startswith("POINTS "))
    npoints = int(lines[points_index].split()[1])
    points = np.array([[float(v) for v in lines[points_index + 1 + k].split()[:3]] for k in range(npoints)])
    x = points[:, 0].reshape(ny, nx)
    y = points[:, 1].reshape(ny, nx)
    data_index = next(i for i, line in enumerate(lines) if line.startswith("POINT_DATA ")) + 1
    scalars = {}
    vectors = {}
    i = data_index
    while i < len(lines):
        if lines[i].startswith("SCALARS "):
            name = lines[i].split()[1]
            i += 2
            values = np.array([float(lines[i + k].split()[0]) for k in range(npoints)])
            scalars[name] = values.reshape(ny, nx)
            i += npoints
        elif lines[i].startswith("VECTORS "):
            name = lines[i].split()[1]
            i += 1
            values = np.array([[float(v) for v in lines[i + k].split()[:3]] for k in range(npoints)])
            vectors[name] = (values[:, 0].reshape(ny, nx), values[:, 1].reshape(ny, nx))
            i += npoints
        else:
            i += 1
    return x, y, scalars, vectors


def plot_vtk_scalar(path, preferred_field=None):
    path = Path(path)
    if not path.is_file():
        print(f"VTK preview skipped: {path.name} was not generated")
        return
    x, y, scalars, _ = read_structured_vtk(path)
    if not scalars:
        print("VTK preview skipped: no scalar field found")
        return
    field = preferred_field if preferred_field in scalars else next(iter(scalars))
    plt.figure(figsize=(8, 4.8))
    image = plt.contourf(x, y, scalars[field], levels=30)
    plt.colorbar(image, label=field.replace("_", " "))
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title(f"{field.replace('_', ' ').title()} - inline field preview")
    plt.axis("equal")
    plt.tight_layout()
    plt.show()


def make_results_zip(output_dir, experiment_number):
    output_dir = Path(output_dir)
    archive = output_dir.parent / f"experiment_{experiment_number:02d}_results.zip"
    if archive.exists():
        archive.unlink()
    with zipfile.ZipFile(archive, "w", compression=zipfile.ZIP_DEFLATED) as handle:
        for path in sorted(output_dir.rglob("*")):
            if path.is_file():
                handle.write(path, path.relative_to(output_dir))
    print(f"Results archive: {archive} ({archive.stat().st_size / 1024:.1f} KB)")
    return archive
```

```
EXPERIMENT = 7
ROOT = Path("/content/cfd_lab_exp07") if IN_COLAB else Path(r"C:\CFD_Runs\_colab_runtime_exp07")
CODE_DIR = ROOT / "code"
OUTPUT_DIR = ROOT / "outputs"
CODE_DIR.mkdir(parents=True, exist_ok=True)
(CODE_DIR / "common.py").write_text('from __future__ import annotations\n\nimport csv\nimport math\nfrom pathlib import Path\nfrom typing import Iterable, Mapping\n\nimport numpy as np\n\n\ndef trapezoidal_integral(values: np.ndarray, coordinates: np.ndarray) -> float:\n    y = np.asarray(values, dtype=float).ravel()\n    x = np.asarray(coordinates, dtype=float).ravel()\n    if y.size != x.size or y.size < 2:\n raise ValueError("trapezoidal integration requires equally sized arrays with at least two points")\n    return float(np.sum(0.5 * (y[1:] + y[:-1]) * np.diff(x)))\n\n\ndef write_csv(path: str | Path, header: Iterable[str], rows: Iterable[Iterable[object]]) -> None:\n    with Path(path).open("w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(list(header))\n writer.writerows(rows)\n\n\ndef mesh_xy(nx: int, ny: int, lx: float = 1.0, ly: float = 1.0, x0: float = 0.0, y0: float = 0.0) -> tuple[np.ndarray, np.ndarray]:\n    x = np.linspace(x0, x0 + lx, nx)\n    y = np.linspace(y0, y0 + ly, ny)\n    return np.meshgrid(x, y)\n\n\ndef write_structured_grid_vtk(\n    path: str | Path,\n    x: np.ndarray,\n    y: np.ndarray,\n    scalars: Mapping[str, np.ndarray] | None = None,\n    vectors: Mapping[str, tuple[np.ndarray, np.ndarray]] | None = None,\n    title: str = "CFD lab output",\n) -> None:\n    x = np.asarray(x, dtype=float)\n    y = np.asarray(y, dtype=float)\n    if x.shape != y.shape or x.ndim != 2:\n raise ValueError("x and y must be equally shaped 2D arrays")\n    ny, nx = x.shape\n    scalar_map = dict(scalars or {})\n    vector_map = dict(vectors or {})\n    for name, values in scalar_map.items():\n if np.asarray(values).shape != x.shape:\n raise ValueError(f"scalar {name} has shape {np.asarray(values).shape}, expected {x.shape}")\n    for name, values in vector_map.items():\n if len(values) != 2 or np.asarray(values[0]).shape != x.shape or np.asarray(values[1]).shape != x.shape:\n raise ValueError(f"vector {name} has incompatible shape")\n    with Path(path).open("w", encoding="utf-8") as handle:\n handle.write("# vtk DataFile Version 3.0\\n")\n handle.write(f"{title}\\n")\n handle.write("ASCII\\n")\n handle.write("DATASET STRUCTURED_GRID\\n")\n handle.write(f"DIMENSIONS {nx} {ny} 1\\n")\n handle.write(f"POINTS {nx * ny} float\\n")\n for j in range(ny):\n for i in range(nx):\n handle.write(f"{x[j, i]:.12e} {y[j, i]:.12e} 0\\n")\n handle.write(f"POINT_DATA {nx * ny}\\n")\n for name, values in scalar_map.items():\n data = np.asarray(values, dtype=float)\n handle.write(f"SCALARS {name} float 1\\n")\n handle.write("LOOKUP_TABLE default\\n")\n for value in data.ravel(order="C"):\n handle.write(f"{value:.12e}\\n")\n for name, values in vector_map.items():\n vx = np.asarray(values[0], dtype=float)\n vy = np.asarray(values[1], dtype=float)\n handle.write(f"VECTORS {name} float\\n")\n for ax, ay in zip(vx.ravel(order="C"), vy.ravel(order="C")):\n handle.write(f"{ax:.12e} {ay:.12e} 0\\n")\n\n\ndef write_polyline_vtk(path: str | Path, x: np.ndarray, y: np.ndarray, title: str = "polyline", close: bool = False) -> None:\n x = np.asarray(x, dtype=float).ravel()\n    y = np.asarray(y, dtype=float).ravel()\n    if x.size != y.size or x.size < 2:\n raise ValueError("polyline requires equally sized x and y arrays")\n    ids = list(range(x.size))\n    if close:\n ids.append(0)\n    with Path(path).open("w", encoding="utf-8") as handle:\n handle.write("# vtk DataFile Version 3.0\\n")\n handle.write(f"{title}\\n")\n handle.write("ASCII\\n")\n handle.write("DATASET POLYDATA\\n")\n handle.write(f"POINTS {x.size} float\\n")\n for xi, yi in zip(x, y):\n handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n handle.write(f"LINES 1 {len(ids) + 1}\\n")\n handle.write(f"{len(ids)} {\' \'.join(str(i) for i in ids)}\\n")\n\n\ndef write_multiline_vtk(path: str | Path, lines: list[tuple[np.ndarray, np.ndarray]], title: str = "lines") -> None:\n    points: list[tuple[float, float]] = []\n connectivity: list[list[int]] = []\n    for x, y in lines:\n xa = np.asarray(x, dtype=float).ravel()\n ya = np.asarray(y, dtype=float).ravel()\n if xa.size != ya.size or xa.size < 2:\n raise ValueError("each line needs equally sized x and y arrays")\n ids = list(range(len(points), len(points) + xa.size))\n points.extend(zip(xa.tolist(), ya.tolist()))\n connectivity.append(ids)\n    size = sum(len(ids) + 1 for ids in connectivity)\n    with Path(path).open("w", encoding="utf-8") as handle:\n handle.write("# vtk DataFile Version 3.0\\n")\n handle.write(f"{title}\\n")\n handle.write("ASCII\\n")\n handle.write("DATASET POLYDATA\\n")\n handle.write(f"POINTS {len(points)} float\\n")\n for xi, yi in points:\n handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n handle.write(f"LINES {len(connectivity)} {size}\\n")\n for ids in connectivity:\n handle.write(f"{len(ids)} {\' \'.join(str(i) for i in ids)}\\n")\n\n\ndef write_points_vtk(path: str | Path, x: np.ndarray, y: np.ndarray, scalars: Mapping[str, np.ndarray] | None = None, title: str = "points") -> None:\n    xa = np.asarray(x, dtype=float).ravel()\n    ya = np.asarray(y, dtype=float).ravel()\n    if xa.size != ya.size:\n raise ValueError("point arrays must have equal size")\n scalar_map = dict(scalars or {})\n    with Path(path).open("w", encoding="utf-8") as handle:\n handle.write("# vtk DataFile Version 3.0\\n")\n handle.write(f"{title}\\n")\n handle.write("ASCII\\n")\n handle.write("DATASET POLYDATA\\n")\n handle.write(f"POINTS {xa.size} float\\n")\n for xi, yi in zip(xa, ya):\n handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n handle.write(f"VERTICES {xa.size} {2 * xa.size}\\n")\n for i in range(xa.size):\n handle.write(f"1 {i}\\n")\n if scalar_map:\n handle.write(f"POINT_DATA {xa.size}\\n")\n for name, values in scalar_map.items():\n data = np.asarray(values, dtype=float).ravel()\n if data.size != xa.size:\n raise ValueError(f"scalar {name} has wrong size")\n handle.write(f"SCALARS {name} float 1\\n")\n handle.write("LOOKUP_TABLE default\\n")\n for value in data:\n handle.write(f"{value:.12e}\\n")\n\n\ndef gradient(field: np.ndarray, dx: float, dy: float) -> tuple[np.ndarray, np.ndarray]:\n    gy, gx = np.gradient(field, dy, dx, edge_order=2)\n    return gx, gy\n\n\ndef laplacian(field: np.ndarray, dx: float, dy: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    out[1:-1, 1:-1] = (\n (field[1:-1, 2:] - 2.0 * field[1:-1, 1:-1] + field[1:-1, :-2]) / (dx * dx)\n + (field[2:, 1:-1] - 2.0 * field[1:-1, 1:-1] + field[:-2, 1:-1]) / (dy * dy)\n    )\n    return out\n\n\ndef upwind_derivative_x(field: np.ndarray, velocity: np.ndarray, dx: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    backward = (field[1:-1, 1:-1] - field[1:-1, :-2]) / dx\n    forward = (field[1:-1, 2:] - field[1:-1, 1:-1]) / dx\n    out[1:-1, 1:-1] = np.where(velocity[1:-1, 1:-1] >= 0.0, backward, forward)\n    return out\n\n\ndef upwind_derivative_y(field: np.ndarray, velocity: np.ndarray, dy: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    backward = (field[1:-1, 1:-1] - field[:-2, 1:-1]) / dy\n    forward = (field[2:, 1:-1] - field[1:-1, 1:-1]) / dy\n    out[1:-1, 1:-1] = np.where(velocity[1:-1, 1:-1] >= 0.0, backward, forward)\n    return out\n\n\ndef poisson_jacobi(phi: np.ndarray, rhs: np.ndarray, dx: float, dy: float, iterations: int, boundary: str = "neumann", mask: np.ndarray | None = None) -> np.ndarray:\n    dx2 = dx * dx\n    dy2 = dy * dy\n    denom = 2.0 * (dx2 + dy2)\n    for _ in range(iterations):\n old = phi.copy()\n phi[1:-1, 1:-1] = (\n (old[1:-1, 2:] + old[1:-1, :-2]) * dy2\n + (old[2:, 1:-1] + old[:-2, 1:-1]) * dx2\n - rhs[1:-1, 1:-1] * dx2 * dy2\n ) / denom\n if mask is not None:\n phi[mask > 0.5] = 0.0\n if boundary == "neumann":\n phi[:, 0] = phi[:, 1]\n phi[:, -1] = 0.0\n phi[0, :] = phi[1, :]\n phi[-1, :] = phi[-2, :]\n elif boundary == "closed":\n phi[:, 0] = phi[:, 1]\n phi[:, -1] = phi[:, -2]\n phi[0, :] = phi[1, :]\n phi[-1, :] = phi[-2, :]\n phi -= np.mean(phi)\n else:\n raise ValueError("unsupported pressure boundary")\n return phi\n\n\ndef tdma(a: np.ndarray, b: np.ndarray, c: np.ndarray, d: np.ndarray) -> np.ndarray:\n    a = np.asarray(a, dtype=float).copy()\n b = np.asarray(b, dtype=float).copy()\n    c = np.asarray(c, dtype=float).copy()\n    d = np.asarray(d, dtype=float).copy()\n    n = d.size\n    if n == 0 or a.size != n or b.size != n or c.size != n:\n raise ValueError("TDMA arrays must have equal nonzero size")\n for i in range(1, n):\n if abs(b[i - 1]) < 1e-30:\n raise FloatingPointError("zero TDMA pivot")\n factor = a[i] / b[i - 1]\n b[i] -= factor * c[i - 1]\n d[i] -= factor * d[i - 1]\n    x = np.empty(n, dtype=float)\n    if abs(b[-1]) < 1e-30:\n raise FloatingPointError("zero TDMA pivot")\n    x[-1] = d[-1] / b[-1]\n    for i in range(n - 2, -1, -1):\n if abs(b[i]) < 1e-30:\n raise FloatingPointError("zero TDMA pivot")\n x[i] = (d[i] - c[i] * x[i + 1]) / b[i]\n    return x\n\n\ndef relative_l2(numerical: np.ndarray, reference: np.ndarray) -> float:\n    a = np.asarray(numerical, dtype=float)\n    b = np.asarray(reference, dtype=float)\n    denominator = float(np.linalg.norm(b.ravel()))\n    numerator = float(np.linalg.norm((a - b).ravel()))\n    return numerator / max(denominator, 1e-30)\n\n\ndef require_finite(*arrays: np.ndarray) -> None:\n    for array in arrays:\n if not np.all(np.isfinite(array)):\n raise FloatingPointError("non-finite numerical field detected")\n\n\ndef positive_float(value: str, name: str) -> float:\n    result = float(value)\n    if not math.isfinite(result) or result <= 0.0:\n raise ValueError(f"{name} must be positive and finite")\n    return result\n\n\ndef positive_int(value: str, name: str, minimum: int = 1) -> int:\n    result = int(value)\n    if result < minimum:\n raise ValueError(f"{name} must be at least {minimum}")\n    return result\n', encoding="utf-8")
(CODE_DIR / 'exp07_lake_wave_hyperbolic.py').write_text('from __future__ import annotations\n\nimport csv\nimport math\nimport sys\n\nimport numpy as np\n\nfrom common import relative_l2, trapezoidal_integral, write_structured_grid_vtk\n\n\ndef gaussian(x: np.ndarray, center: float, sigma: float) -> np.ndarray:\n    return np.exp(-0.5 * ((x - center) / sigma) ** 2)\n\n\ndef analytical_eta(x: np.ndarray, time: float, speed: float, amplitude: float, center: float, sigma: float) -> np.ndarray:\n    return 0.5 * amplitude * (gaussian(x, center + speed * time, sigma) + gaussian(x, center - speed * time, sigma))\n\n\ndef upwind(field: np.ndarray, courant: float) -> np.ndarray:\n    if courant >= 0.0:\n return field - courant * (field - np.roll(field, 1))\n    return field - courant * (np.roll(field, -1) - field)\n\n\ndef lax_wendroff(field: np.ndarray, courant: float) -> np.ndarray:\n    return field - 0.5 * courant * (np.roll(field, -1) - np.roll(field, 1)) + 0.5 * courant * courant * (np.roll(field, -1) - 2.0 * field + np.roll(field, 1))\n\n\ndef reconstruct(wp: np.ndarray, wm: np.ndarray, depth: float, speed: float) -> tuple[np.ndarray, np.ndarray]:\n    velocity = 0.5 * (wp + wm)\n eta = depth * (wp - wm) / (2.0 * speed)\n    return eta, velocity\n\n\ndef write_visual(path: str, x: np.ndarray, eta: np.ndarray, velocity: np.ndarray, depth: float, scale: float, nz: int) -> None:\n    vertical_fraction = np.linspace(0.0, 1.0, nz)\n    xx = np.repeat(x[None, :], nz, axis=0)\n    bottom = -depth * scale\n    surface = eta * scale\n    yy = bottom + vertical_fraction[:, None] *
```

```
(surface[None, :] - bottom)\n    eta_field = np.repeat(eta[None, :], nz, axis=0)\n    velocity_field = np.repeat(velocity[None, :], nz, axis=0)\n    fraction = np.repeat(vertical_fraction[:, None], x.size, axis=1)\n    write_structured_grid_vtk(path, xx, yy, {"surface_elevation": eta_field, "vertical_fraction": fraction, "velocity_magnitude": np.abs(velocity_field)}, {"velocity": (velocity_field, np.zeros_like(velocity_field))}, "Still-lake shallow-water visualization")\n\n\ndef write_surface_ribbon(path: str, x: np.ndarray, eta: np.ndarray, scale: float) -> None:\n    xx = np.repeat(x[None, :], 2, axis=0)\n    yy = np.vstack((eta * scale - 0.01 * scale, eta * scale + 0.01 * scale))\n    eta_field = np.repeat(eta[None, :], 2, axis=0)\n    write_structured_grid_vtk(path, xx, yy, {"surface_elevation": eta_field}, title="Lake free-surface ribbon")\n\n\ndef main() -> int:\n    nx = int(sys.argv[1]) if len(sys.argv) > 1 else 401\n    length = float(sys.argv[2]) if len(sys.argv) > 2 else 20.0\n    depth = float(sys.argv[3]) if len(sys.argv) > 3 else 1.0\n    amplitude = float(sys.argv[4]) if len(sys.argv) > 4 else 0.02\n    sigma = float(sys.argv[5]) if len(sys.argv) > 5 else 0.5\n    center = float(sys.argv[6]) if len(sys.argv) > 6 else 10.0\n    end_time = float(sys.argv[7]) if len(sys.argv) > 7 else 2.0\n    cfl = float(sys.argv[8]) if len(sys.argv) > 8 else 0.8\n    output_every = int(sys.argv[9]) if len(sys.argv) > 9 else 25\n    gravity = float(sys.argv[10]) if len(sys.argv) > 10 else 9.81\n    vertical_scale = float(sys.argv[11]) if len(sys.argv) > 11 else 180.0\n    nz_visual = int(sys.argv[12]) if len(sys.argv) > 12 else 61\n    if nx < 41 or length <= 0.0 or depth <= 0.0 or sigma <= 0.0 or end_time < 0.0 or gravity <= 0.0 or output_every < 1 or nz_visual < 3:\n        raise ValueError("usage: python exp07_lake_wave_hyperbolic.py nx L H amplitude sigma x0 tEnd CFL outputEvery gravity verticalScale nzVisual")\n    cfl = min(0.99, max(0.01, cfl))\n    wave_speed = math.sqrt(gravity * depth)\n    x = np.linspace(0.0, length, nx)\n    dx = x[1] - x[0]\n    dt_nominal = cfl * dx / wave_speed\n    eta0 = amplitude * gaussian(x, center, sigma)\n    wp_up = wave_speed * eta0 / depth\n    wm_up = -wave_speed * eta0 / depth\n    wp_lw = wp_up.copy()\n    wm_lw = wm_up.copy()\n    eta_up, u_up = reconstruct(wp_up, wm_up, depth, wave_speed)\n    eta_lw, u_lw = reconstruct(wp_lw, wm_lw, depth, wave_speed)\n    initial_mass = trapezoidal_integral(eta0, x)\n    snapshot_times = [0.0]\n    snapshot_eta = [eta_lw.copy()]\n    history: list[list[float]] = []\n    time = 0.0\n    step = 0\n    while time < end_time - 1e-14:\n        dt = min(dt_nominal, end_time - time)\n        courant = wave_speed * dt / dx\n        wp_up = upwind(wp_up, courant)\n        wm_up = upwind(wm_up, -courant)\n        wp_lw = lax_wendroff(wp_lw, courant)\n        wm_lw = lax_wendroff(wm_lw, -courant)\n        time += dt\n        step += 1\n        eta_up, u_up = reconstruct(wp_up, wm_up, depth, wave_speed)\n        eta_lw, u_lw = reconstruct(wp_lw, wm_lw, depth, wave_speed)\n        reference = analytical_eta(x, time, wave_speed, amplitude, center, sigma)\n        error_up = relative_l2(eta_up, reference)\n        error_lw = relative_l2(eta_lw, reference)\n        mass_up = trapezoidal_integral(eta_up, x)\n        mass_lw = trapezoidal_integral(eta_lw, x)\n        history.append([step, time, dt, courant, error_up, error_lw, mass_up, mass_lw])\n        if step % output_every == 0 or time >= end_time - 1e-14:\n            write_visual(f"lake_water_body_{step:06d}.vtk", x, eta_lw, u_lw, depth, vertical_scale, nz_visual)\n            write_surface_ribbon(f"lake_surface_scaled_{step:06d}.vtk", x, eta_lw, vertical_scale)\n            snapshot_times.append(time)\n            snapshot_eta.append(eta_lw.copy())\n            print(f"step={step} time={time:.6e} upwind_error={error_up:.6e} lax_wendroff_error={error_lw:.6e}")\n    reference = analytical_eta(x, time, wave_speed, amplitude, center, sigma)\n    write_visual("OPEN_THIS_lake_visual_final.vtk", x, eta_lw, u_lw, depth, vertical_scale, nz_visual)\n    write_surface_ribbon("OPEN_THIS_lake_surface_ribbon_final.vtk", x, eta_lw, vertical_scale)\n    write_visual("lake_water_body_final.vtk", x, eta_lw, u_lw, depth, vertical_scale, nz_visual)\n    write_surface_ribbon("lake_surface_scaled_final.vtk", x, eta_lw, vertical_scale)\n    snapshot_array = np.asarray(snapshot_eta)\n    time_array = np.asarray(snapshot_times)\n    xx = np.repeat(x[None, :], time_array.size, axis=0)\n    tt = np.repeat(time_array[:, None], nx, axis=1)\n    write_structured_grid_vtk("lake_space_time.vtk", xx, tt, {"surface_elevation": snapshot_array}, title="Lake wave space-time field")\n    with open("lake_wave_history.csv", "w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(["step", "time", "dt", "CFL", "relative_l2_error_upwind", "relative_l2_error_lax_wendroff", "mass_upwind", "mass_lax_wendroff"])\n        writer.writerows(history)\n    with open("lake_wave_profiles.csv", "w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(["x", "eta_upwind", "velocity_upwind", "eta_lax_wendroff", "velocity_lax_wendroff", "eta_analytical"])\n        for values in zip(x, eta_up, u_up, eta_lw, u_lw, reference):\n            writer.writerow(values)\n    with open("lake_wave_summary.txt", "w", encoding="utf-8") as handle:\n        handle.write("Linearized one-dimensional shallow-water equations\\n")\n        handle.write(f"nx {nx}\\nlength_m {length:.12e}\\nmean_depth_m {depth:.12e}\\ngravity {gravity:.12e}\\nwave_speed {wave_speed:.12e}\\n")\n        handle.write(f"initial_amplitude {amplitude:.12e}\\nsigma {sigma:.12e}\\ninitial_center {center:.12e}\\nfinal_time {time:.12e}\\ntime_steps {step}\\nCFL {cfl:.12e}\\n")\n        handle.write(f"initial_eta_mass {initial_mass:.12e}\\nfinal_eta_mass_upwind {trapezoidal_integral(eta_up, x):.12e}\\nfinal_eta_mass_lax_wendroff {trapezoidal_integral(eta_lw, x):.12e}\\n")\n        handle.write(f"relative_L2_error_upwind {relative_l2(eta_up, reference):.12e}\\nrelative_L2_error_lax_wendroff {relative_l2(eta_lw, reference):.12e}\\n")\n    print("Wrote OPEN_THIS_lake_visual_final.vtk, OPEN_THIS_lake_surface_ribbon_final.vtk, lake_space_time.vtk, lake_wave_profiles.csv, lake_wave_history.csv, lake_wave_summary.txt")\n    return 0\n\n\nif __name__ == "__main__":\n    raise SystemExit(main())\n', encoding="utf-8")
SCRIPT = CODE_DIR / 'exp07_lake_wave_hyperbolic.py'
print("Runtime:", "Google Colab" if IN_COLAB else "local notebook test")
print("Working folder:", ROOT)
print("No package installation or external file upload is required.")


# ===== NOTEBOOK CODE CELL 1 =====
#@title 2. Select run mode and parameters
RUN_MODE = "custom" #@param ["quick", "lab", "custom"]

nx = 401 #@param {type:"integer"}
ny_vtk = 20 #@param {type:"integer"}
depth = 1.0 #@param {type:"number"}
amplitude = 0.02 #@param {type:"number"}
x0 = 0.5 #@param {type:"number"}
length = 10.0 #@param {type:"number"}
t_end = 2.0 #@param {type:"number"}
CFL = 1.1 #@param {type:"number"}
output_every = 25 #@param {type:"integer"}
gravity = 9.81 #@param {type:"number"}
n_snapshots = 180 #@param {type:"integer"}
space_time_rows = 61 #@param {type:"integer"}

QUICK_ARGS = [81, 20, 1.0, 0.02, 0.5, 10.0, 0.1, 1.1, 10, 9.81, 30, 15]
LAB_ARGS = [401, 20, 1.0, 0.02, 0.5, 10.0, 2.0, 1.1, 25, 9.81, 180, 61]
CUSTOM_ARGS = [nx, ny_vtk, depth, amplitude, x0, length, t_end, CFL, output_every, gravity, n_snapshots, space_time_rows]
ARGS = QUICK_ARGS if RUN_MODE == "quick" else LAB_ARGS if RUN_MODE == "lab" else CUSTOM_ARGS
print("Run mode:", RUN_MODE)
print("Arguments:", ARGS)


# ===== NOTEBOOK CODE CELL 2 =====
#@title 3. Run the numerical solver
OUT = run_solver(SCRIPT, ARGS, OUTPUT_DIR)


# ===== NOTEBOOK CODE CELL 3 =====
#@title 4. Read the solver summary
summary_text = show_summary(OUT / 'lake_wave_summary.txt')


# ===== NOTEBOOK CODE CELL 4 =====
#@title 5. Plot the primary numerical results
profiles = read_csv(OUT / "lake_wave_profiles.csv")
plt.figure(figsize=(8, 4))
plt.plot(profiles["x"], profiles["eta_upwind"], label="Upwind")
plt.plot(profiles["x"], profiles["eta_lax_wendroff"], label="Lax-Wendroff")
plt.plot(profiles["x"], profiles["eta_analytical"], "--", label="Analytical")
plt.xlabel("x")
```

```
plt.ylabel("Surface displacement")
plt.title("Final wave profile")
plt.legend()
plt.grid(True)
plt.show()

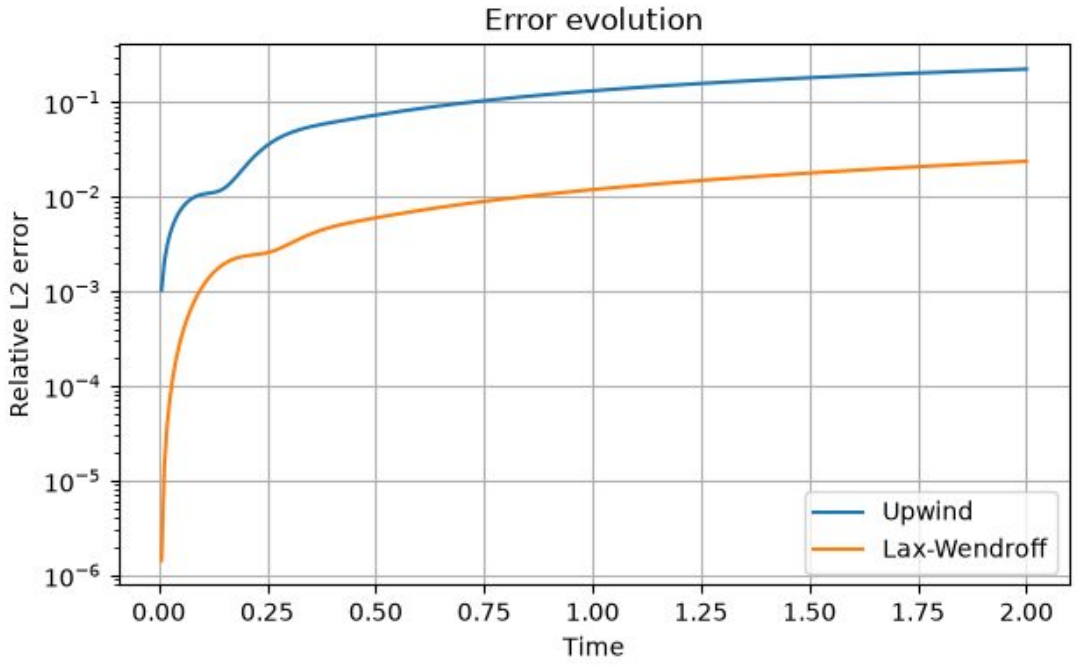
history = read_csv(OUT / "lake_wave_history.csv")
plt.figure(figsize=(7, 4))
plt.semilogy(history["time"], history["relative_l2_error_upwind"], label="Upwind")
plt.semilogy(history["time"], history["relative_l2_error_lax_wendroff"], label="Lax-Wendroff")
plt.xlabel("Time")
plt.ylabel("Relative L2 error")
plt.title("Error evolution")
plt.legend()
plt.grid(True)
plt.show()

# ===== NOTEBOOK CODE CELL 5 =====
#@title 6. Display the final field and generated files
plot_vtk_scalar(OUT / 'lake_space_time.vtk', 'surface_elevation')
files = list_outputs(OUT)

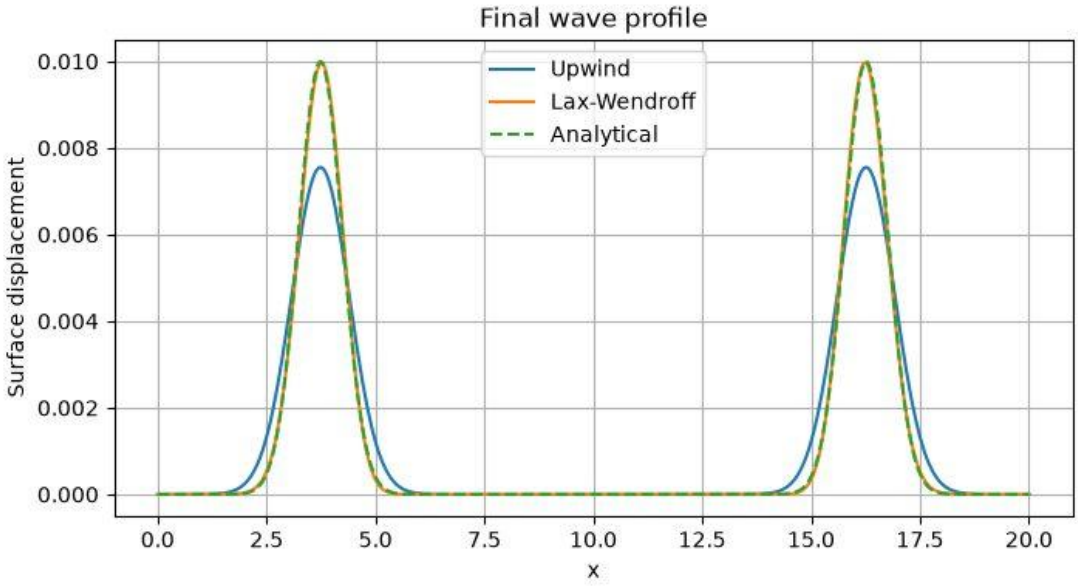

# ===== NOTEBOOK CODE CELL 6 =====
#@title 7. Package and download the results
DOWNLOAD_RESULTS = False #@param {type:"boolean"}
RESULTS_ZIP = make_results_zip(OUT, 7)
if DOWNLOAD_RESULTS:
    if IN_COLAB:
        colab_files.download(str(RESULTS_ZIP))
    else:
        print("Local test mode. Archive retained at:", RESULTS_ZIP)
else:
    print("Set DOWNLOAD_RESULTS to True and run this cell when ready to download.")
```

Output

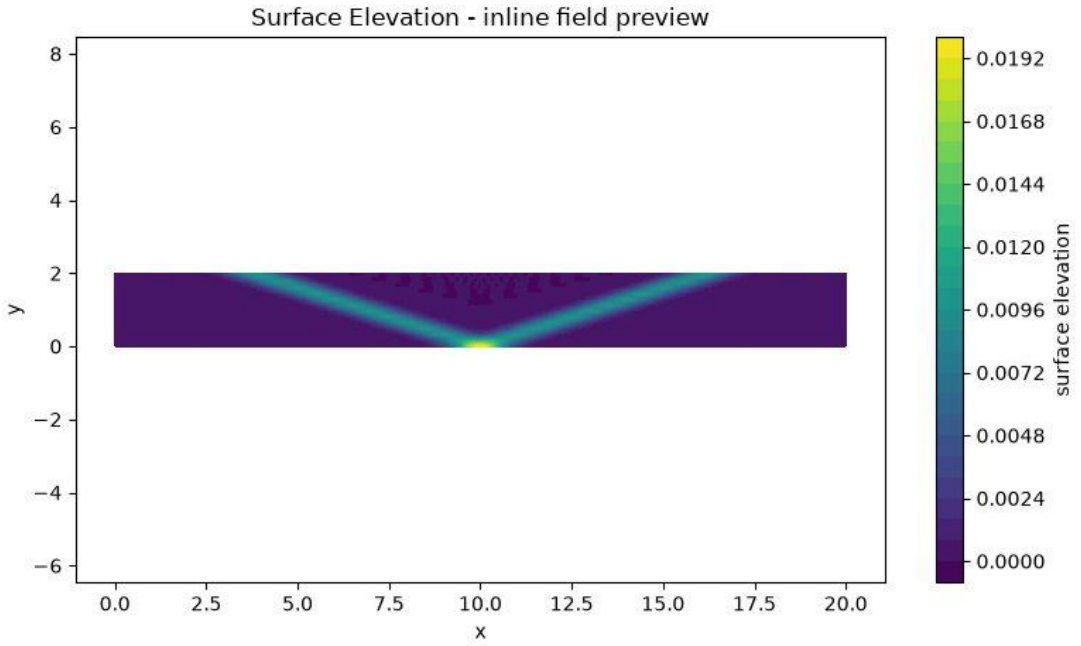
The following figures are reproduced from the output folder supplied for this experiment.



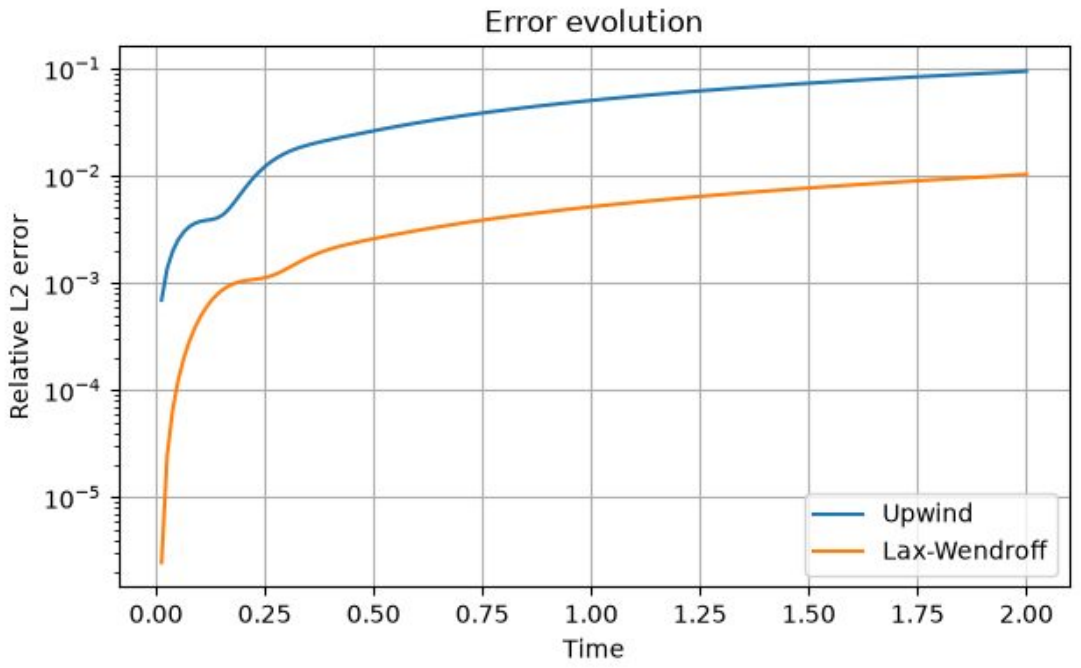
Experiment 7 output: Output at CFL 0.4/Error Evolution



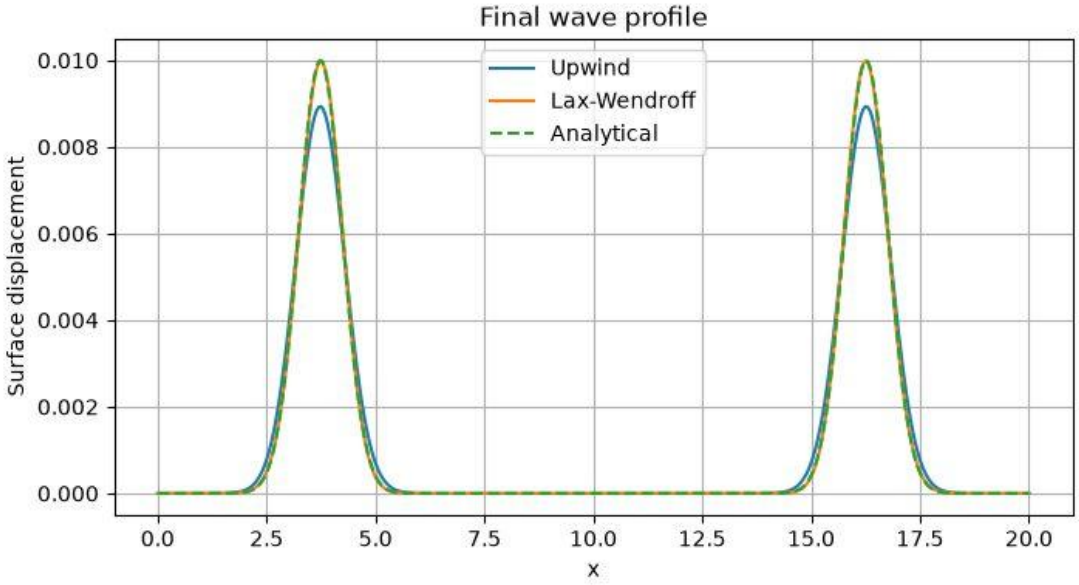
Experiment 7 output: Output at CFL 0.4/Final Wave Profile



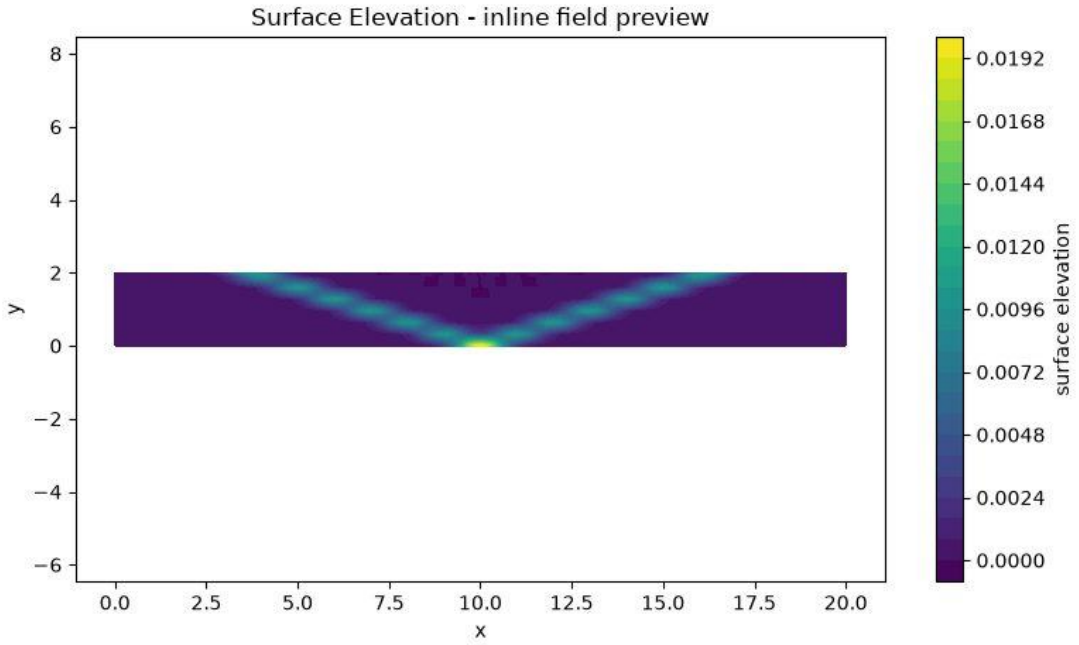
Experiment 7 output: Output at CFL 0.4/Surface Elevation



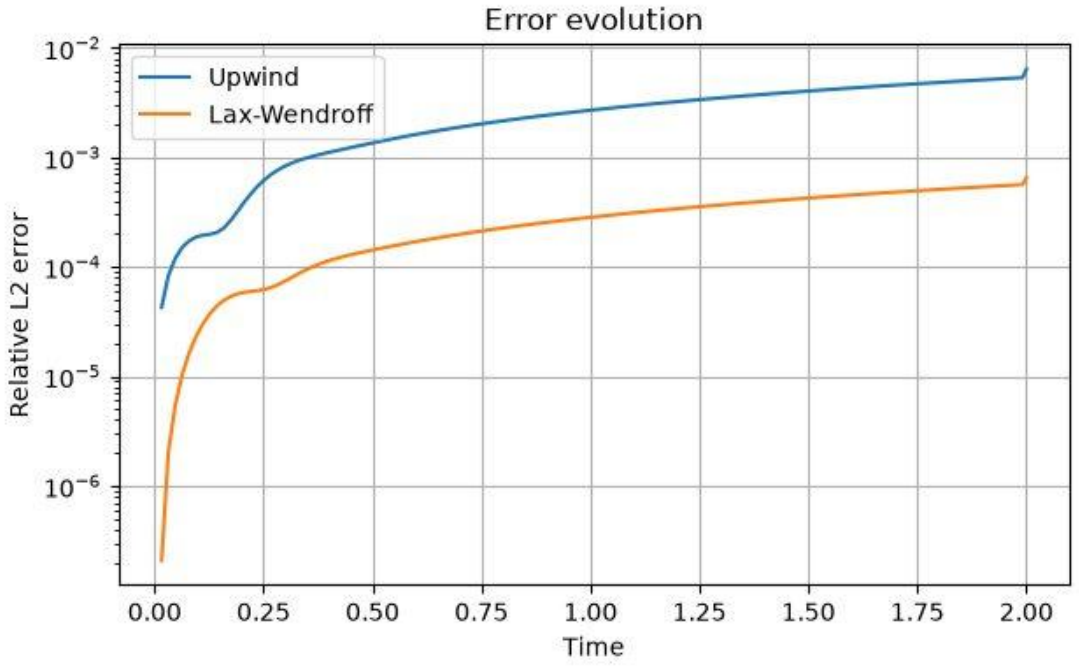
Experiment 7 output: Output at CFL 0.8/Error Evolution



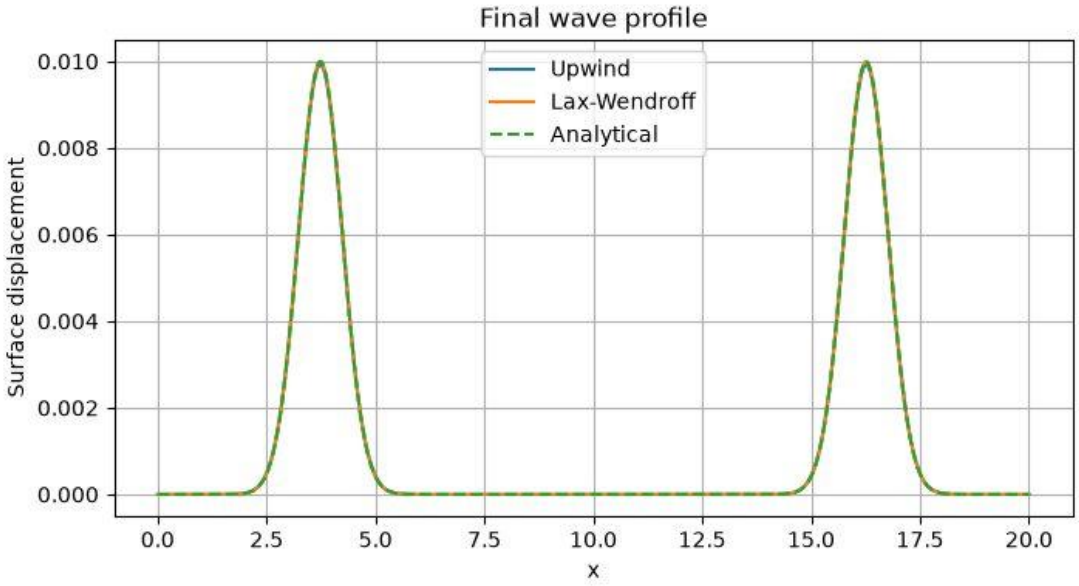
Experiment 7 output: Output at CFL 0.8/Final Wave Profile



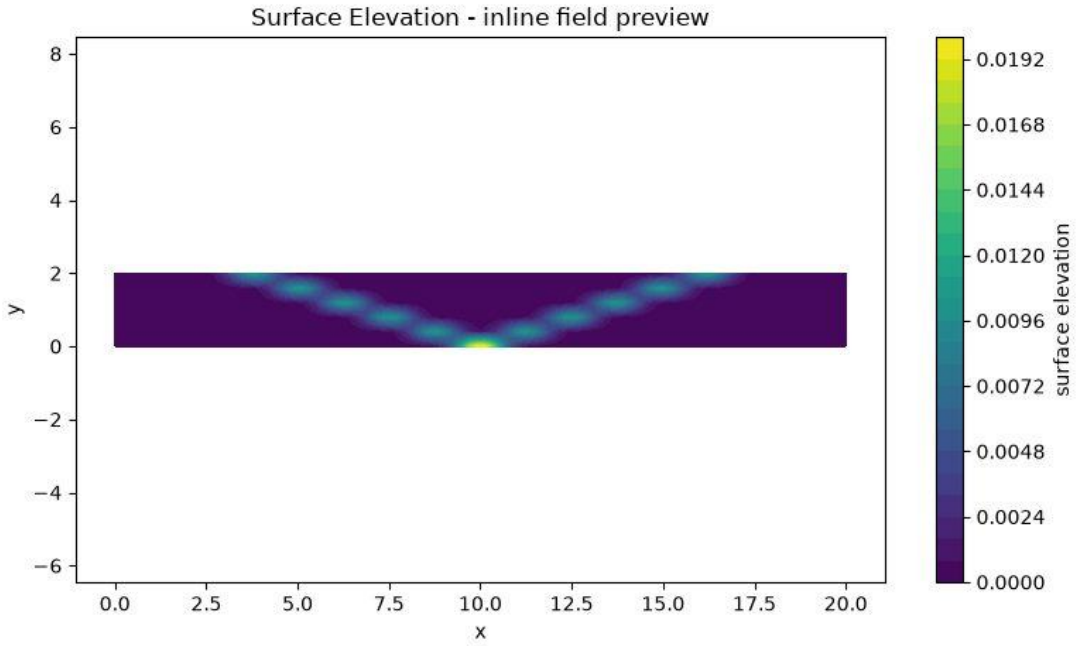
Experiment 7 output: Output at CFL 0.8/Surface Elevation



Experiment 7 output: Output at CFL 1.1/Error Evolution



Experiment 7 output: Output at CFL 1.1/Final Wave Profile



Experiment 7 output: Output at CFL 1.1/Surface Elevation

Results and Discussion

The saved outputs include final wave profiles, relative L2-error histories and a surface-elevation field preview for each CFL case. The Upwind profile is more diffusive than the Lax–Wendroff profile, while Lax–Wendroff tracks the analytical pulse more closely in the supplied plots. The CFL=1.1 case is included specifically to assess stability; interpret any oscillations or error growth from its saved figures rather than assuming instability solely from the CFL label.

Conclusion

The experiment demonstrates the effect of time-step selection on numerical wave propagation. The Upwind method is more dissipative, whereas Lax–Wendroff generally preserves wave amplitude more accurately but may show dispersive behaviour. Stability and conservation should be judged from the saved error and mass-history data.

EXPERIMENT 8 – Rayleigh–Taylor instability using a VOF-type model

Aim

To investigate the development of a Rayleigh–Taylor interface under gravity and compare the effect of Atwood number on spike, bubble and mixing-layer evolution.

Objective

- Initialise a perturbed interface between two fluids of different densities.
- Advect the heavy-fluid volume fraction using a conservative flux update.
- Solve the simplified velocity/pressure response with gravity and viscosity effects represented in the teaching model.
- Compare Atwood numbers 0.2, 0.5 and 0.8 using spike/bubble positions, mixing width, fraction bounds and mass conservation.

Governing Equation

Atwood number: $A=(\rho_h-\rho_l)/(\rho_h+\rho_l)$.

Volume-fraction transport:

$\partial\alpha/\partial t + \nabla\cdot(\alpha u)=0$.

The model also uses an incompressible velocity field with pressure correction and gravity/viscous terms. The supplied notebook is a simplified VOF-type teaching implementation; it is not a full production multiphase solver with geometric interface reconstruction and all interfacial physics.

Boundary Condition

- Velocity is set to zero on all four domain boundaries (no-slip/closed-box boundary in the supplied function).
- The initial interface is sinusoidally perturbed about mid-height and smoothed over a small vertical distance.
- The volume fraction is bounded to [0,1] after each conservative advection update; a redistribution step attempts to preserve its total sum.

Code

Parameters: nx=96, ny=192, Atwood=0.2/0.5/0.8, gravity=1.0, nu=0.0008, t_end=2.0, max_steps=800, poisson_iterations=60 and output_every=200.

```
# ===== NOTEBOOK CODE CELL 0 =====
from pathlib import Path
import shutil
import subprocess
import sys
import zipfile
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Markdown, display

IN_COLAB = False
try:
    from google.colab import files as colab_files
    IN_COLAB = True
except ImportError:
    colab_files = None


def run_solver(script_path, arguments, output_dir, timeout=1800):
    output_dir = Path(output_dir)
    if output_dir.exists():
        shutil.rmtree(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    command = [sys.executable, str(script_path), *[str(value) for value in arguments]]
    print("Running:", " ".join(command))
    completed = subprocess.run(command, cwd=output_dir, capture_output=True, text=True, timeout=timeout)
    (output_dir / "solver_stdout.log").write_text(completed.stdout, encoding="utf-8")
    (output_dir / "solver_stderr.log").write_text(completed.stderr, encoding="utf-8")
    if completed.stdout.strip():
```

```
        print(completed.stdout.strip())
    if completed.returncode != 0:
        raise RuntimeError(completed.stderr.strip() or f"Solver exited with code {completed.returncode}")
    if completed.stderr.strip():
        print(completed.stderr.strip())
    return output_dir


def read_csv(path):
    data = np.genfromtxt(path, delimiter=",", names=True, dtype=None, encoding="utf-8")
    return np.atleast_1d(data)


def show_summary(path):
    text = Path(path).read_text(encoding="utf-8").strip()
    display(Markdown(f"### Solver summary\n```text\n{text}\n```"))
    return text


def list_outputs(output_dir):
    rows = []
    for path in sorted(Path(output_dir).iterdir()):
        if path.is_file():
            size = path.stat().st_size
            if size < 1024:
                size_text = f"{size} B"
            elif size < 1024**2:
                size_text = f"{size / 1024:.1f} KB"
            else:
                size_text = f"{size / 1024**2:.1f} MB"
            rows.append((path.name, size_text))
    table = "\n".join(f"| `{name}` | {size} |" for name, size in rows)
    display(Markdown(f"### Generated files\n| File | Size |\n|---|---:|\n{table}"))
    return rows


def read_structured_vtk(path):
    lines = Path(path).read_text(encoding="utf-8").splitlines()
    dims_index = next(i for i, line in enumerate(lines) if line.startswith("DIMENSIONS "))
    _, nx, ny, nz = lines[dims_index].split()
    nx, ny, nz = int(nx), int(ny), int(nz)
    if nz != 1:
        raise ValueError("Only 2D structured VTK files are supported by this notebook preview")
    points_index = next(i for i, line in enumerate(lines) if line.startswith("POINTS "))
    npoints = int(lines[points_index].split()[1])
    points = np.array([[float(v) for v in lines[points_index + 1 + k].split()[:3]] for k in range(npoints)])
    x = points[:, 0].reshape(ny, nx)
    y = points[:, 1].reshape(ny, nx)
    data_index = next(i for i, line in enumerate(lines) if line.startswith("POINT_DATA ")) + 1
    scalars = {}
    vectors = {}
    i = data_index
    while i < len(lines):
        if lines[i].startswith("SCALARS "):
            name = lines[i].split()[1]
            i += 2
            values = np.array([float(lines[i + k].split()[0]) for k in range(npoints)])
            scalars[name] = values.reshape(ny, nx)
            i += npoints
        elif lines[i].startswith("VECTORS "):
            name = lines[i].split()[1]
            i += 1
            values = np.array([[float(v) for v in lines[i + k].split()[:3]] for k in range(npoints)])
            vectors[name] = (values[:, 0].reshape(ny, nx), values[:, 1].reshape(ny, nx))
            i += npoints
        else:
            i += 1
    return x, y, scalars, vectors


def plot_vtk_scalar(path, preferred_field=None):
    path = Path(path)
    if not path.is_file():
        print(f"VTK preview skipped: {path.name} was not generated")
        return
    x, y, scalars, _ = read_structured_vtk(path)
    if not scalars:
        print("VTK preview skipped: no scalar field found")
        return
    field = preferred_field if preferred_field in scalars else next(iter(scalars))
    plt.figure(figsize=(8, 4.8))
    image = plt.contourf(x, y, scalars[field], levels=30)
    plt.colorbar(image, label=field.replace("_", " "))
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title(f"{field.replace('_', ' ').title()} - inline field preview")
    plt.axis("equal")
    plt.tight_layout()
    plt.show()


def make_results_zip(output_dir, experiment_number):
    output_dir = Path(output_dir)
    archive = output_dir.parent / f"experiment_{experiment_number:02d}_results.zip"
    if archive.exists():
        archive.unlink()
    with zipfile.ZipFile(archive, "w", compression=zipfile.ZIP_DEFLATED) as handle:
        for path in sorted(output_dir.rglob("*")):
            if path.is_file():
```

```
            handle.write(path, path.relative_to(output_dir))
    print(f"Results archive: {archive} ({archive.stat().st_size / 1024:.1f} KB)")
    return archive


EXPERIMENT = 8
ROOT = Path("/content/cfd_lab_exp08") if IN_COLAB else Path(r"C:\CFD_Runs\_colab_runtime_exp08")
CODE_DIR = ROOT / "code"
OUTPUT_DIR = ROOT / "outputs"
CODE_DIR.mkdir(parents=True, exist_ok=True)
(CODE_DIR / "common.py").write_text('from __future__ import annotations\n\nimport csv\nimport math\nfrom pathlib import Path\nfrom typing
import Iterable, Mapping\n\nimport numpy as np\n\n\ndef trapezoidal_integral(values: np.ndarray, coordinates: np.ndarray) -> float:\n    y
= np.asarray(values, dtype=float).ravel()\n    x = np.asarray(coordinates, dtype=float).ravel()\n    if y.size != x.size or y.size < 2:\n
raise ValueError("trapezoidal integration requires equally sized arrays with at least two points")\n    return float(np.sum(0.5 * (y[1:] +
y[:-1]) * np.diff(x)))\n\n\ndef write_csv(path: str | Path, header: Iterable[str], rows: Iterable[Iterable[object]]) -> None:\n    with
Path(path).open("w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(list(header))\n
writer.writerows(rows)\n\n\ndef mesh_xy(nx: int, ny: int, lx: float = 1.0, ly: float = 1.0, x0: float = 0.0, y0: float = 0.0) ->
tuple[np.ndarray, np.ndarray]:\n    x = np.linspace(x0, x0 + lx, nx)\n    y = np.linspace(y0, y0 + ly, ny)\n    return np.meshgrid(x,
y)\n\n\ndef write_structured_grid_vtk(\n    path: str | Path,\n    x: np.ndarray,\n    y: np.ndarray,\n    scalars: Mapping[str,
np.ndarray] | None = None,\n    vectors: Mapping[str, tuple[np.ndarray, np.ndarray]] | None = None,\n    title: str = "CFD lab output",\n)
-> None:\n    x = np.asarray(x, dtype=float)\n    y = np.asarray(y, dtype=float)\n    if x.shape != y.shape or x.ndim != 2:\n        raise
ValueError("x and y must be equally shaped 2D arrays")\n    ny, nx = x.shape\n    scalar_map = dict(scalars or {})\n    vector_map =
dict(vectors or {})\n    for name, values in scalar_map.items():\n        if np.asarray(values).shape != x.shape:\n            raise
ValueError(f"scalar {name} has shape {np.asarray(values).shape}, expected {x.shape}")\n    for name, values in vector_map.items():\n
if len(values) != 2 or np.asarray(values[0]).shape != x.shape or np.asarray(values[1]).shape != x.shape:\n            raise
ValueError(f"vector {name} has incompatible shape")\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("#
vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET
STRUCTURED_GRID\\n")\n        handle.write(f"DIMENSIONS {nx} {ny} 1\\n")\n        handle.write(f"POINTS {nx * ny} float\\n")\n        for j
in range(ny):\n            for i in range(nx):\n                handle.write(f"{x[j, i]:.12e} {y[j, i]:.12e} 0\\n")\n
handle.write(f"POINT_DATA {nx * ny}\\n")\n        for name, values in scalar_map.items():\n            data = np.asarray(values,
dtype=float)\n            handle.write(f"SCALARS {name} float 1\\n")\n            handle.write("LOOKUP_TABLE default\\n")\n            for
value in data.ravel(order="C"):\n                handle.write(f"{value:.12e}\\n")\n        for name, values in vector_map.items():\n
vx = np.asarray(values[0], dtype=float)\n            vy = np.asarray(values[1], dtype=float)\n            handle.write(f"VECTORS {name}
float\\n")\n            for ax, ay in zip(vx.ravel(order="C"), vy.ravel(order="C")):\n                handle.write(f"{ax:.12e} {ay:.12e}
0\\n")\n\n\ndef write_polyline_vtk(path: str | Path, x: np.ndarray, y: np.ndarray, title: str = "polyline", close: bool = False) -> None:\n
x = np.asarray(x, dtype=float).ravel()\n    y = np.asarray(y, dtype=float).ravel()\n    if x.size != y.size or x.size < 2:\n        raise
ValueError("polyline requires equally sized x and y arrays")\n    ids = list(range(x.size))\n    if close:\n        ids.append(0)\n    with
Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n
handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET POLYDATA\\n")\n        handle.write(f"POINTS
{x.size} float\\n")\n        for xi, yi in zip(x, y):\n            handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n        handle.write(f"LINES
1 {len(ids) + 1}\\n")\n        handle.write(f"{len(ids)} {\' \'.join(str(i) for i in ids)}\\n")\n\n\ndef write_multiline_vtk(path: str |
Path, lines: list[tuple[np.ndarray, np.ndarray]], title: str = "lines") -> None:\n    points: list[tuple[float, float]] = []\n
connectivity: list[list[int]] = []\n    for x, y in lines:\n        xa = np.asarray(x, dtype=float).ravel()\n        ya = np.asarray(y,
dtype=float).ravel()\n        if xa.size != ya.size or xa.size < 2:\n            raise ValueError("each line needs equally sized x and y
arrays")\n        ids = list(range(len(points), len(points) + xa.size))\n        points.extend(zip(xa.tolist(), ya.tolist()))\n
connectivity.append(ids)\n    size = sum(len(ids) + 1 for ids in connectivity)\n    with Path(path).open("w", encoding="utf-8") as
handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n
handle.write("DATASET POLYDATA\\n")\n        handle.write(f"POINTS {len(points)} float\\n")\n        for xi, yi in points:\n
handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n        handle.write(f"LINES {len(connectivity)} {size}\\n")\n        for ids in connectivity:\n
handle.write(f"{len(ids)} {\' \'.join(str(i) for i in ids)}\\n")\n\n\ndef write_points_vtk(path: str | Path, x: np.ndarray, y: np.ndarray,
scalars: Mapping[str, np.ndarray] | None = None, title: str = "points") -> None:\n    xa = np.asarray(x, dtype=float).ravel()\n    ya =
np.asarray(y, dtype=float).ravel()\n    if xa.size != ya.size:\n        raise ValueError("point arrays must have equal size")\n
scalar_map = dict(scalars or {})\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version
3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET POLYDATA\\n")\n
handle.write(f"POINTS {xa.size} float\\n")\n        for xi, yi in zip(xa, ya):\n            handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n
handle.write(f"VERTICES {xa.size} {2 * xa.size}\\n")\n        for i in range(xa.size):\n            handle.write(f"1 {i}\\n")\n        if
scalar_map:\n            handle.write(f"POINT_DATA {xa.size}\\n")\n            for name, values in scalar_map.items():\n
data = np.asarray(values, dtype=float).ravel()\n                if data.size != xa.size:\n                    raise ValueError(f"scalar
{name} has wrong size")\n                handle.write(f"SCALARS {name} float 1\\n")\n                handle.write("LOOKUP_TABLE
default\\n")\n                for value in data:\n                    handle.write(f"{value:.12e}\\n")\n\n\ndef gradient(field: np.ndarray,
dx: float, dy: float) -> tuple[np.ndarray, np.ndarray]:\n    gy, gx = np.gradient(field, dy, dx, edge_order=2)\n    return gx, gy\n\n\ndef
laplacian(field: np.ndarray, dx: float, dy: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    out[1:-1, 1:-1] = (\n
(field[1:-1, 2:] - 2.0 * field[1:-1, 1:-1] + field[1:-1, :-2]) / (dx * dx)\n        + (field[2:, 1:-1] - 2.0 * field[1:-1, 1:-1] + field[:-
2, 1:-1]) / (dy * dy)\n    )\n    return out\n\n\ndef upwind_derivative_x(field: np.ndarray, velocity: np.ndarray, dx: float) ->
np.ndarray:\n    out = np.zeros_like(field)\n    backward = (field[1:-1, 1:-1] - field[1:-1, :-2]) / dx\n    forward = (field[1:-1, 2:] -
field[1:-1, 1:-1]) / dx\n    out[1:-1, 1:-1] = np.where(velocity[1:-1, 1:-1] >= 0.0, backward, forward)\n    return out\n\n\ndef
upwind_derivative_y(field: np.ndarray, velocity: np.ndarray, dy: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    backward =
(field[1:-1, 1:-1] - field[:-2, 1:-1]) / dy\n    forward = (field[2:, 1:-1] - field[1:-1, 1:-1]) / dy\n    out[1:-1, 1:-1] =
np.where(velocity[1:-1, 1:-1] >= 0.0, backward, forward)\n    return out\n\n\ndef poisson_jacobi(phi: np.ndarray, rhs: np.ndarray, dx:
float, dy: float, iterations: int, boundary: str = "neumann", mask: np.ndarray | None = None) -> np.ndarray:\n    dx2 = dx * dx\n    dy2 =
dy * dy\n    denom = 2.0 * (dx2 + dy2)\n    for _ in range(iterations):\n        old = phi.copy()\n        phi[1:-1, 1:-1] = (\n
(old[1:-1, 2:] + old[1:-1, :-2]) * dy2\n            + (old[2:, 1:-1] + old[:-2, 1:-1]) * dx2\n            - rhs[1:-1, 1:-1] * dx2 * dy2\n
) / denom\n        if mask is not None:\n            phi[mask > 0.5] = 0.0\n        if boundary == "neumann":\n            phi[:, 0] =
phi[:, 1]\n            phi[:, -1] = 0.0\n            phi[0, :] = phi[1, :]\n            phi[-1, :] = phi[-2, :]\n        elif boundary ==
"closed":\n            phi[:, 0] = phi[:, 1]\n            phi[:, -1] = phi[:, -2]\n            phi[0, :] = phi[1, :]\n            phi[-1,
:] = phi[-2, :]\n        phi -= np.mean(phi)\n    else:\n        raise ValueError("unsupported pressure boundary")\n    return
phi\n\n\ndef tdma(a: np.ndarray, b: np.ndarray, c: np.ndarray, d: np.ndarray) -> np.ndarray:\n    a = np.asarray(a, dtype=float).copy()\n
b = np.asarray(b, dtype=float).copy()\n    c = np.asarray(c, dtype=float).copy()\n    d = np.asarray(d, dtype=float).copy()\n    n =
d.size\n    if n == 0 or a.size != n or b.size != n or c.size != n:\n        raise ValueError("TDMA arrays must have equal nonzero size")\n
for i in range(1, n):\n        if abs(b[i - 1]) < 1e-30:\n            raise FloatingPointError("zero TDMA pivot")\n        factor = a[i] /
b[i - 1]\n        b[i] -= factor * c[i - 1]\n        d[i] -= factor * d[i - 1]\n    x = np.empty(n, dtype=float)\n    if abs(b[-1]) < 1e-
30:\n        raise FloatingPointError("zero TDMA pivot")\n    x[-1] = d[-1] / b[-1]\n    for i in range(n - 2, -1, -1):\n        if
abs(b[i]) < 1e-30:\n            raise FloatingPointError("zero TDMA pivot")\n        x[i] = (d[i] - c[i] * x[i + 1]) / b[i]\n    return
x\n\n\ndef relative_l2(numerical: np.ndarray, reference: np.ndarray) -> float:\n    a = np.asarray(numerical, dtype=float)\n    b =
np.asarray(reference, dtype=float)\n    denominator = float(np.linalg.norm(b.ravel()))\n    numerator = float(np.linalg.norm((a -
b).ravel()))\n    return numerator / max(denominator, 1e-30)\n\n\ndef require_finite(*arrays: np.ndarray) -> None:\n    for array in
arrays:\n        if not np.all(np.isfinite(array)):\n            raise FloatingPointError("non-finite numerical field detected")\n\n\ndef
positive_float(value: str, name: str) -> float:\n    result = float(value)\n    if not math.isfinite(result) or result <= 0.0:\n
raise ValueError(f"{name} must be positive and finite")\n    return result\n\n\ndef positive_int(value: str, name: str, minimum: int = 1) -
> int:\n    result = int(value)\n    if result < minimum:\n        raise ValueError(f"{name} must be at least {minimum}")\n    return
result\n', encoding="utf-8")
(CODE_DIR / 'exp08_rayleigh_taylor_vof.py').write_text('from __future__ import annotations\n\nimport csv\nimport math\nimport sys\n\nimport
numpy as np\n\nfrom common import gradient, laplacian, poisson_jacobi, require_finite, upwind_derivative_x, upwind_derivative_y,
write_multiline_vtk, write_points_vtk, write_structured_grid_vtk\n\n\ndef apply_velocity_bc(u: np.ndarray, v: np.ndarray) -> None:\n
u[:, 0] = 0.0\n    u[:, -1] = 0.0\n    u[0, :] = 0.0\n    u[-1, :] = 0.0\n    v[:, 0] = 0.0\n    v[:, -1] = 0.0\n    v[0, :] = 0.0\n    v[-
1, :] = 0.0\n\n\ndef advect_conservative(volume_fraction: np.ndarray, u: np.ndarray, v: np.ndarray, dt: float, dx: float, dy: float) ->
np.ndarray:\n    ny, nx = volume_fraction.shape\n    flux_x = np.zeros((ny, nx + 1), dtype=float)\n    flux_y = np.zeros((ny + 1, nx),
dtype=float)\n    u_face = 0.5 * (u[:, :-1] + u[:, 1:])\n    v_face = 0.5 * (v[:-1, :] + v[1:, :])\n    flux_x[:, 1:nx] = u_face *
np.where(u_face >= 0.0, volume_fraction[:, :-1], volume_fraction[:, 1:])\n    flux_y[1:ny, :] = v_face * np.where(v_face >= 0.0,
```

```
volume_fraction[:-1, :], volume_fraction[1:, :])\n    updated = volume_fraction - dt * ((flux_x[:, 1:] - flux_x[:, :-1]) / dx + (flux_y[1:, :] - flux_y[:-1, :]) / dy)\n    target_sum = float(np.sum(volume_fraction))\n    updated = np.clip(updated, 0.0, 1.0)\n    for _ in range(4):\n        difference = target_sum - float(np.sum(updated))\n        if abs(difference) < 1e-12:\n            break\n        weights = updated * (1.0 - updated) + 1e-6\n        updated += difference * weights / np.sum(weights)\n        updated = np.clip(updated, 0.0, 1.0)\n    return updated\n\n\ndef interface_coordinates(volume_fraction: np.ndarray, x: np.ndarray, y: np.ndarray) -> tuple[np.ndarray, np.ndarray]:\n    indices = np.argmin(np.abs(volume_fraction - 0.5), axis=0)\n    return x[0, :].copy(), y[indices, np.arange(volume_fraction.shape[1])]\n\n\ndef main() -> int:\n    nx = int(sys.argv[1]) if len(sys.argv) > 1 else 96\n    ny = int(sys.argv[2]) if len(sys.argv) > 2 else 192\n    atwood = float(sys.argv[3]) if len(sys.argv) > 3 else 0.5\n    gravity = float(sys.argv[4]) if len(sys.argv) > 4 else 1.0\n    nu = float(sys.argv[5]) if len(sys.argv) > 5 else 0.0008\n    end_time = float(sys.argv[6]) if len(sys.argv) > 6 else 2.0\n    max_steps = int(sys.argv[7]) if len(sys.argv) > 7 else 800\n    poisson_iterations = int(sys.argv[8]) if len(sys.argv) > 8 else 60\n    output_every = int(sys.argv[9]) if len(sys.argv) > 9 else 200\n    if nx < 24 or ny < 48 or not 0.0 < atwood < 0.95 or gravity <= 0.0 or nu <= 0.0 or end_time <= 0.0 or max_steps < 1 or poisson_iterations < 1 or output_every < 1:\n        raise ValueError("usage: python exp08_rayleigh_taylor_vof.py nx ny Atwood gravity nu tEnd maxSteps poissonIterations outputEvery")\n    length = 0.5\n    height = 1.0\n    x1 = np.linspace(0.0, length, nx)\n    y1 = np.linspace(0.0, height, ny)\n    x, y = np.meshgrid(x1, y1)\n    dx = x1[1] - x1[0]\n    dy = y1[1] - y1[0]\n    interface = 0.5 * height + 0.02 * height * np.cos(2.0 * math.pi * x / length)\n    smoothing = 1.5 * dy\n    volume_fraction = 0.5 * (1.0 + np.tanh((y - interface) / smoothing))\n    u = np.zeros_like(volume_fraction)\n    v = np.zeros_like(volume_fraction)\n    p = np.zeros_like(volume_fraction)\n    apply_velocity_bc(u, v)\n    rho_heavy = 1.0 + atwood\n    rho_light = 1.0 - atwood\n    initial_mass = float(np.sum(volume_fraction) * dx * dy)\n    history: list[list[float]] = []\n    time = 0.0\n    completed_steps = 0\n    for step in range(1, max_steps + 1):\n        speed_max = max(float(np.max(np.sqrt(u * u + v * v))), 1e-8)\n        dt_convective = 0.25 * min(dx, dy) / speed_max\n        dt_viscous = 0.15 * min(dx * dx, dy * dy) / nu\n        dt = min(dt_convective, dt_viscous, end_time - time)\n        if dt <= 1e-14:\n            break\n        du_dx = upwind_derivative_x(u, u, dx)\n        du_dy = upwind_derivative_y(u, v, dy)\n        dv_dx = upwind_derivative_x(v, u, dx)\n        dv_dy = upwind_derivative_y(v, v, dy)\n        buoyancy = -gravity * atwood * (2.0 * volume_fraction - 1.0)\n        us = u + dt * (-(u * du_dx + v * du_dy) + nu * laplacian(u, dx, dy))\n        vs = v + dt * (-(u * dv_dx + v * dv_dy) + nu * laplacian(v, dx, dy) + buoyancy)\n        apply_velocity_bc(us, vs)\n        rhs = np.zeros_like(p)\n        rhs[1:-1, 1:-1] = ((us[1:-1, 2:] - us[1:-1, :-2]) / (2.0 * dx) + (vs[2:, 1:-1] - vs[:-2, 1:-1]) / (2.0 * dy)) / dt\n        rhs -= np.mean(rhs)\n        phi = poisson_jacobi(np.zeros_like(p), rhs, dx, dy, poisson_iterations, boundary="closed")\n        gx, gy = gradient(phi, dx, dy)\n        u = us - dt * gx\n        v = vs - dt * gy\n        p += phi\n        apply_velocity_bc(u, v)\n        volume_fraction = advect_conservative(volume_fraction, u, v, dt, dx, dy)\n        time += dt\n        completed_steps = step\n        density = rho_light + (rho_heavy - rho_light) * volume_fraction\n        divergence = np.zeros_like(u)\n        divergence[1:-1, 1:-1] = (u[1:-1, 2:] - u[1:-1, :-2]) / (2.0 * dx) + (v[2:, 1:-1] - v[:-2, 1:-1]) / (2.0 * dy)\n        div_l2 = float(np.sqrt(np.mean(divergence[1:-1, 1:-1] ** 2)))\n        interface_x, interface_y = interface_coordinates(volume_fraction, x, y)\n        spike = float(np.min(interface_y))\n        bubble = float(np.max(interface_y))\n        mixing_width = bubble - spike\n        mass = float(np.sum(volume_fraction) * dx * dy)\n        mass_error = (mass - initial_mass) / max(initial_mass, 1e-30)\n        history.append([step, time, dt, div_l2, spike, bubble, mixing_width, mass, mass_error, float(np.min(volume_fraction)), float(np.max(volume_fraction))])\n        require_finite(u, v, p, volume_fraction)\n        if step % output_every == 0 or time >= end_time - 1e-14 or step == max_steps:\n            vorticity = np.zeros_like(u)\n            vorticity[1:-1, 1:-1] = (v[1:-1, 2:] - v[1:-1, :-2]) / (2.0 * dx) - (u[2:, 1:-1] - u[:-2, 1:-1]) / (2.0 * dy)\n            write_structured_grid_vtk(f"rti_{step:06d}.vtk", x, y, {"heavy_fluid_fraction": volume_fraction, "density": density, "pressure": p, "vorticity": vorticity, "speed": np.sqrt(u * u + v * v)}, {"velocity": (u, v)}, "Rayleigh-Taylor instability")\n            write_points_vtk(f"rti_interface_{step:06d}.vtk", interface_x, interface_y, title="RTI interface")\n            print(f"step={step} time={time:.6e} dt={dt:.6e} divergence_l2={div_l2:.6e} mixing_width={mixing_width:.6e} mass_error={mass_error:.6e}")\n        if time >= end_time - 1e-14:\n            break\n    density = rho_light + (rho_heavy - rho_light) * volume_fraction\n    vorticity = np.zeros_like(u)\n    vorticity[1:-1, 1:-1] = (v[1:-1, 2:] - v[1:-1, :-2]) / (2.0 * dx) - (u[2:, 1:-1] - u[:-2, 1:-1]) / (2.0 * dy)\n    write_structured_grid_vtk("rti_final.vtk", x, y, {"heavy_fluid_fraction": volume_fraction, "density": density, "pressure": p, "vorticity": vorticity, "speed": np.sqrt(u * u + v * v)}, {"velocity": (u, v)}, "Rayleigh-Taylor instability final")\n    interface_x, interface_y = interface_coordinates(volume_fraction, x, y)\n    write_points_vtk("rti_interface_final.vtk", interface_x, interface_y, title="RTI final interface")\n    write_multiline_vtk("rti_domain_boundary.vtk", [(np.array([0.0, length]), np.array([0.0, 0.0])), (np.array([length, length]), np.array([0.0, height])), (np.array([length, 0.0]), np.array([height, height])), (np.array([0.0, height]), np.array([0.0, 0.0]))], "RTI domain boundary")\n    with open("rti_history.csv", "w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(["step", "time", "dt", "divergence_l2", "spike_position", "bubble_position", "mixing_width", "heavy_fluid_mass", "relative_mass_error", "minimum_fraction", "maximum_fraction"])\n        writer.writerows(history)\n    final_mass = float(np.sum(volume_fraction) * dx * dy)\n    with open("rti_summary.txt", "w", encoding="utf-8") as handle:\n        handle.write("Teaching-level algebraic VOF-style Rayleigh-Taylor solver with Boussinesq projection flow\\n")\n        handle.write(f"nx {nx}\\nny {ny}\\nAtwood {atwood:.12e}\\ngravity {gravity:.12e}\\nnu {nu:.12e}\\n")\n        handle.write(f"rho_heavy {rho_heavy:.12e}\\nrho_light {rho_light:.12e}\\ncompleted_steps {completed_steps}\\nfinal_time {time:.12e}\\n")\n        handle.write(f"initial_heavy_fraction_integral {initial_mass:.12e}\\nfinal_heavy_fraction_integral {final_mass:.12e}\\nrelative_mass_error {(final_mass - initial_mass) / max(initial_mass, 1e-30):.12e}\\n")\n        handle.write(f"minimum_volume_fraction {float(np.min(volume_fraction)):.12e}\\nmaximum_volume_fraction {float(np.max(volume_fraction)):.12e}\\n")\n    handle.write("model_scope educational_boussinesq_vof_style_not_industrial_variable_density_vof\\n")\n    print("Wrote rti_final.vtk, rti_interface_final.vtk, rti_domain_boundary.vtk, rti_history.csv, rti_summary.txt")\n    return 0\n\n\nif __name__ == "__main__":\n    raise SystemExit(main())\n', encoding="utf-8")
SCRIPT = CODE_DIR / 'exp08_rayleigh_taylor_vof.py'
print("Runtime:", "Google Colab" if IN_COLAB else "local notebook test")
print("Working folder:", ROOT)
print("No package installation or external file upload is required.")


# ===== NOTEBOOK CODE CELL 2 =====
#@title 2. Select run mode and parameters
RUN_MODE = "custom" #@param ["quick", "lab", "custom"]

nx = 96 #@param {type:"integer"}
ny = 192 #@param {type:"integer"}
Atwood = 0.8 #@param {type:"number"}
gravity = 1.0 #@param {type:"number"}
nu = 0.0008 #@param {type:"number"}
t_end = 2.0 #@param {type:"number"}
max_steps = 800 #@param {type:"integer"}
poisson_iterations = 60 #@param {type:"integer"}
output_every = 200 #@param {type:"integer"}

QUICK_ARGS = [32, 64, 0.2, 1.0, 0.001, 0.02, 5, 5, 5]
LAB_ARGS = [96, 192, 0.2, 1.0, 0.0008, 2.0, 800, 60, 200]
CUSTOM_ARGS = [nx, ny, Atwood, gravity, nu, t_end, max_steps, poisson_iterations, output_every]
ARGS = QUICK_ARGS if RUN_MODE == "quick" else LAB_ARGS if RUN_MODE == "lab" else CUSTOM_ARGS
print("Run mode:", RUN_MODE)
print("Arguments:", ARGS)


# ===== NOTEBOOK CODE CELL 4 =====
#@title 3. Run the numerical solver
OUT = run_solver(SCRIPT, ARGS, OUTPUT_DIR)


# ===== NOTEBOOK CODE CELL 6 =====
#@title 4. Read the solver summary
summary_text = show_summary(OUT / 'rti_summary.txt')
```

```
# ===== NOTEBOOK CODE CELL 8 =====
#@title 5. Plot the primary numerical results
history = read_csv(OUT / "rti_history.csv")
plt.figure(figsize=(7, 4))
plt.plot(history["time"], history["spike_position"], label="Spike")
plt.plot(history["time"], history["bubble_position"], label="Bubble")
plt.xlabel("Time")
plt.ylabel("Vertical position")
plt.title("Spike and bubble evolution")
plt.legend()
plt.grid(True)
plt.show()

plt.figure(figsize=(7, 4))
plt.plot(history["time"], history["mixing_width"])
plt.xlabel("Time")
plt.ylabel("Mixing width")
plt.title("Mixing-layer growth")
plt.grid(True)
plt.show()

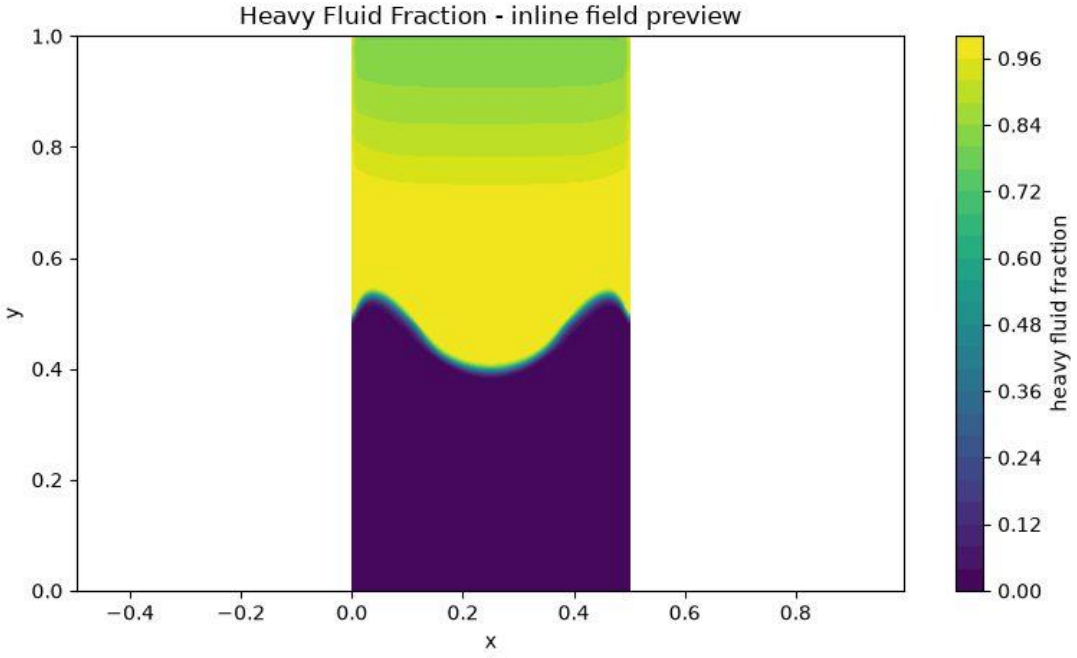
plt.figure(figsize=(7, 4))
plt.semilogy(history["time"], np.maximum(np.abs(history["relative_mass_error"]), 1e-18))
plt.xlabel("Time")
plt.ylabel("Absolute relative mass error")
plt.title("Volume-fraction conservation")
plt.grid(True)
plt.show()

# ===== NOTEBOOK CODE CELL 10 =====
#@title 6. Display the final field and generated files
plot_vtk_scalar(OUT / 'rti_final.vtk', 'heavy_fluid_fraction')
files = list_outputs(OUT)

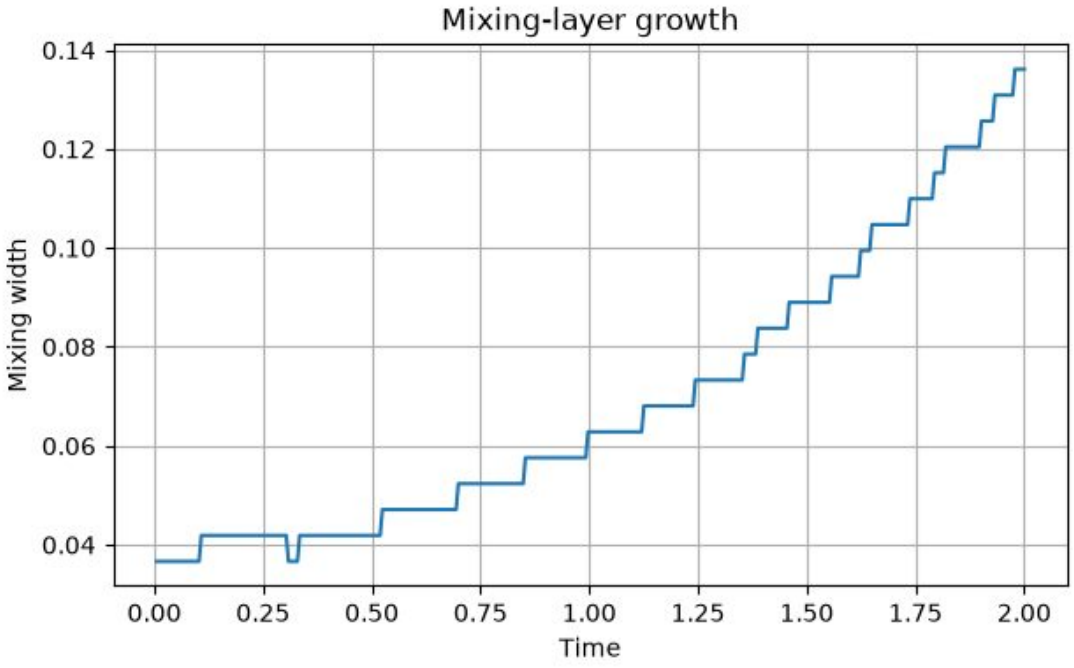

# ===== NOTEBOOK CODE CELL 12 =====
#@title 7. Package and download the results
DOWNLOAD_RESULTS = False #@param {type:"boolean"}
RESULTS_ZIP = make_results_zip(OUT, 8)
if DOWNLOAD_RESULTS:
    if IN_COLAB:
        colab_files.download(str(RESULTS_ZIP))
    else:
        print("Local test mode. Archive retained at:", RESULTS_ZIP)
else:
    print("Set DOWNLOAD_RESULTS to True and run this cell when ready to download.")
```

Output

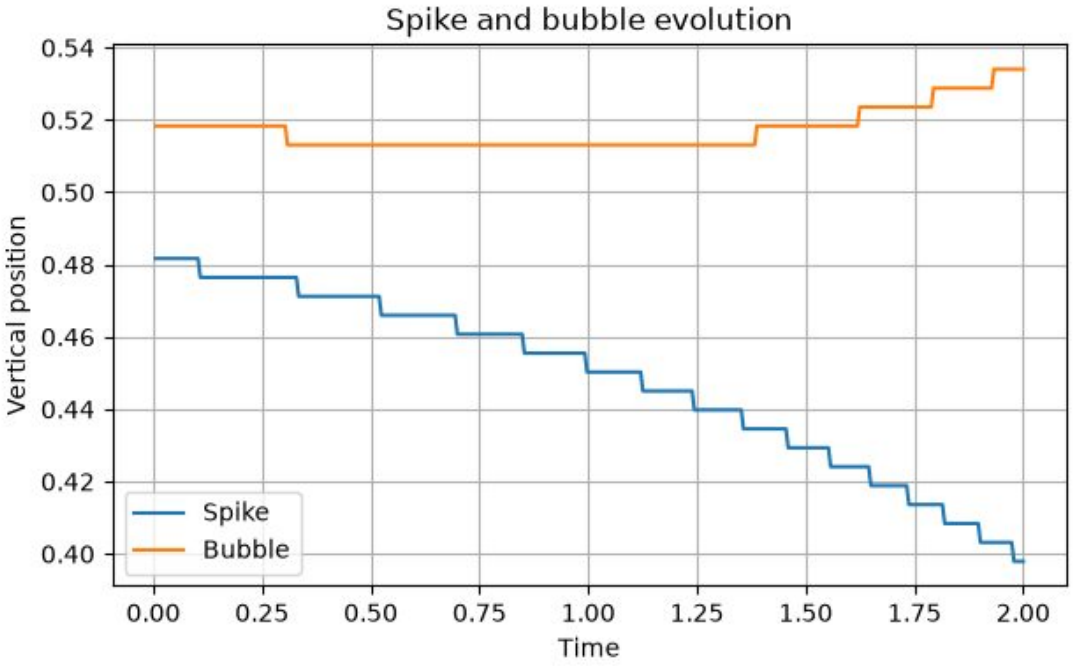
The following figures are reproduced from the output folder supplied for this experiment.



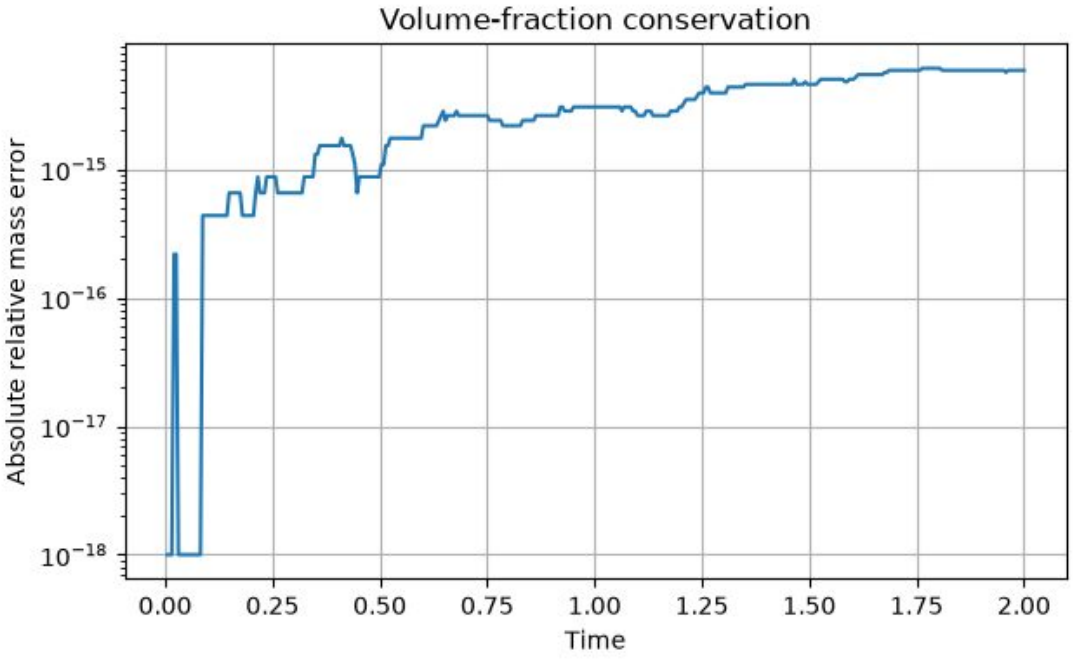
Experiment 8 output: Atwood 0.2/Heavy Fluid Fraction



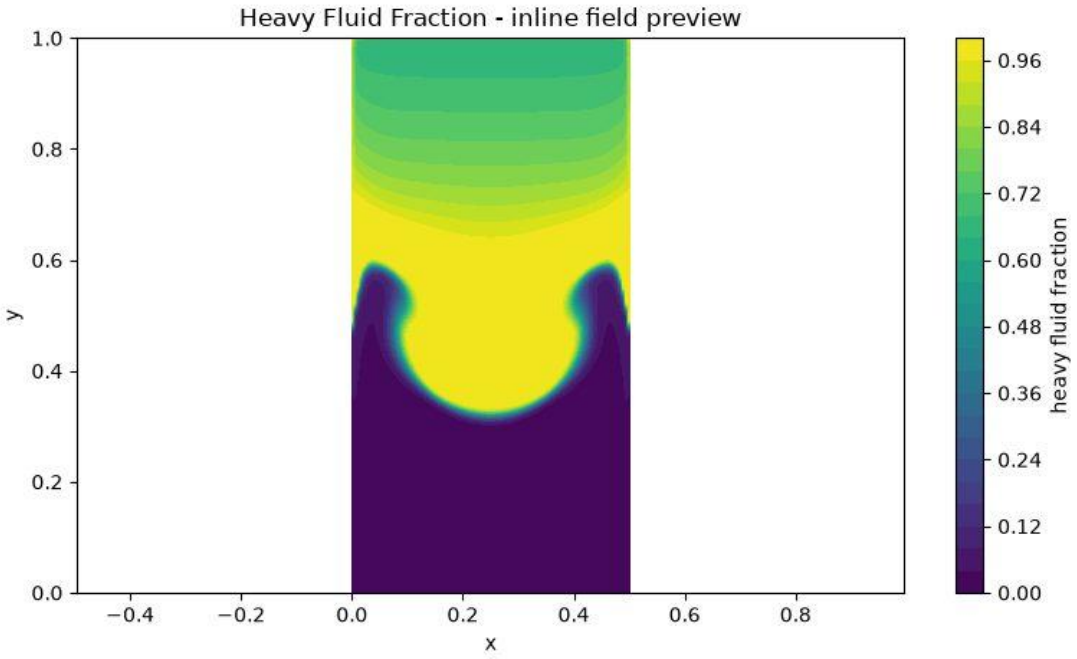
Experiment 8 output: Atwood 0.2/Mixing Layer Growth



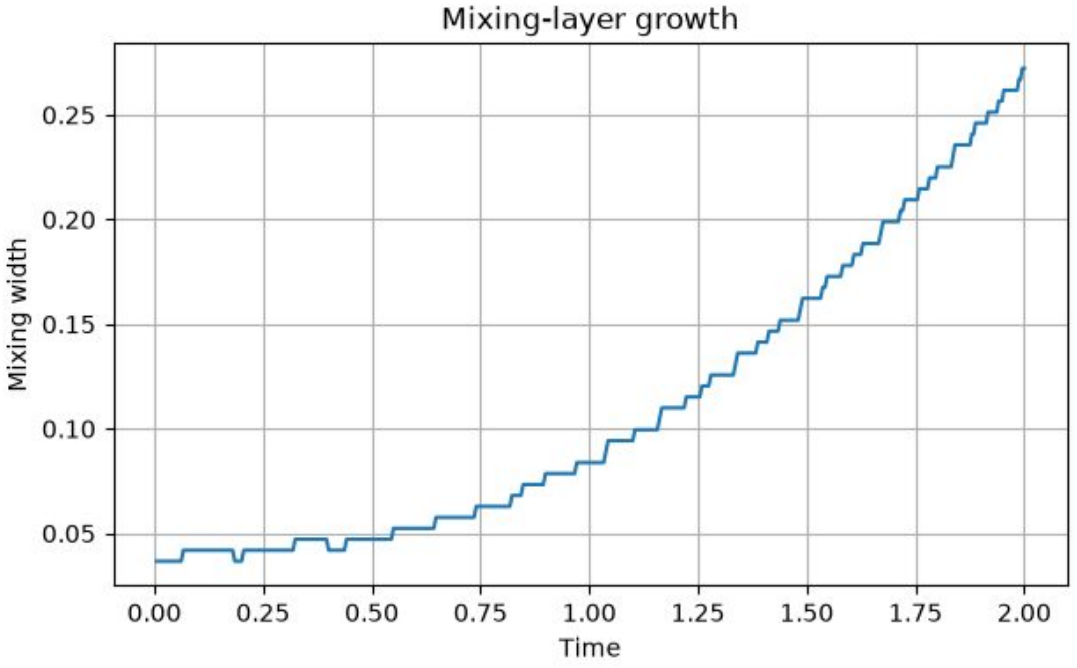
Experiment 8 output: Atwood 0.2/Spike and Bubble Evolution



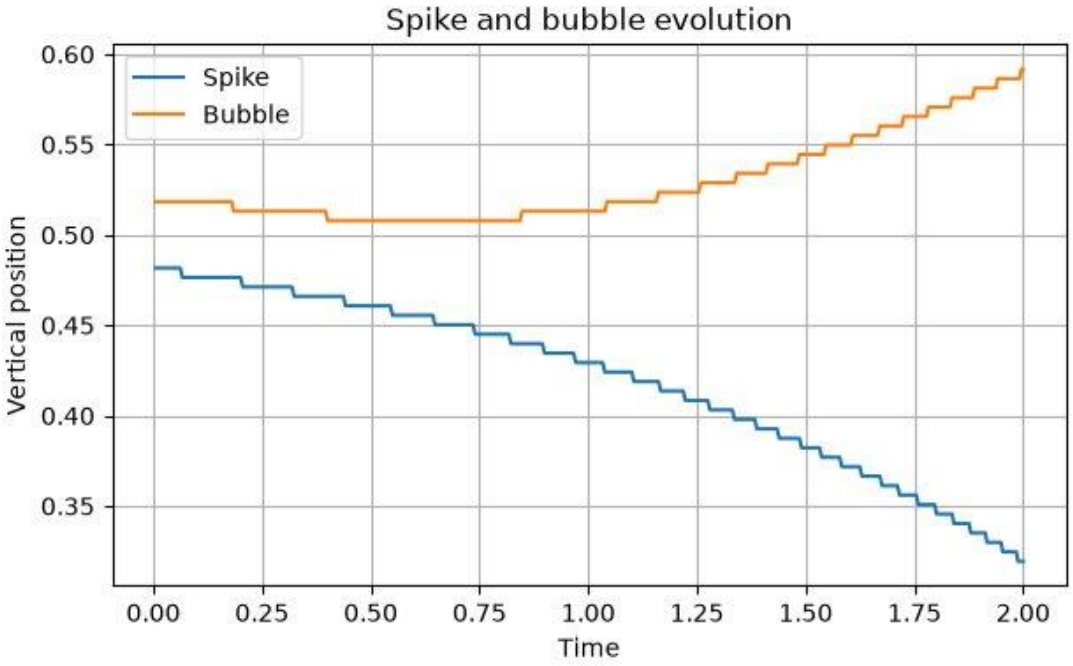
Experiment 8 output: Atwood 0.2/Volume Fraction Conservation



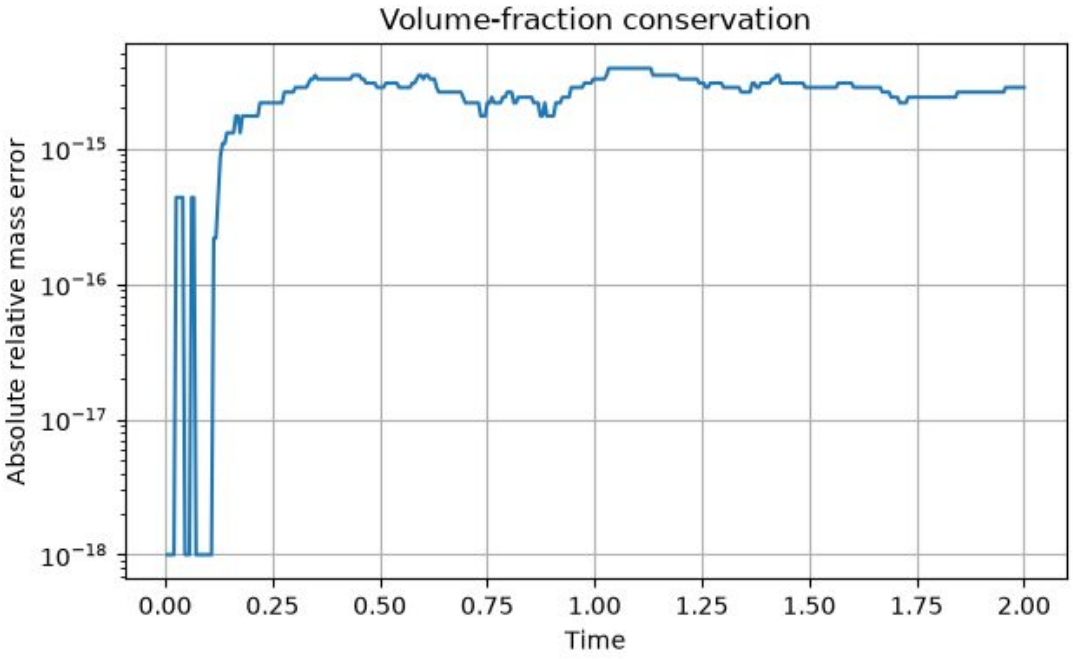
Experiment 8 output: Atwood 0.5/Heavy Fluid Fraction



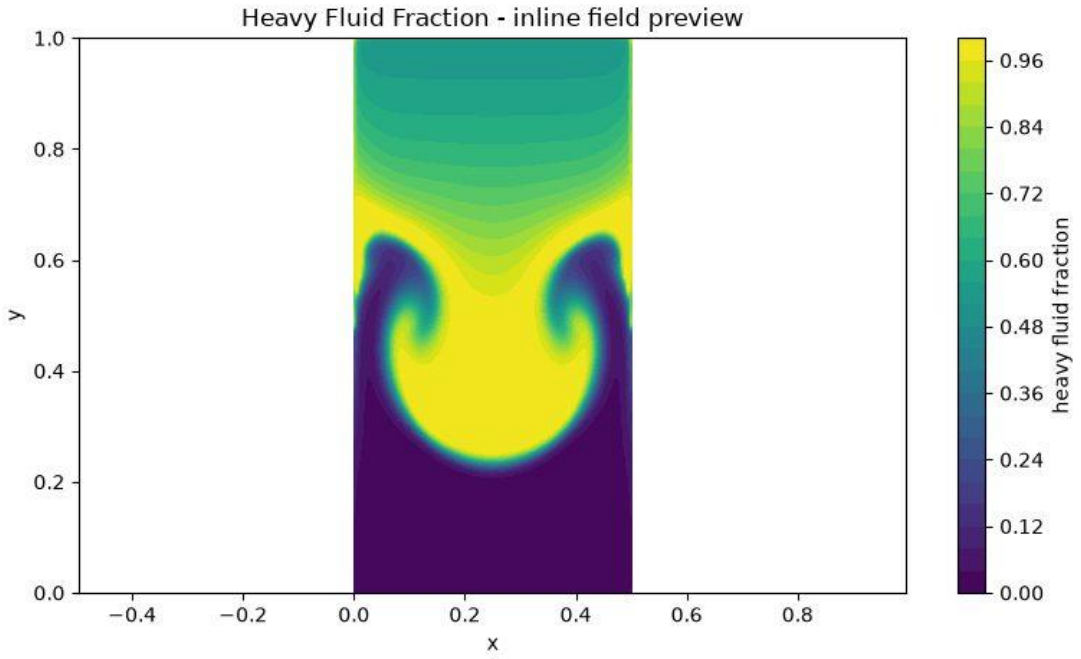
Experiment 8 output: Atwood 0.5/Mixing Layer Growth



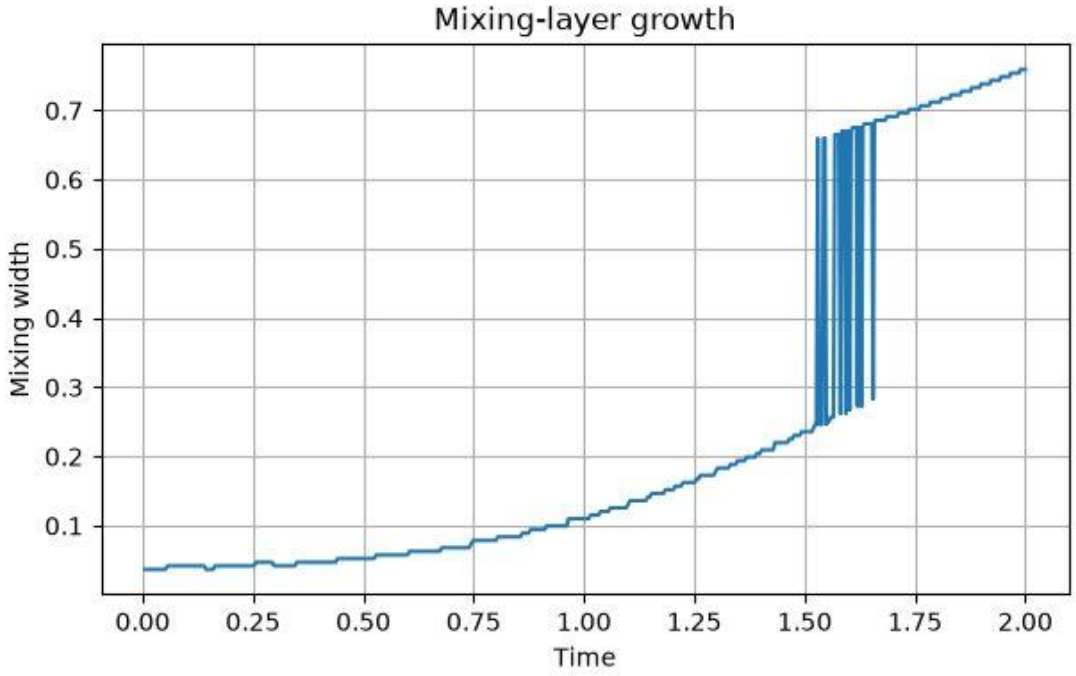
Experiment 8 output: Atwood 0.5/Spike and Bubble Evolution



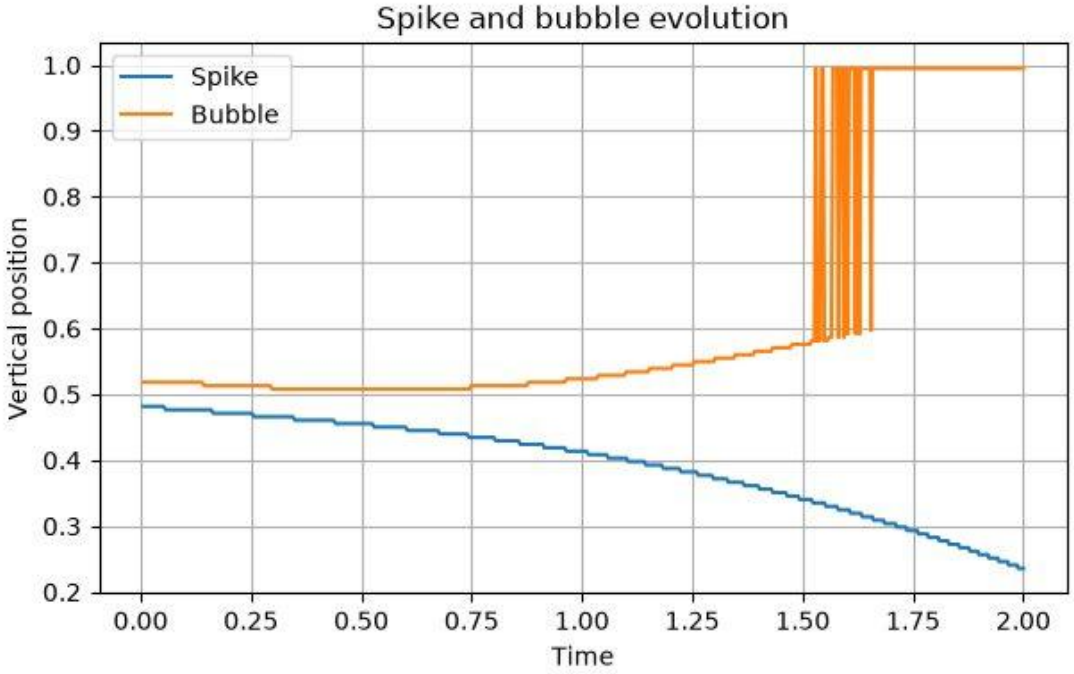
Experiment 8 output: Atwood 0.5/Volume Fraction



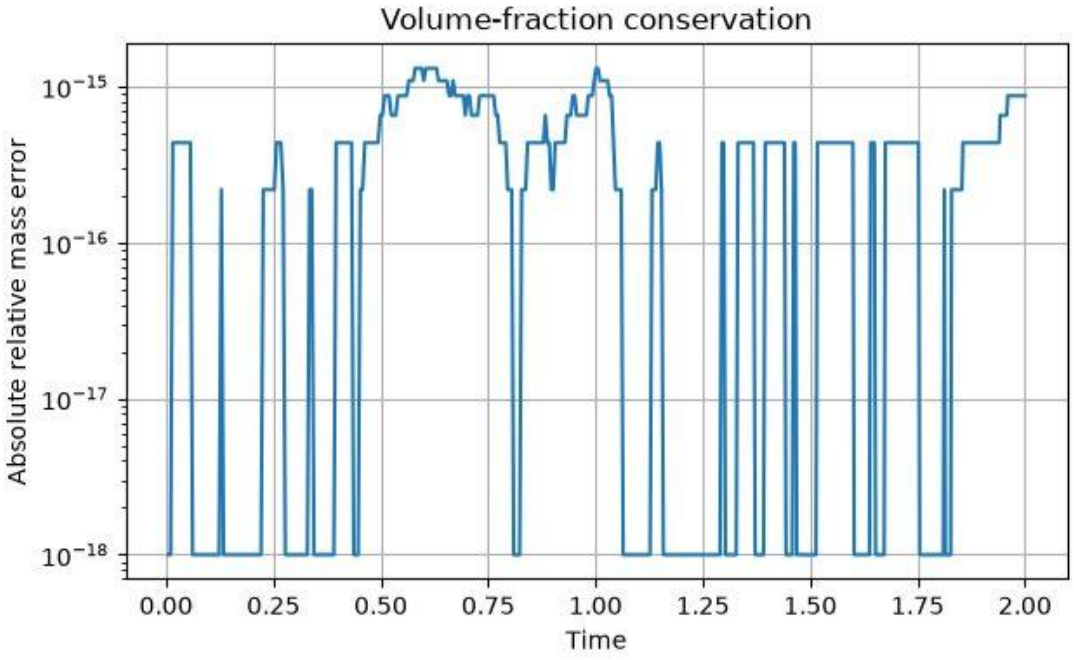
Experiment 8 output: Atwood 0.8/Heavy Fluid Fraction



Experiment 8 output: Atwood 0.8/Mixing Layer Growth



Experiment 8 output: Atwood 0.8/Spike and Bubble Evolution



Experiment 8 output: Atwood 0.8/Volume Fraction Conservation

Results and Discussion

The archive contains heavy-fluid-fraction fields, spike/bubble evolution, mixing-layer growth and volume-fraction conservation plots for the three Atwood cases. The Atwood number controls the density contrast; increasing it increases the buoyancy contrast in the model. Compare the actual saved curves and fraction-conservation plots for quantitative conclusions. The volume-fraction field should remain bounded between zero and one; any departures or mass drift should be recorded from the plots.

Conclusion

The experiment visualises the growth of a perturbed heavy–light fluid interface and provides a comparison of the three density-contrast cases. The model is suitable for qualitative teaching comparisons; quantitative interpretation is limited by the simplified VOF and flow treatment.

EXPERIMENT 9 – Lid-driven cavity flow

Aim

To simulate two-dimensional incompressible flow in a square cavity driven by a moving top wall and compare the flow field at Reynolds numbers 100 and 400.

Objective

- Solve the streamfunction–vorticity form of the incompressible Navier–Stokes equations.
- Apply a moving-lid velocity at the top and stationary no-slip walls on the other sides.
- Obtain the primary recirculation vortex and centreline velocity distributions.
- Compare convergence and secondary-corner-vortex development at Re=100 and Re=400.

Governing Equation

Streamfunction definition: $u=\partial\psi/\partial y$, $v=-\partial\psi/\partial x$.

Vorticity: $\omega=\partial v/\partial x-\partial u/\partial y=-\nabla^2\psi$.

Vorticity transport:

$\partial\omega/\partial t + u\partial\omega/\partial x + v\partial\omega/\partial y = (1/Re)\nabla^2\omega$.

The streamfunction Poisson equation is solved iteratively using SOR; the vorticity equation is advanced in time.

Boundary Condition

- Top moving wall: u=U_lid=1, v=0.
- Bottom, left and right walls: no-slip stationary walls, u=v=0.
- Streamfunction: ψ=0 on all cavity walls.
- Vorticity at the walls is computed from the streamfunction and wall-velocity treatment in the solver.

Code

Parameters: nx=65, ny=65, Re=100 and 400, U_lid=1.0, max_steps=2000, output_every=500, poisson_iterations=60, tolerance=10^{-8}, CFL=0.8 and SOR=1.7.

```
# ===== NOTEBOOK CODE CELL 0 =====
from pathlib import Path
import shutil
import subprocess
import sys
import zipfile
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Markdown, display

IN_COLAB = False
try:
    from google.colab import files as colab_files
    IN_COLAB = True
except ImportError:
    colab_files = None


def run_solver(script_path, arguments, output_dir, timeout=1800):
    output_dir = Path(output_dir)
    if output_dir.exists():
        shutil.rmtree(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    command = [sys.executable, str(script_path), *[str(value) for value in arguments]]
    print("Running:", " ".join(command))
    completed = subprocess.run(command, cwd=output_dir, capture_output=True, text=True, timeout=timeout)
    (output_dir / "solver_stdout.log").write_text(completed.stdout, encoding="utf-8")
    (output_dir / "solver_stderr.log").write_text(completed.stderr, encoding="utf-8")
    if completed.stdout.strip():
        print(completed.stdout.strip())
    if completed.returncode != 0:
        raise RuntimeError(completed.stderr.strip() or f"Solver exited with code {completed.returncode}")
    if completed.stderr.strip():
        print(completed.stderr.strip())
    return output_dir
```

```
def read_csv(path):
    data = np.genfromtxt(path, delimiter=",", names=True, dtype=None, encoding="utf-8")
    return np.atleast_1d(data)


def show_summary(path):
    text = Path(path).read_text(encoding="utf-8").strip()
    display(Markdown(f"### Solver summary\n```text\n{text}\n```"))
    return text


def list_outputs(output_dir):
    rows = []
    for path in sorted(Path(output_dir).iterdir()):
        if path.is_file():
            size = path.stat().st_size
            if size < 1024:
                size_text = f"{size} B"
            elif size < 1024**2:
                size_text = f"{size / 1024:.1f} KB"
            else:
                size_text = f"{size / 1024**2:.1f} MB"
            rows.append((path.name, size_text))
    table = "\n".join(f"| `{name}` | {size} |" for name, size in rows)
    display(Markdown(f"### Generated files\n| File | Size |\n|---|---:|\n{table}"))
    return rows


def read_structured_vtk(path):
    lines = Path(path).read_text(encoding="utf-8").splitlines()
    dims_index = next(i for i, line in enumerate(lines) if line.startswith("DIMENSIONS "))
    _, nx, ny, nz = lines[dims_index].split()
    nx, ny, nz = int(nx), int(ny), int(nz)
    if nz != 1:
        raise ValueError("Only 2D structured VTK files are supported by this notebook preview")
    points_index = next(i for i, line in enumerate(lines) if line.startswith("POINTS "))
    npoints = int(lines[points_index].split()[1])
    points = np.array([[float(v) for v in lines[points_index + 1 + k].split()[:3]] for k in range(npoints)])
    x = points[:, 0].reshape(ny, nx)
    y = points[:, 1].reshape(ny, nx)
    data_index = next(i for i, line in enumerate(lines) if line.startswith("POINT_DATA ")) + 1
    scalars = {}
    vectors = {}
    i = data_index
    while i < len(lines):
        if lines[i].startswith("SCALARS "):
            name = lines[i].split()[1]
            i += 2
            values = np.array([float(lines[i + k].split()[0]) for k in range(npoints)])
            scalars[name] = values.reshape(ny, nx)
            i += npoints
        elif lines[i].startswith("VECTORS "):
            name = lines[i].split()[1]
            i += 1
            values = np.array([[float(v) for v in lines[i + k].split()[:3]] for k in range(npoints)])
            vectors[name] = (values[:, 0].reshape(ny, nx), values[:, 1].reshape(ny, nx))
            i += npoints
        else:
            i += 1
    return x, y, scalars, vectors


def plot_vtk_scalar(path, preferred_field=None):
    path = Path(path)
    if not path.is_file():
        print(f"VTK preview skipped: {path.name} was not generated")
        return
    x, y, scalars, _ = read_structured_vtk(path)
    if not scalars:
        print("VTK preview skipped: no scalar field found")
        return
    field = preferred_field if preferred_field in scalars else next(iter(scalars))
    plt.figure(figsize=(8, 4.8))
    image = plt.contourf(x, y, scalars[field], levels=30)
    plt.colorbar(image, label=field.replace("_", " "))
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title(f"{field.replace('_', ' ').title()} - inline field preview")
    plt.axis("equal")
    plt.tight_layout()
    plt.show()


def make_results_zip(output_dir, experiment_number):
    output_dir = Path(output_dir)
    archive = output_dir.parent / f"experiment_{experiment_number:02d}_results.zip"
    if archive.exists():
        archive.unlink()
    with zipfile.ZipFile(archive, "w", compression=zipfile.ZIP_DEFLATED) as handle:
        for path in sorted(output_dir.rglob("*")):
            if path.is_file():
                handle.write(path, path.relative_to(output_dir))
    print(f"Results archive: {archive} ({archive.stat().st_size / 1024:.1f} KB)")
    return archive


EXPERIMENT = 9
```

```
ROOT = Path("/content/cfd_lab_exp09") if IN_COLAB else Path.home() / "CFD_Runs" / "exp09"
CODE_DIR = ROOT / "code"
OUTPUT_DIR = ROOT / "outputs"
CODE_DIR.mkdir(parents=True, exist_ok=True)
(CODE_DIR / "common.py").write_text('from __future__ import annotations\n\nimport csv\nimport math\nfrom pathlib import Path\nfrom typing import Iterable, Mapping\n\nimport numpy as np\n\n\ndef trapezoidal_integral(values: np.ndarray, coordinates: np.ndarray) -> float:\n    y = np.asarray(values, dtype=float).ravel()\n    x = np.asarray(coordinates, dtype=float).ravel()\n    if y.size != x.size or y.size < 2:\n raise ValueError("trapezoidal integration requires equally sized arrays with at least two points")\n    return float(np.sum(0.5 * (y[1:] + y[:-1]) * np.diff(x)))\n\n\ndef write_csv(path: str | Path, header: Iterable[str], rows: Iterable[Iterable[object]]) -> None:\n    with Path(path).open("w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(list(header))\n writer.writerows(rows)\n\n\ndef mesh_xy(nx: int, ny: int, lx: float = 1.0, ly: float = 1.0, x0: float = 0.0, y0: float = 0.0) -> tuple[np.ndarray, np.ndarray]:\n    x = np.linspace(x0, x0 + lx, nx)\n    y = np.linspace(y0, y0 + ly, ny)\n    return np.meshgrid(x, y)\n\n\ndef write_structured_grid_vtk(\n    path: str | Path,\n    x: np.ndarray,\n    y: np.ndarray,\n    scalars: Mapping[str, np.ndarray] | None = None,\n    vectors: Mapping[str, tuple[np.ndarray, np.ndarray]] | None = None,\n    title: str = "CFD lab output",\n) -> None:\n    x = np.asarray(x, dtype=float)\n    y = np.asarray(y, dtype=float)\n    if x.shape != y.shape or x.ndim != 2:\n raise ValueError("x and y must be equally shaped 2D arrays")\n    ny, nx = x.shape\n    scalar_map = dict(scalars or {})\n    vector_map = dict(vectors or {})\n    for name, values in scalar_map.items():\n        if np.asarray(values).shape != x.shape:\n raise ValueError(f"scalar {name} has shape {np.asarray(values).shape}, expected {x.shape}")\n    for name, values in vector_map.items():\n if len(values) != 2 or np.asarray(values[0]).shape != x.shape or np.asarray(values[1]).shape != x.shape:\n raise ValueError(f"vector {name} has incompatible shape")\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET STRUCTURED_GRID\\n")\n        handle.write(f"DIMENSIONS {nx} {ny} 1\\n")\n        handle.write(f"POINTS {nx * ny} float\\n")\n        for j in range(ny):\n            for i in range(nx):\n                handle.write(f"{x[j, i]:.12e} {y[j, i]:.12e} 0\\n")\n handle.write(f"POINT_DATA {nx * ny}\\n")\n        for name, values in scalar_map.items():\n            data = np.asarray(values, dtype=float)\n            handle.write(f"SCALARS {name} float 1\\n")\n            handle.write("LOOKUP_TABLE default\\n")\n            for value in data.ravel(order="C"):\n                handle.write(f"{value:.12e}\\n")\n        for name, values in vector_map.items():\n vx = np.asarray(values[0], dtype=float)\n            vy = np.asarray(values[1], dtype=float)\n            handle.write(f"VECTORS {name} float\\n")\n            for ax, ay in zip(vx.ravel(order="C"), vy.ravel(order="C")):\n                handle.write(f"{ax:.12e} {ay:.12e} 0\\n")\n\n\ndef write_polyline_vtk(path: str | Path, x: np.ndarray, y: np.ndarray, title: str = "polyline", close: bool = False) -> None:\n x = np.asarray(x, dtype=float).ravel()\n    y = np.asarray(y, dtype=float).ravel()\n    if x.size != y.size or x.size < 2:\n raise ValueError("polyline requires equally sized x and y arrays")\n    ids = list(range(x.size))\n    if close:\n        ids.append(0)\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET POLYDATA\\n")\n        handle.write(f"POINTS {x.size} float\\n")\n        for xi, yi in zip(x, y):\n            handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n        handle.write(f"LINES 1 {len(ids) + 1}\\n")\n        handle.write(f"{len(ids)} {\' \'.join(str(i) for i in ids)}\\n")\n\n\ndef write_multiline_vtk(path: str | Path, lines: list[tuple[np.ndarray, np.ndarray]], title: str = "lines") -> None:\n    points: list[tuple[float, float]] = []\n connectivity: list[list[int]] = []\n    for x, y in lines:\n        xa = np.asarray(x, dtype=float).ravel()\n        ya = np.asarray(y, dtype=float).ravel()\n        if xa.size != ya.size or xa.size < 2:\n            raise ValueError("each line needs equally sized x and y arrays")\n        ids = list(range(len(points), len(points) + xa.size))\n        points.extend(zip(xa.tolist(), ya.tolist()))\n connectivity.append(ids)\n    size = sum(len(ids) + 1 for ids in connectivity)\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n handle.write("DATASET POLYDATA\\n")\n        handle.write(f"POINTS {len(points)} float\\n")\n        for xi, yi in points:\n handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n        handle.write(f"LINES {len(connectivity)} {size}\\n")\n        for ids in connectivity:\n handle.write(f"{len(ids)} {\' \'.join(str(i) for i in ids)}\\n")\n\n\ndef write_points_vtk(path: str | Path, x: np.ndarray, y: np.ndarray, scalars: Mapping[str, np.ndarray] | None = None, title: str = "points") -> None:\n    xa = np.asarray(x, dtype=float).ravel()\n    ya = np.asarray(y, dtype=float).ravel()\n    if xa.size != ya.size:\n        raise ValueError("point arrays must have equal size")\n scalar_map = dict(scalars or {})\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET POLYDATA\\n")\n handle.write(f"POINTS {xa.size} float\\n")\n        for xi, yi in zip(xa, ya):\n            handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n handle.write(f"VERTICES {xa.size} {2 * xa.size}\\n")\n        for i in range(xa.size):\n            handle.write(f"1 {i}\\n")\n        if scalar_map:\n            handle.write(f"POINT_DATA {xa.size}\\n")\n            for name, values in scalar_map.items():\n data = np.asarray(values, dtype=float).ravel()\n                if data.size != xa.size:\n                    raise ValueError(f"scalar {name} has wrong size")\n                handle.write(f"SCALARS {name} float 1\\n")\n                handle.write("LOOKUP_TABLE default\\n")\n                for value in data:\n                    handle.write(f"{value:.12e}\\n")\n\n\ndef gradient(field: np.ndarray, dx: float, dy: float) -> tuple[np.ndarray, np.ndarray]:\n    gy, gx = np.gradient(field, dy, dx, edge_order=2)\n    return gx, gy\n\n\ndef laplacian(field: np.ndarray, dx: float, dy: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    out[1:-1, 1:-1] = (\n (field[1:-1, 2:] - 2.0 * field[1:-1, 1:-1] + field[1:-1, :-2]) / (dx * dx)\n        + (field[2:, 1:-1] - 2.0 * field[1:-1, 1:-1] + field[:-2, 1:-1]) / (dy * dy)\n    )\n    return out\n\n\ndef upwind_derivative_x(field: np.ndarray, velocity: np.ndarray, dx: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    backward = (field[1:-1, 1:-1] - field[1:-1, :-2]) / dx\n    forward = (field[1:-1, 2:] - field[1:-1, 1:-1]) / dx\n    out[1:-1, 1:-1] = np.where(velocity[1:-1, 1:-1] >= 0.0, backward, forward)\n    return out\n\n\ndef upwind_derivative_y(field: np.ndarray, velocity: np.ndarray, dy: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    backward = (field[1:-1, 1:-1] - field[:-2, 1:-1]) / dy\n    forward = (field[2:, 1:-1] - field[1:-1, 1:-1]) / dy\n    out[1:-1, 1:-1] = np.where(velocity[1:-1, 1:-1] >= 0.0, backward, forward)\n    return out\n\n\ndef poisson_jacobi(phi: np.ndarray, rhs: np.ndarray, dx: float, dy: float, iterations: int, boundary: str = "neumann", mask: np.ndarray | None = None) -> np.ndarray:\n    dx2 = dx * dx\n    dy2 = dy * dy\n    denom = 2.0 * (dx2 + dy2)\n    for _ in range(iterations):\n        old = phi.copy()\n        phi[1:-1, 1:-1] = (\n (old[1:-1, 2:] + old[1:-1, :-2]) * dy2\n            + (old[2:, 1:-1] + old[:-2, 1:-1]) * dx2\n            - rhs[1:-1, 1:-1] * dx2 * dy2\n ) / denom\n        if mask is not None:\n            phi[mask > 0.5] = 0.0\n        if boundary == "neumann":\n            phi[:, 0] = phi[:, 1]\n            phi[:, -1] = 0.0\n            phi[0, :] = phi[1, :]\n            phi[-1, :] = phi[-2, :]\n        elif boundary == "closed":\n            phi[:, 0] = phi[:, 1]\n            phi[:, -1] = phi[:, -2]\n            phi[0, :] = phi[1, :]\n            phi[-1, :] = phi[-2, :]\n        phi -= np.mean(phi)\n    else:\n        raise ValueError("unsupported pressure boundary")\n    return phi\n\n\ndef tdma(a: np.ndarray, b: np.ndarray, c: np.ndarray, d: np.ndarray) -> np.ndarray:\n    a = np.asarray(a, dtype=float).copy()\n b = np.asarray(b, dtype=float).copy()\n    c = np.asarray(c, dtype=float).copy()\n    d = np.asarray(d, dtype=float).copy()\n    n = d.size\n    if n == 0 or a.size != n or b.size != n or c.size != n:\n        raise ValueError("TDMA arrays must have equal nonzero size")\n for i in range(1, n):\n        if abs(b[i - 1]) < 1e-30:\n            raise FloatingPointError("zero TDMA pivot")\n        factor = a[i] / b[i - 1]\n        b[i] -= factor * c[i - 1]\n        d[i] -= factor * d[i - 1]\n    x = np.empty(n, dtype=float)\n    if abs(b[-1]) < 1e-30:\n raise FloatingPointError("zero TDMA pivot")\n    x[-1] = d[-1] / b[-1]\n    for i in range(n - 2, -1, -1):\n        if abs(b[i]) < 1e-30:\n            raise FloatingPointError("zero TDMA pivot")\n        x[i] = (d[i] - c[i] * x[i + 1]) / b[i]\n    return x\n\n\ndef relative_l2(numerical: np.ndarray, reference: np.ndarray) -> float:\n    a = np.asarray(numerical, dtype=float)\n    b = np.asarray(reference, dtype=float)\n    denominator = float(np.linalg.norm(b.ravel()))\n    numerator = float(np.linalg.norm((a - b).ravel()))\n    return numerator / max(denominator, 1e-30)\n\n\ndef require_finite(*arrays: np.ndarray) -> None:\n    for array in arrays:\n        if not np.all(np.isfinite(array)):\n            raise FloatingPointError("non-finite numerical field detected")\n\n\ndef positive_float(value: str, name: str) -> float:\n    result = float(value)\n    if not math.isfinite(result) or result <= 0.0:\n raise ValueError(f"{name} must be positive and finite")\n    return result\n\n\ndef positive_int(value: str, name: str, minimum: int = 1) -> int:\n    result = int(value)\n    if result < minimum:\n        raise ValueError(f"{name} must be at least {minimum}")\n    return result\n', encoding="utf-8")
(CODE_DIR / 'exp09_lid_cavity_psi_omega.py').write_text('from __future__ import annotations\n\nimport csv\nimport sys\n\nimport numpy as np\n\nfrom common import laplacian, require_finite, upwind_derivative_x, upwind_derivative_y, write_multiline_vtk, write_points_vtk, write_structured_grid_vtk\n\n\ndef solve_streamfunction(psi: np.ndarray, omega_field: np.ndarray, dx: float, dy: float, iterations: int, sor: float) -> float:\n    dx2 = dx * dx\n    dy2 = dy * dy\n    denominator = 2.0 / dx2 + 2.0 / dy2\n    residual = 0.0\n    for _ in range(iterations):\n        residual = 0.0\n        for parity in (0, 1):\n            j_indices, i_indices = np.indices((psi.shape[0] - 2, psi.shape[1] - 2))\n            active = ((i_indices + j_indices) % 2) == parity\n            candidate = (\n                (psi[1:-1, 2:] + psi[1:-1, :-2]) / dx2\n                + (psi[2:, 1:-1] + psi[:-2, 1:-1]) / dy2\n                + omega_field[1:-1, 1:-1]\n            ) / denominator\n            old = psi[1:-1, 1:-1].copy()\n            updated = np.where(active, (1.0 - sor) * old + sor * candidate, old)\n psi[1:-1, 1:-1] = updated\n            residual = max(residual, float(np.max(np.abs(updated - old))))\n        psi[0, :] = 0.0\n psi[-1, :] = 0.0\n        psi[:, 0] = 0.0\n        psi[:, -1] = 0.0\n    return residual\n\n\ndef velocity_from_streamfunction(psi: np.ndarray, dx: float, dy: float, lid_speed: float) -> tuple[np.ndarray, np.ndarray]:\n    u = np.zeros_like(psi)\n    v = np.zeros_like(psi)\n    u[1:-1, 1:-1] = (psi[2:, 1:-1] - psi[:-2, 1:-1]) / (2.0 * dy)\n    v[1:-1, 1:-1] = -(psi[1:-1, 2:] - psi[1:-1, :-2]) / (2.0 * dx)\n    u[-1, :] = lid_speed\n    u[0, :] = 0.0\n    u[:, 0] = 0.0\n    u[:, -1] = 0.0\n    v[0, :] = 0.0\n    v[-1, :] = 0.0\n    v[:, 0] = 0.0\n    v[:, -1] = 0.0\n    return u, v\n\n\ndef set_wall_vorticity(omega_field: np.ndarray, psi: np.ndarray, dx:
```

```
float, dy: float, lid_speed: float) -> None:\n    omega_field[0, 1:-1] = -2.0 * psi[1, 1:-1] / (dy * dy)\n    omega_field[-1, 1:-1] = -2.0
* psi[-2, 1:-1] / (dy * dy) - 2.0 * lid_speed / dy\n    omega_field[1:-1, 0] = -2.0 * psi[1:-1, 1] / (dx * dx)\n    omega_field[1:-1, -1] =
-2.0 * psi[1:-1, -2] / (dx * dx)\n    omega_field[0, 0] = 0.5 * (omega_field[0, 1] + omega_field[1, 0])\n    omega_field[0, -1] = 0.5 *
(omega_field[0, -2] + omega_field[1, -1])\n    omega_field[-1, 0] = 0.5 * (omega_field[-1, 1] + omega_field[-2, 0])\n    omega_field[-1, -
1] = 0.5 * (omega_field[-1, -2] + omega_field[-2, -1])\n\n\ndef main() -> int:\n    nx = int(sys.argv[1]) if len(sys.argv) > 1 else 65\n
ny = int(sys.argv[2]) if len(sys.argv) > 2 else 65\n    reynolds = float(sys.argv[3]) if len(sys.argv) > 3 else 100.0\n    lid_speed =
float(sys.argv[4]) if len(sys.argv) > 4 else 1.0\n    max_steps = int(sys.argv[5]) if len(sys.argv) > 5 else 2000\n    output_every =
int(sys.argv[6]) if len(sys.argv) > 6 else 500\n    poisson_iterations = int(sys.argv[7]) if len(sys.argv) > 7 else 60\n    tolerance =
float(sys.argv[8]) if len(sys.argv) > 8 else 1e-8\n    cfl = float(sys.argv[9]) if len(sys.argv) > 9 else 0.8\n    sor =
float(sys.argv[10]) if len(sys.argv) > 10 else 1.7\n    if nx < 11 or ny < 11 or reynolds <= 0.0 or lid_speed <= 0.0 or max_steps < 1 or
output_every < 1 or poisson_iterations < 1 or tolerance <= 0.0 or not 0.0 < sor < 2.0:\n        raise ValueError("usage: python
exp09_lid_cavity_psi_omega.py nx ny Re U maxSteps outputEvery poissonIterations tolerance CFL SOR")\n    x1 = np.linspace(0.0, 1.0, nx)\n
y1 = np.linspace(0.0, 1.0, ny)\n    x, y = np.meshgrid(x1, y1)\n    dx = x1[1] - x1[0]\n    dy = y1[1] - y1[0]\n    nu = lid_speed /
reynolds\n    dt = min(0.25 * cfl * min(dx, dy) / lid_speed, 0.05 * min(dx * dx, dy * dy) / nu)\n    psi = np.zeros((ny, nx),
dtype=float)\n    omega_field = np.zeros_like(psi)\n    u = np.zeros_like(psi)\n    v = np.zeros_like(psi)\n    history: list[list[float]]
= []\n    completed_steps = 0\n    for step in range(1, max_steps + 1):\n        poisson_residual = solve_streamfunction(psi, omega_field,
dx, dy, poisson_iterations, sor)\n        u, v = velocity_from_streamfunction(psi, dx, dy, lid_speed)\n
set_wall_vorticity(omega_field, psi, dx, dy, lid_speed)\n        old = omega_field.copy()\n        convection = u *
upwind_derivative_x(old, u, dx) + v * upwind_derivative_y(old, v, dy)\n        omega_field[1:-1, 1:-1] = old[1:-1, 1:-1] + dt * (-
convection[1:-1, 1:-1] + nu * laplacian(old, dx, dy)[1:-1, 1:-1])\n        set_wall_vorticity(omega_field, psi, dx, dy, lid_speed)\n
change = float(np.sqrt(np.mean((omega_field[1:-1, 1:-1] - old[1:-1, 1:-1]) ** 2)))\n        max_speed = float(np.max(np.sqrt(u * u + v *
v)))\n        history.append([step, step * dt, change, poisson_residual, max_speed])\n        completed_steps = step\n
require_finite(psi, omega_field, u, v)\n        if step == 1 or step == max_steps or step % max(1, output_every) == 0:\n
print(f"step={step} time={step * dt:.6e} vorticity_change_l2={change:.6e} poisson_residual={poisson_residual:.6e}")\n        if step %
output_every == 0:\n            write_structured_grid_vtk(f"cavity_{step:06d}.vtk", x, y, {"streamfunction": psi, "vorticity": omega_field,
"speed": np.sqrt(u * u + v * v)}, {"velocity": (u, v)}, "Lid-driven cavity")\n        if change < tolerance and step > 100:\n
break\n    solve_streamfunction(psi, omega_field, dx, dy, poisson_iterations, sor)\n    u, v = velocity_from_streamfunction(psi, dx, dy,
lid_speed)\n    write_structured_grid_vtk("cavity_final.vtk", x, y, {"streamfunction": psi, "vorticity": omega_field, "speed": np.sqrt(u *
u + v * v)}, {"velocity": (u, v)}, "Lid-driven cavity final")\n    write_multiline_vtk("cavity_boundaries.vtk", [(np.array([0.0, 1.0]),
np.array([0.0, 0.0])), (np.array([1.0, 1.0]), np.array([0.0, 1.0])), (np.array([1.0, 0.0]), np.array([1.0, 1.0])), (np.array([0.0, 0.0]),
np.array([1.0, 0.0]))], "Cavity boundaries")\n    seed = np.linspace(0.05, 0.95, 15)\n    sx, sy = np.meshgrid(seed, seed)\n
write_points_vtk("cavity_stream_seeds.vtk", sx.ravel(), sy.ravel(), title="Cavity stream seeds")\n    i_mid = int(np.argmin(np.abs(x1 -
0.5)))\n    j_mid = int(np.argmin(np.abs(y1 - 0.5)))\n    with open("cavity_centerline_profiles.csv", "w", newline="", encoding="utf-8") as
handle:\n        writer = csv.writer(handle)\n        writer.writerow(["vertical_y", "u_at_x_0p5", "horizontal_x", "v_at_y_0p5"])\n
nrows = max(nx, ny)\n        for k in range(nrows):\n            j = min(int(round(k * (ny - 1) / max(nrows - 1, 1))), ny - 1)\n
i = min(int(round(k * (nx - 1) / max(nrows - 1, 1))), nx - 1)\n            writer.writerow([y1[j], u[j, i_mid], x1[i], v[j_mid, i]])\n
with open("cavity_history.csv", "w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n
writer.writerow(["step", "time", "vorticity_change_l2", "poisson_residual_linf", "maximum_speed"])\n        writer.writerows(history)\n
minimum_index = np.unravel_index(np.argmin(psi), psi.shape)\n    with open("cavity_summary.txt", "w", encoding="utf-8") as handle:\n
handle.write("Lid-driven cavity vorticity-streamfunction solution\\n")\n        handle.write(f"nx {nx}\\nny {ny}\\nRe
{reynolds:.12e}\\nlid_speed {lid_speed:.12e}\\nnu {nu:.12e}\\ndt {dt:.12e}\\ncompleted_steps {completed_steps}\\n")\n
handle.write(f"final_vorticity_change_l2 {history[-1][2]:.12e}\\nminimum_streamfunction {float(np.min(psi)):.12e}\\n")\n
handle.write(f"primary_vortex_x {x1[minimum_index[1]]:.12e}\\nprimary_vortex_y {y1[minimum_index[0]]:.12e}\\n")\n
handle.write("benchmark_requirement compare_centerlines_with_Ghia_Ghia_Shin\\n")\n    print("Wrote cavity_final.vtk, cavity_boundaries.vtk,
cavity_stream_seeds.vtk, cavity_centerline_profiles.csv, cavity_history.csv, cavity_summary.txt")\n    return 0\n\n\nif __name__ ==
"__main__":\n    raise SystemExit(main())\n', encoding="utf-8")
SCRIPT = CODE_DIR / 'exp09_lid_cavity_psi_omega.py'
print("Runtime:", "Google Colab" if IN_COLAB else "local notebook test")
print("Working folder:", ROOT)
print("No package installation or external file upload is required.")


# ===== NOTEBOOK CODE CELL 2 =====
#@title 2. Select run mode and parameters
RUN_MODE = "custom" #@param ["quick", "lab", "custom"]

nx = 65 #@param {type:"integer"}
ny = 65 #@param {type:"integer"}
Re = 400.0 #@param {type:"number"}
U_lid = 1.0 #@param {type:"number"}
max_steps = 2000 #@param {type:"integer"}
output_every = 500 #@param {type:"integer"}
poisson_iterations = 60 #@param {type:"integer"}
tolerance = 1e-08 #@param {type:"number"}
CFL = 0.8 #@param {type:"number"}
SOR = 1.7 #@param {type:"number"}

QUICK_ARGS = [21, 21, 100, 1.0, 10, 10, 10, 1e-05, 0.8, 1.5]
LAB_ARGS = [65, 65, 100, 1.0, 2000, 500, 60, 1e-08, 0.8, 1.7]
CUSTOM_ARGS = [nx, ny, Re, U_lid, max_steps, output_every, poisson_iterations, tolerance, CFL, SOR]
ARGS = QUICK_ARGS if RUN_MODE == "quick" else LAB_ARGS if RUN_MODE == "lab" else CUSTOM_ARGS
print("Run mode:", RUN_MODE)
print("Arguments:", ARGS)


# ===== NOTEBOOK CODE CELL 4 =====
#@title 3. Run the numerical solver
OUT = run_solver(SCRIPT, ARGS, OUTPUT_DIR)


# ===== NOTEBOOK CODE CELL 6 =====
#@title 4. Read the solver summary
summary_text = show_summary(OUT / 'cavity_summary.txt')


# ===== NOTEBOOK CODE CELL 8 =====
#@title 5. Plot the primary numerical results
profiles = read_csv(OUT / "cavity_centerline_profiles.csv")
plt.figure(figsize=(7, 4))
plt.plot(profiles["u_at_x_0p5"], profiles["vertical_y"])
plt.xlabel("u at x = 0.5")
plt.ylabel("y")
plt.title("Vertical centreline velocity")
plt.grid(True)
plt.show()

plt.figure(figsize=(7, 4))
plt.plot(profiles["horizontal_x"], profiles["v_at_y_0p5"])
plt.xlabel("x")
```

```
plt.ylabel("v at y = 0.5")
plt.title("Horizontal centreline velocity")
plt.grid(True)
plt.show()

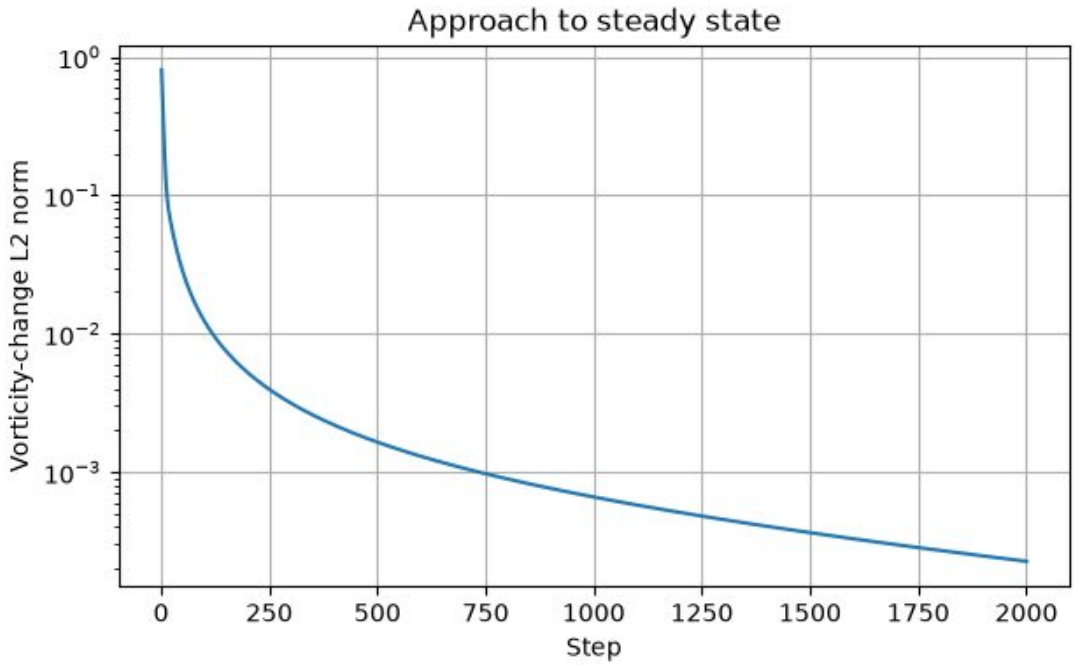
history = read_csv(OUT / "cavity_history.csv")
plt.figure(figsize=(7, 4))
plt.semilogy(history["step"], history["vorticity_change_l2"])
plt.xlabel("Step")
plt.ylabel("Vorticity-change L2 norm")
plt.title("Approach to steady state")
plt.grid(True)
plt.show()

# ===== NOTEBOOK CODE CELL 10 =====
#@title 6. Display the final field and generated files
plot_vtk_scalar(OUT / 'cavity_final.vtk', 'streamfunction')
files = list_outputs(OUT)

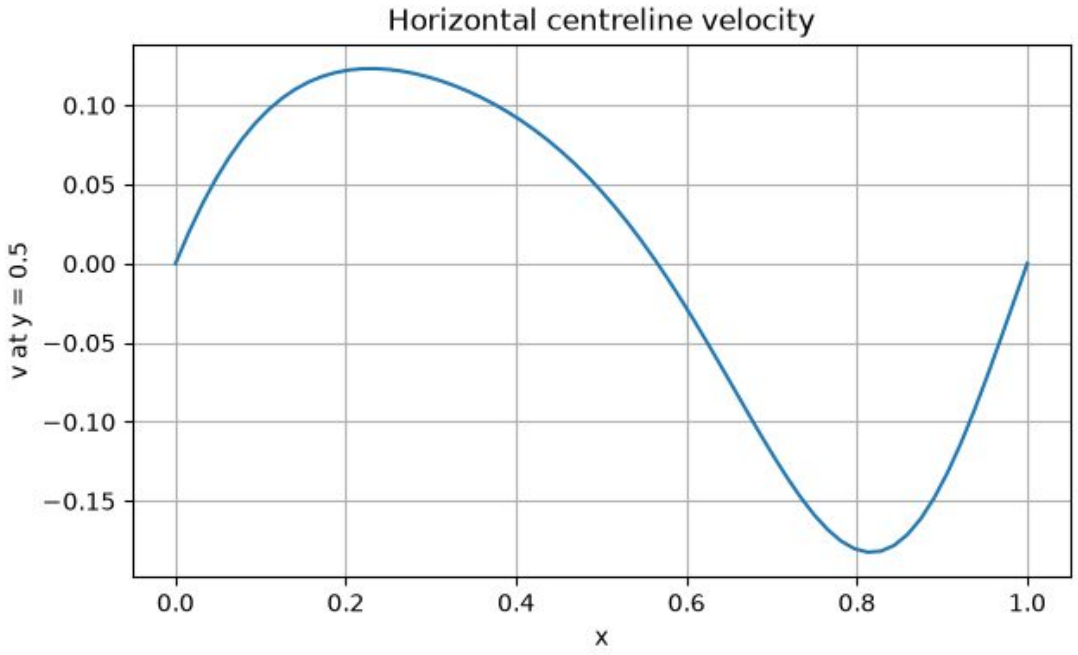

# ===== NOTEBOOK CODE CELL 12 =====
#@title 7. Package and download the results
DOWNLOAD_RESULTS = False #@param {type:"boolean"}
RESULTS_ZIP = make_results_zip(OUT, 9)
if DOWNLOAD_RESULTS:
    if IN_COLAB:
        colab_files.download(str(RESULTS_ZIP))
    else:
        print("Local test mode. Archive retained at:", RESULTS_ZIP)
else:
    print("Set DOWNLOAD_RESULTS to True and run this cell when ready to download.")
```

Output

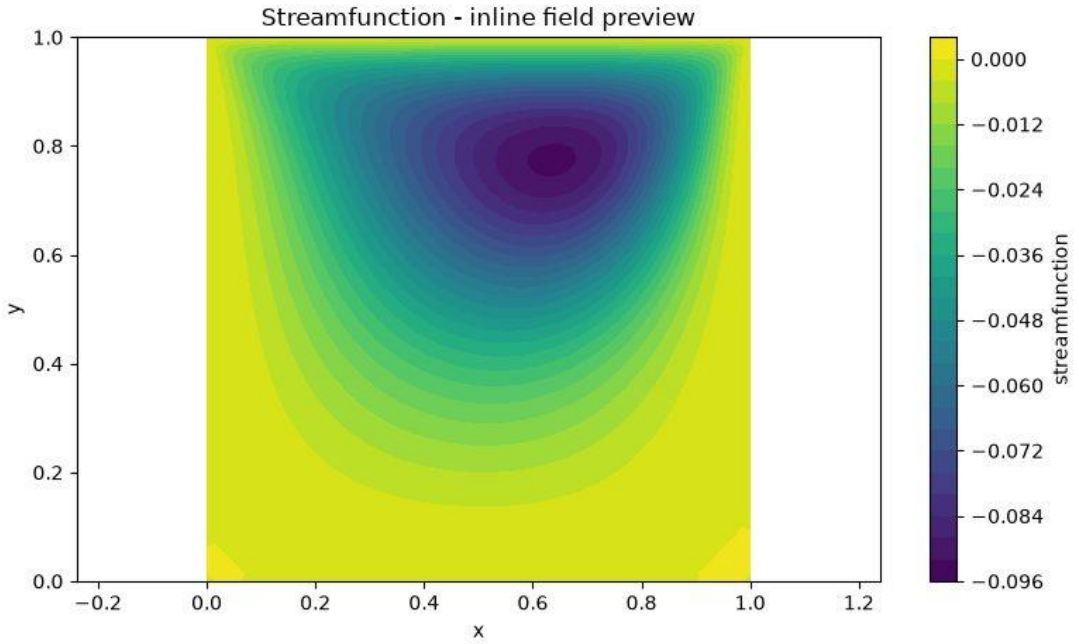
The following figures are reproduced from the output folder supplied for this experiment.



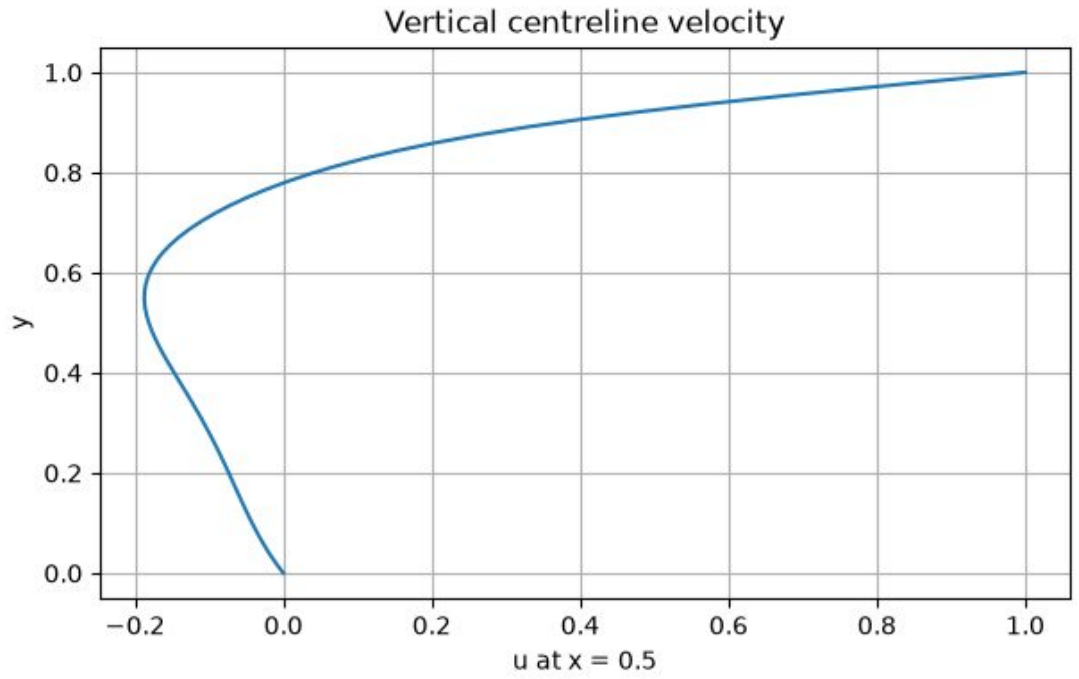
Experiment 9 output: Re 100/Approach to Steady State



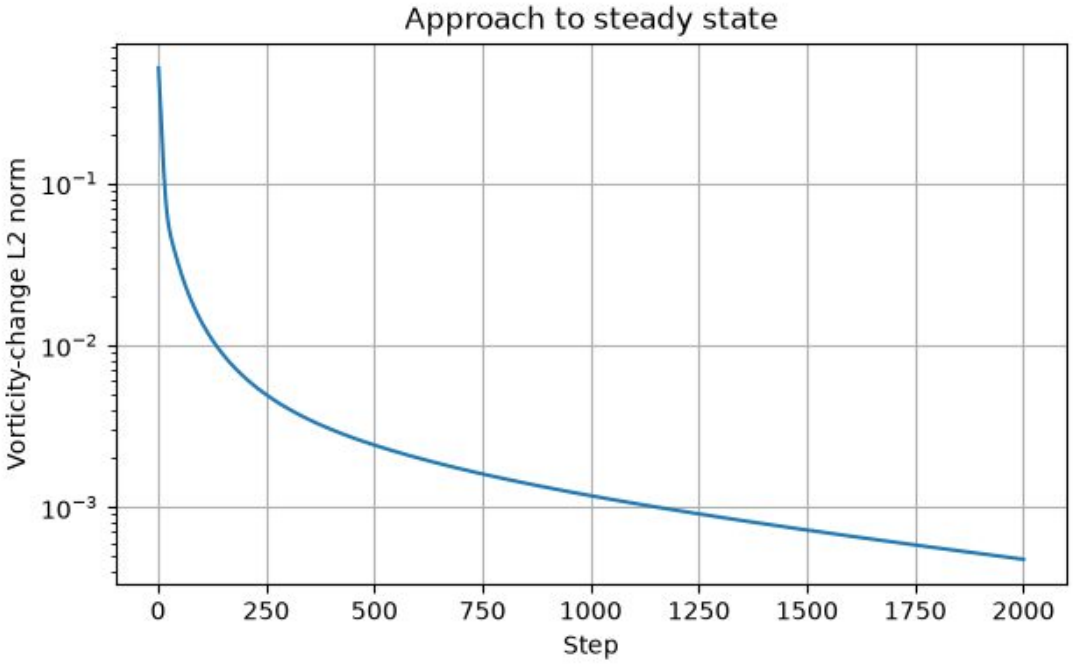
Experiment 9 output: Re 100/Horizontal Centerline Velocity



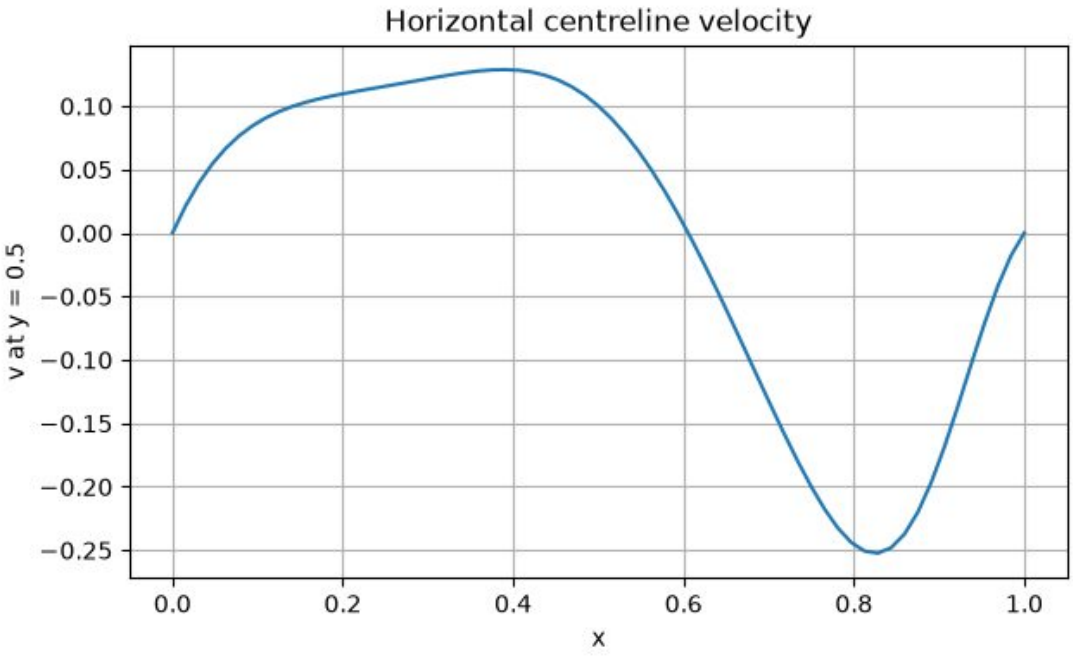
Experiment 9 output: Re 100/Stream Function



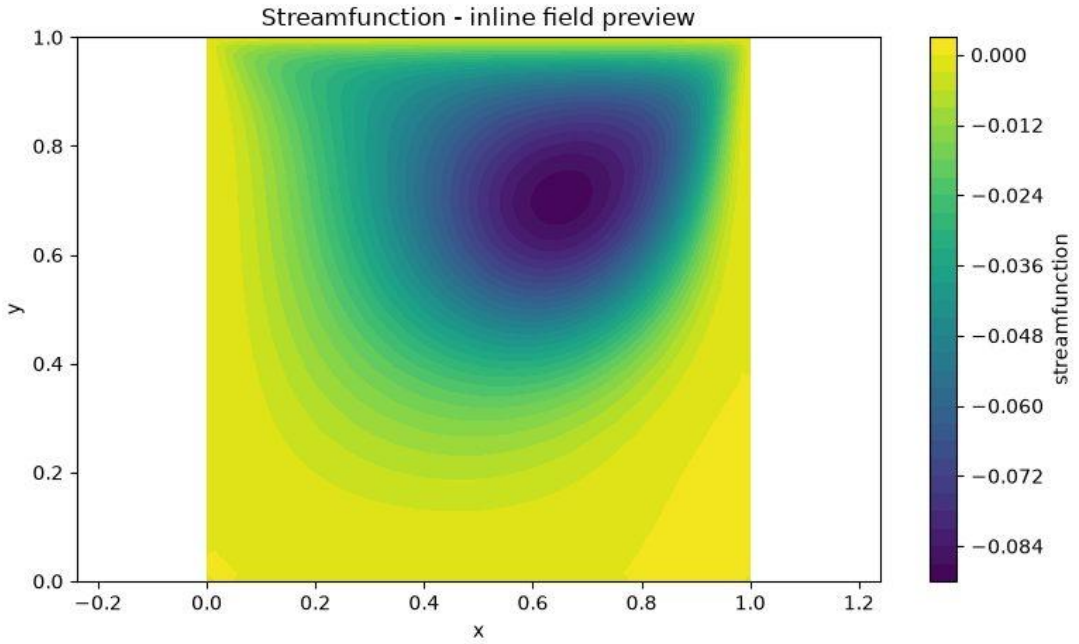
Experiment 9 output: Re 100/Vertical Centerline Velocity



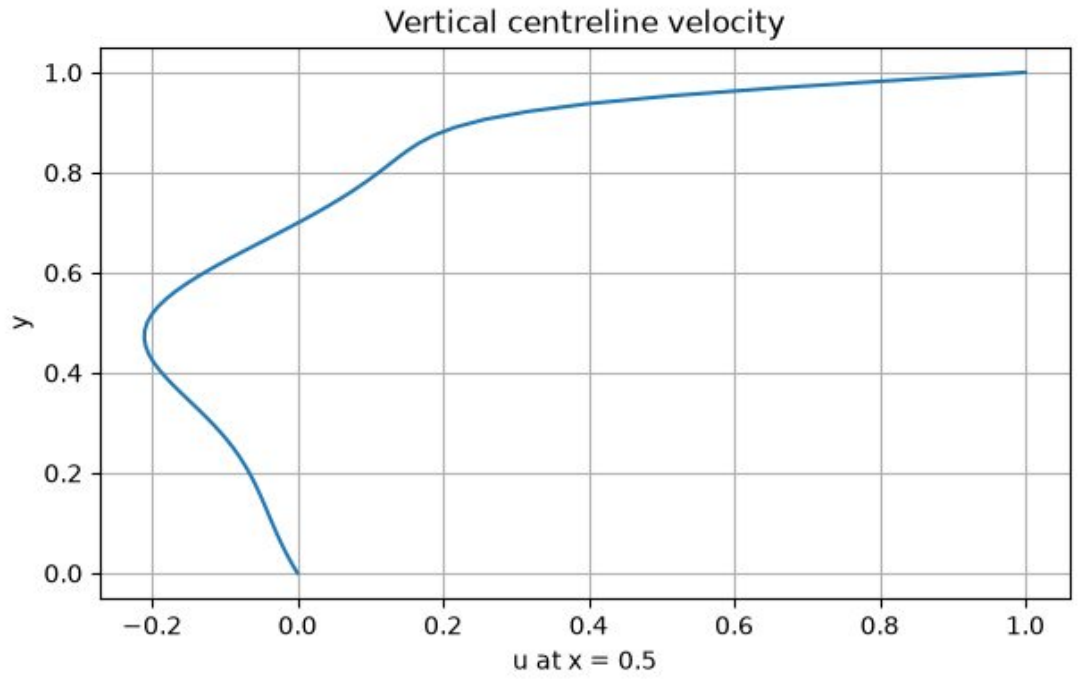
Experiment 9 output: Re 400/Approach to Steady State



Experiment 9 output: Re 400/Horizontal Centreline Velocity



Experiment 9 output: Re 400/Stream Function



Experiment 9 output: Re 400/Vertical Centerline Velocity

Results and Discussion

For each Reynolds number, the archive includes streamfunction contours, horizontal and vertical centreline velocity profiles and approach-to-steady-state history. Increasing Re reduces the relative influence of viscosity and can strengthen the primary recirculation and secondary corner structures. Use the saved centreline and streamfunction figures to report the actual vortex location and any corner vortices; do not infer exact coordinates without extracting the data.

Conclusion

The solver generates the principal cavity-flow diagnostics for Re=100 and 400. The paired centreline, streamfunction and convergence plots allow the effect of Reynolds number to be compared. Grid resolution and convergence tolerance limit the precision of vortex-position predictions.

EXPERIMENT 10 – Turbulent pipe-flow model comparison

Aim

To compare laminar and turbulence-model predictions for fully developed pipe flow at three Reynolds numbers and evaluate velocity profiles, turbulent viscosity ratios and friction factors.

Objective

- Compute radial velocity profiles for laminar flow and four turbulence closures: mixing length, van Driest, k–ε and k–ω SST.
- Evaluate the eddy-viscosity ratio vt/v across the pipe radius.
- Run Re=10^4, approximately 3.16×10^4 and 10^5.
- Compare Darcy friction factors with Blasius and Haaland smooth-pipe correlations.

Governing Equation

Steady fully developed pipe momentum balance in radial form:
0 = −(1/ρ)dp/dx + (1/r) d/dr[r(ν+νt) dU/dr].

The code uses an eddy-viscosity closure vt based on the selected model. The laminar case sets vt=0. Turbulent model equations are simplified algebraic/iterative teaching closures rather than full transport-equation RANS implementations.

Darcy friction references used: Blasius f_D=0.3164 Re^(−0.25) (for turbulent smooth-pipe range) and Haaland smooth-pipe reference as implemented in the notebook.

Boundary Condition

- Pipe centreline: symmetry, zero radial velocity gradient for axial velocity.
- Pipe wall: no-slip, U=0; turbulent viscosity tends to zero at the wall through the wall-damping treatment/model implementation.
- Bulk flow is normalised to the target bulk velocity used by the solver.
- Radial grid is cell-centred and extends from the centre region toward the wall.

Code

Parameters: n_radial=121, VTK grid=160×61, Re_min=10,000, Re_max=100,000, n_Re=3, max_iterations=1500 and tolerance=10^{-7}. Reynolds numbers are logarithmically spaced.

```
# ===== NOTEBOOK CODE CELL 0 =====
from pathlib import Path
import shutil
import subprocess
import sys
import zipfile
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Markdown, display

IN_COLAB = False
try:
    from google.colab import files as colab_files
    IN_COLAB = True
except ImportError:
    colab_files = None


def run_solver(script_path, arguments, output_dir, timeout=1800):
    output_dir = Path(output_dir)
    if output_dir.exists():
        shutil.rmtree(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    command = [sys.executable, str(script_path), *[str(value) for value in arguments]]
    print("Running:", " ".join(command))
    completed = subprocess.run(command, cwd=output_dir, capture_output=True, text=True, timeout=timeout)
    (output_dir / "solver_stdout.log").write_text(completed.stdout, encoding="utf-8")
    (output_dir / "solver_stderr.log").write_text(completed.stderr, encoding="utf-8")
    if completed.stdout.strip():
        print(completed.stdout.strip())
    if completed.returncode != 0:
```

```
        raise RuntimeError(completed.stderr.strip() or f"Solver exited with code {completed.returncode}")
    if completed.stderr.strip():
        print(completed.stderr.strip())
    return output_dir


def read_csv(path):
    data = np.genfromtxt(path, delimiter=",", names=True, dtype=None, encoding="utf-8")
    return np.atleast_1d(data)


def show_summary(path):
    text = Path(path).read_text(encoding="utf-8").strip()
    display(Markdown(f"### Solver summary\n```text\n{text}\n```"))
    return text


def list_outputs(output_dir):
    rows = []
    for path in sorted(Path(output_dir).iterdir()):
        if path.is_file():
            size = path.stat().st_size
            if size < 1024:
                size_text = f"{size} B"
            elif size < 1024**2:
                size_text = f"{size / 1024:.1f} KB"
            else:
                size_text = f"{size / 1024**2:.1f} MB"
            rows.append((path.name, size_text))
    table = "\n".join(f"| `{name}` | {size} |" for name, size in rows)
    display(Markdown(f"### Generated files\n| File | Size |\n|---|---:|\n{table}"))
    return rows


def read_structured_vtk(path):
    lines = Path(path).read_text(encoding="utf-8").splitlines()
    dims_index = next(i for i, line in enumerate(lines) if line.startswith("DIMENSIONS "))
    _, nx, ny, nz = lines[dims_index].split()
    nx, ny, nz = int(nx), int(ny), int(nz)
    if nz != 1:
        raise ValueError("Only 2D structured VTK files are supported by this notebook preview")
    points_index = next(i for i, line in enumerate(lines) if line.startswith("POINTS "))
    npoints = int(lines[points_index].split()[1])
    points = np.array([[float(v) for v in lines[points_index + 1 + k].split()[:3]] for k in range(npoints)])
    x = points[:, 0].reshape(ny, nx)
    y = points[:, 1].reshape(ny, nx)
    data_index = next(i for i, line in enumerate(lines) if line.startswith("POINT_DATA ")) + 1
    scalars = {}
    vectors = {}
    i = data_index
    while i < len(lines):
        if lines[i].startswith("SCALARS "):
            name = lines[i].split()[1]
            i += 2
            values = np.array([float(lines[i + k].split()[0]) for k in range(npoints)])
            scalars[name] = values.reshape(ny, nx)
            i += npoints
        elif lines[i].startswith("VECTORS "):
            name = lines[i].split()[1]
            i += 1
            values = np.array([[float(v) for v in lines[i + k].split()[:3]] for k in range(npoints)])
            vectors[name] = (values[:, 0].reshape(ny, nx), values[:, 1].reshape(ny, nx))
            i += npoints
        else:
            i += 1
    return x, y, scalars, vectors


def plot_vtk_scalar(path, preferred_field=None):
    path = Path(path)
    if not path.is_file():
        print(f"VTK preview skipped: {path.name} was not generated")
        return
    x, y, scalars, _ = read_structured_vtk(path)
    if not scalars:
        print("VTK preview skipped: no scalar field found")
        return
    field = preferred_field if preferred_field in scalars else next(iter(scalars))
    plt.figure(figsize=(8, 4.8))
    image = plt.contourf(x, y, scalars[field], levels=30)
    plt.colorbar(image, label=field.replace("_", " "))
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title(f"{field.replace('_', ' ').title()} - inline field preview")
    plt.axis("equal")
    plt.tight_layout()
    plt.show()


def make_results_zip(output_dir, experiment_number):
    output_dir = Path(output_dir)
    archive = output_dir.parent / f"experiment_{experiment_number:02d}_results.zip"
    if archive.exists():
        archive.unlink()
    with zipfile.ZipFile(archive, "w", compression=zipfile.ZIP_DEFLATED) as handle:
        for path in sorted(output_dir.rglob("*")):
            if path.is_file():
                handle.write(path, path.relative_to(output_dir))
    print(f"Results archive: {archive} ({archive.stat().st_size / 1024:.1f} KB)")
```

```
    return archive


EXPERIMENT = 10
ROOT = Path("/content/cfd_lab_exp10") if IN_COLAB else Path.home() / "CFD_Runs" / "exp10"
CODE_DIR = ROOT / "code"
OUTPUT_DIR = ROOT / "outputs"
CODE_DIR.mkdir(parents=True, exist_ok=True)
(CODE_DIR / "common.py").write_text('from __future__ import annotations\n\nimport csv\nimport math\nfrom pathlib import Path\nfrom typing import Iterable, Mapping\n\nimport numpy as np\n\n\ndef trapezoidal_integral(values: np.ndarray, coordinates: np.ndarray) -> float:\n    y = np.asarray(values, dtype=float).ravel()\n    x = np.asarray(coordinates, dtype=float).ravel()\n    if y.size != x.size or y.size < 2:\n        raise ValueError("trapezoidal integration requires equally sized arrays with at least two points")\n    return float(np.sum(0.5 * (y[1:] + y[:-1]) * np.diff(x)))\n\n\ndef write_csv(path: str | Path, header: Iterable[str], rows: Iterable[Iterable[object]]) -> None:\n    with Path(path).open("w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(list(header))\n        writer.writerows(rows)\n\n\ndef mesh_xy(nx: int, ny: int, lx: float = 1.0, ly: float = 1.0, x0: float = 0.0, y0: float = 0.0) -> tuple[np.ndarray, np.ndarray]:\n    x = np.linspace(x0, x0 + lx, nx)\n    y = np.linspace(y0, y0 + ly, ny)\n    return np.meshgrid(x, y)\n\n\ndef write_structured_grid_vtk(\n    path: str | Path,\n    x: np.ndarray,\n    y: np.ndarray,\n    scalars: Mapping[str, np.ndarray] | None = None,\n    vectors: Mapping[str, tuple[np.ndarray, np.ndarray]] | None = None,\n    title: str = "CFD lab output",\n) -> None:\n    x = np.asarray(x, dtype=float)\n    y = np.asarray(y, dtype=float)\n    if x.shape != y.shape or x.ndim != 2:\n        raise ValueError("x and y must be equally shaped 2D arrays")\n    ny, nx = x.shape\n    scalar_map = dict(scalars or {})\n    vector_map = dict(vectors or {})\n    for name, values in scalar_map.items():\n        if np.asarray(values).shape != x.shape:\n            raise ValueError(f"scalar {name} has shape {np.asarray(values).shape}, expected {x.shape}")\n    for name, values in vector_map.items():\n        if len(values) != 2 or np.asarray(values[0]).shape != x.shape or np.asarray(values[1]).shape != x.shape:\n            raise ValueError(f"vector {name} has incompatible shape")\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET STRUCTURED_GRID\\n")\n        handle.write(f"DIMENSIONS {nx} {ny} 1\\n")\n        handle.write(f"POINTS {nx * ny} float\\n")\n        for j in range(ny):\n            for i in range(nx):\n                handle.write(f"{x[j, i]:.12e} {y[j, i]:.12e} 0\\n")\n        handle.write(f"POINT_DATA {nx * ny}\\n")\n        for name, values in scalar_map.items():\n            data = np.asarray(values, dtype=float)\n            handle.write(f"SCALARS {name} float 1\\n")\n            handle.write("LOOKUP_TABLE default\\n")\n            for value in data.ravel(order="C"):\n                handle.write(f"{value:.12e}\\n")\n        for name, values in vector_map.items():\n            vx = np.asarray(values[0], dtype=float)\n            vy = np.asarray(values[1], dtype=float)\n            handle.write(f"VECTORS {name} float\\n")\n            for ax, ay in zip(vx.ravel(order="C"), vy.ravel(order="C")):\n                handle.write(f"{ax:.12e} {ay:.12e} 0\\n")\n\n\ndef write_polyline_vtk(path: str | Path, x: np.ndarray, y: np.ndarray, title: str = "polyline", close: bool = False) -> None:\n    x = np.asarray(x, dtype=float).ravel()\n    y = np.asarray(y, dtype=float).ravel()\n    if x.size != y.size or x.size < 2:\n        raise ValueError("polyline requires equally sized x and y arrays")\n    ids = list(range(x.size))\n    if close:\n        ids.append(0)\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET POLYDATA\\n")\n        handle.write(f"POINTS {x.size} float\\n")\n        for xi, yi in zip(x, y):\n            handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n        handle.write(f"LINES 1 {len(ids) + 1}\\n")\n        handle.write(f"{len(ids)} {\' \'.join(str(i) for i in ids)}\\n")\n\n\ndef write_multiline_vtk(path: str | Path, lines: list[tuple[np.ndarray, np.ndarray]], title: str = "lines") -> None:\n    points: list[tuple[float, float]] = []\n    connectivity: list[list[int]] = []\n    for x, y in lines:\n        xa = np.asarray(x, dtype=float).ravel()\n        ya = np.asarray(y, dtype=float).ravel()\n        if xa.size != ya.size or xa.size < 2:\n            raise ValueError("each line needs equally sized x and y arrays")\n        ids = list(range(len(points), len(points) + xa.size))\n        points.extend(zip(xa.tolist(), ya.tolist()))\n        connectivity.append(ids)\n    size = sum(len(ids) + 1 for ids in connectivity)\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET POLYDATA\\n")\n        handle.write(f"POINTS {len(points)} float\\n")\n        for xi, yi in points:\n            handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n        handle.write(f"LINES {len(connectivity)} {size}\\n")\n        for ids in connectivity:\n            handle.write(f"{len(ids)} {\' \'.join(str(i) for i in ids)}\\n")\n\n\ndef write_points_vtk(path: str | Path, x: np.ndarray, y: np.ndarray, scalars: Mapping[str, np.ndarray] | None = None, title: str = "points") -> None:\n    xa = np.asarray(x, dtype=float).ravel()\n    ya = np.asarray(y, dtype=float).ravel()\n    if xa.size != ya.size:\n        raise ValueError("point arrays must have equal size")\n    scalar_map = dict(scalars or {})\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET POLYDATA\\n")\n        handle.write(f"POINTS {xa.size} float\\n")\n        for xi, yi in zip(xa, ya):\n            handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n        handle.write(f"VERTICES {xa.size} {2 * xa.size}\\n")\n        for i in range(xa.size):\n            handle.write(f"1 {i}\\n")\n        if scalar_map:\n            handle.write(f"POINT_DATA {xa.size}\\n")\n            for name, values in scalar_map.items():\n                data = np.asarray(values, dtype=float).ravel()\n                if data.size != xa.size:\n                    raise ValueError(f"scalar {name} has wrong size")\n                handle.write(f"SCALARS {name} float 1\\n")\n                handle.write("LOOKUP_TABLE default\\n")\n                for value in data:\n                    handle.write(f"{value:.12e}\\n")\n\n\ndef gradient(field: np.ndarray, dx: float, dy: float) -> tuple[np.ndarray, np.ndarray]:\n    gy, gx = np.gradient(field, dy, dx, edge_order=2)\n    return gx, gy\n\n\ndef laplacian(field: np.ndarray, dx: float, dy: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    out[1:-1, 1:-1] = (\n        (field[1:-1, 2:] - 2.0 * field[1:-1, 1:-1] + field[1:-1, :-2]) / (dx * dx)\n        + (field[2:, 1:-1] - 2.0 * field[1:-1, 1:-1] + field[:-2, 1:-1]) / (dy * dy)\n    )\n    return out\n\n\ndef upwind_derivative_x(field: np.ndarray, velocity: np.ndarray, dx: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    backward = (field[1:-1, 1:-1] - field[1:-1, :-2]) / dx\n    forward = (field[1:-1, 2:] - field[1:-1, 1:-1]) / dx\n    out[1:-1, 1:-1] = np.where(velocity[1:-1, 1:-1] >= 0.0, backward, forward)\n    return out\n\n\ndef upwind_derivative_y(field: np.ndarray, velocity: np.ndarray, dy: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    backward = (field[1:-1, 1:-1] - field[:-2, 1:-1]) / dy\n    forward = (field[2:, 1:-1] - field[1:-1, 1:-1]) / dy\n    out[1:-1, 1:-1] = np.where(velocity[1:-1, 1:-1] >= 0.0, backward, forward)\n    return out\n\n\ndef poisson_jacobi(phi: np.ndarray, rhs: np.ndarray, dx: float, dy: float, iterations: int, boundary: str = "neumann", mask: np.ndarray | None = None) -> np.ndarray:\n    dx2 = dx * dx\n    dy2 = dy * dy\n    denom = 2.0 * (dx2 + dy2)\n    for _ in range(iterations):\n        old = phi.copy()\n        phi[1:-1, 1:-1] = (\n            (old[1:-1, 2:] + old[1:-1, :-2]) * dy2\n            + (old[2:, 1:-1] + old[:-2, 1:-1]) * dx2\n            - rhs[1:-1, 1:-1] * dx2 * dy2\n        ) / denom\n        if mask is not None:\n            phi[mask > 0.5] = 0.0\n        if boundary == "neumann":\n            phi[:, 0] = phi[:, 1]\n            phi[:, -1] = 0.0\n            phi[0, :] = phi[1, :]\n            phi[-1, :] = phi[-2, :]\n        elif boundary == "closed":\n            phi[:, 0] = phi[:, 1]\n            phi[:, -1] = phi[:, -2]\n            phi[0, :] = phi[1, :]\n            phi[-1, :] = phi[-2, :]\n        phi -= np.mean(phi)\n    else:\n        raise ValueError("unsupported pressure boundary")\n    return phi\n\n\ndef tdma(a: np.ndarray, b: np.ndarray, c: np.ndarray, d: np.ndarray) -> np.ndarray:\n    a = np.asarray(a, dtype=float).copy()\n    b = np.asarray(b, dtype=float).copy()\n    c = np.asarray(c, dtype=float).copy()\n    d = np.asarray(d, dtype=float).copy()\n    n = d.size\n    if n == 0 or a.size != n or b.size != n or c.size != n:\n        raise ValueError("TDMA arrays must have equal nonzero size")\n    for i in range(1, n):\n        if abs(b[i - 1]) < 1e-30:\n            raise FloatingPointError("zero TDMA pivot")\n        factor = a[i] / b[i - 1]\n        b[i] -= factor * c[i - 1]\n        d[i] -= factor * d[i - 1]\n    x = np.empty(n, dtype=float)\n    if abs(b[-1]) < 1e-30:\n        raise FloatingPointError("zero TDMA pivot")\n    x[-1] = d[-1] / b[-1]\n    for i in range(n - 2, -1, -1):\n        if abs(b[i]) < 1e-30:\n            raise FloatingPointError("zero TDMA pivot")\n        x[i] = (d[i] - c[i] * x[i + 1]) / b[i]\n    return x\n\n\ndef relative_l2(numerical: np.ndarray, reference: np.ndarray) -> float:\n    a = np.asarray(numerical, dtype=float)\n    b = np.asarray(reference, dtype=float)\n    denominator = float(np.linalg.norm(b.ravel()))\n    numerator = float(np.linalg.norm((a - b).ravel()))\n    return numerator / max(denominator, 1e-30)\n\n\ndef require_finite(*arrays: np.ndarray) -> None:\n    for array in arrays:\n        if not np.all(np.isfinite(array)):\n            raise FloatingPointError("non-finite numerical field detected")\n\n\ndef positive_float(value: str, name: str) -> float:\n    result = float(value)\n    if not math.isfinite(result) or result <= 0.0:\n        raise ValueError(f"{name} must be positive and finite")\n    return result\n\n\ndef positive_int(value: str, name: str, minimum: int = 1) -> int:\n    result = int(value)\n    if result < minimum:\n        raise ValueError(f"{name} must be at least {minimum}")\n    return result\n', encoding="utf-8")
(CODE_DIR / 'exp10_pipe_turbulence_models.py').write_text('from __future__ import annotations\n\nimport csv\nimport math\nimport sys\nfrom dataclasses import dataclass\n\nimport numpy as np\n\nfrom common import tdma, write_multiline_vtk, write_structured_grid_vtk\n\n\n@dataclass\nclass ModelResult:\n    name: str\n    reynolds: float\n    nu: float\n    friction: float\n    u_tau: float\n    pressure_drop: float\n    residual: float\n    iterations: int\n    radius: np.ndarray\n    velocity: np.ndarray\n    nut: np.ndarray\n    k: np.ndarray\n    epsilon: np.ndarray\n    omega: np.ndarray\n\n\ndef bulk_velocity(radius: np.ndarray, velocity: np.ndarray, pipe_radius: float, dr: float) -> float:\n    return 2.0 * float(np.sum(velocity * radius) * dr) / (pipe_radius * pipe_radius)\n\n\ndef solve_model(model: int, name: str, n: int, reynolds: float, max_iterations: int, tolerance: float) -> ModelResult:\n    pipe_radius = 0.5\n    diameter = 1.0\n    target_bulk = 1.0\n    nu = target_bulk * diameter / reynolds\n    dr = pipe_radius / n\n    radius = (np.arange(n) + 0.5) * dr\n    velocity = 2.0 * (1.0 - (radius / pipe_radius) ** 2)\n    nut = np.zeros(n)\n    k = np.zeros(n)\n    epsilon = np.zeros(n)\n    omega_field = np.zeros(n)\n    kappa = 0.41\n    a_plus = 26.0\n    c_mu = 0.09\n    beta_star = 0.09\n    a1 =
```

```
0.31\n    friction_initial = 0.3164 * max(reynolds, 4000.0) ** (-0.25)\n    u_tau = math.sqrt(max(friction_initial / 8.0, 1e-12))\n
residual = 1.0\n    completed = 0\n    for iteration in range(1, max_iterations + 1):\n        old = velocity.copy()\n        derivative =
np.gradient(velocity, dr, edge_order=2)\n        wall_distance = np.maximum(pipe_radius - radius, 0.5 * dr)\n        y_plus = wall_distance
* u_tau / nu\n        damping = 1.0 - np.exp(-y_plus / a_plus)\n        strain = np.abs(derivative)\n        k.fill(0.0)\n
epsilon.fill(0.0)\n        omega_field.fill(0.0)\n        if model == 0:\n            nut.fill(0.0)\n        elif model == 1:\n
mixing_length = kappa * wall_distance\n            nut = mixing_length * mixing_length * strain\n        elif model == 2:\n
mixing_length = kappa * wall_distance * damping\n            nut = mixing_length * mixing_length * strain\n        elif model == 3:\n
kdamp = 1.0 - np.exp(-y_plus / 12.0)\n            k = u_tau * u_tau / math.sqrt(c_mu) * kdamp * kdamp\n            y_use =
np.maximum(wall_distance, 1.5 * nu / max(u_tau, 1e-12))\n            epsilon = c_mu ** 0.75 * np.maximum(k, 1e-16) ** 1.5 / (kappa *
y_use)\n            nut = c_mu * k * k / np.maximum(epsilon, 1e-18) * damping * damping\n        elif model == 4:\n            kdamp = 1.0
- np.exp(-y_plus / 10.0)\n            k = u_tau * u_tau / math.sqrt(beta_star) * kdamp * kdamp\n            y_use =
np.maximum(wall_distance, 1.5 * nu / max(u_tau, 1e-12))\n            omega_field = u_tau / (math.sqrt(beta_star) * kappa * y_use)\n
f2_arg = 2.0 * np.sqrt(np.maximum(k, 0.0)) / np.maximum(beta_star * omega_field * y_use, 1e-18)\n            f2 =
np.tanh(np.maximum(f2_arg, 0.0) ** 2)\n            denominator = np.maximum(a1 * omega_field, strain * f2)\n            nut = a1 * k /
np.maximum(denominator, 1e-18) * damping * damping\n            epsilon = beta_star * k * omega_field\n        else:\n            raise
ValueError("unknown turbulence model")\n        nut = np.clip(nut, 0.0, 300.0 * nu)\n        effective = nu + nut\n        a =
np.zeros(n)\n        b = np.zeros(n)\n        c = np.zeros(n)\n        rhs = radius * dr\n        for i in range(n):\n            diagonal
= 0.0\n            if i > 0:\n                rw = i * dr\n                nuw = 0.5 * (effective[i] + effective[i - 1])\n
aw = rw * nuw / dr\n                a[i] = -aw\n                diagonal += aw\n            re = (i + 1) * dr\n            if i < n - 1:\n
nue = 0.5 * (effective[i] + effective[i + 1])\n                ae = re * nue / dr\n                c[i] = -ae\n                diagonal +=
ae\n            else:\n                ae = re * nu / (0.5 * dr)\n            diagonal += ae\n            b[i] = diagonal\n
candidate = tdma(a, b, c, rhs)\n        candidate *= target_bulk / max(bulk_velocity(radius, candidate, pipe_radius, dr), 1e-30)\n
wall_gradient = (0.0 - candidate[-1]) / (0.5 * dr)\n        new_u_tau = math.sqrt(max(nu * abs(wall_gradient), 1e-18))\n        relax = 1.0
if model == 0 else 0.45\n        velocity = relax * candidate + (1.0 - relax) * velocity\n        residual = float(np.linalg.norm(velocity
- old) / max(np.linalg.norm(old), 1e-30))\n        u_tau = 0.35 * new_u_tau + 0.65 * u_tau\n        completed = iteration\n        if
residual < tolerance and iteration > 20:\n            break\n    friction = 8.0 * u_tau * u_tau\n    pressure_drop = friction * 10.0 *
0.5\n    return ModelResult(name, reynolds, nu, friction, u_tau, pressure_drop, residual, completed, radius, velocity, nut, k, epsilon,
omega_field)\n\n\ndef radial_interpolate(result: ModelResult, radial_position: np.ndarray) -> np.ndarray:\n    pipe_radius = 0.5\n    xp =
np.concatenate(([0.0], result.radius, [pipe_radius]))\n    fp = np.concatenate(([result.velocity[0]], result.velocity, [0.0]))\n    return
np.interp(radial_position, xp, fp)\n\n\ndef interpolate_field(radius: np.ndarray, values: np.ndarray, radial_position: np.ndarray,
wall_value: float = 0.0) -> np.ndarray:\n    pipe_radius = 0.5\n    xp = np.concatenate(([0.0], radius, [pipe_radius]))\n    fp =
np.concatenate(([values[0]], values, [wall_value]))\n    return np.interp(radial_position, xp, fp)\n\n\ndef main() -> int:\n    n_radial =
int(sys.argv[1]) if len(sys.argv) > 1 else 121\n    vtk_nx = int(sys.argv[2]) if len(sys.argv) > 2 else 160\n    vtk_ny = int(sys.argv[3])
if len(sys.argv) > 3 else 61\n    re_min = float(sys.argv[4]) if len(sys.argv) > 4 else 10000.0\n    re_max = float(sys.argv[5]) if
len(sys.argv) > 5 else 100000.0\n    n_re = int(sys.argv[6]) if len(sys.argv) > 6 else 3\n    max_iterations = int(sys.argv[7]) if
len(sys.argv) > 7 else 1500\n    tolerance = float(sys.argv[8]) if len(sys.argv) > 8 else 1e-7\n    if n_radial < 21 or vtk_nx < 10 or
vtk_ny < 11 or re_min <= 0.0 or re_max < re_min or n_re < 1 or max_iterations < 1 or tolerance <= 0.0:\n        raise ValueError("usage:
python exp10_pipe_turbulence_models.py nRadial vtkNx vtkNy ReMin ReMax nRe maxIterations tolerance")\n    models = [(0, "laminar"), (1,
"mixing_length"), (2, "van_driest"), (3, "kepsilon"), (4, "komega_sst")]\n    reynolds_values = np.array([re_min]) if n_re == 1 else
np.geomspace(re_min, re_max, n_re)\n    summary_rows: list[list[object]] = []\n    for reynolds in reynolds_values:\n        results:
list[ModelResult] = []\n        for model, name in models:\n            result = solve_model(model, name, n_radial, float(reynolds),
max_iterations, tolerance)\n            results.append(result)\n            blasius = 0.3164 * reynolds ** (-0.25)\n            haaland =
1.0 / (-1.8 * math.log10(6.9 / reynolds)) ** 2\n            summary_rows.append([reynolds, name, result.friction, blasius, haaland,
result.u_tau, result.pressure_drop, result.velocity[0], float(np.max(result.nut / result.nu)), result.iterations, result.residual])\n
print(f"Re={reynolds:.0f} model={name} friction={result.friction:.6e} U_center={result.velocity[0]:.6e} residual={result.residual:.6e}")\n
x1 = np.linspace(0.0, 10.0, vtk_nx)\n        y1 = np.linspace(-0.5, 0.5, vtk_ny)\n        x, y = np.meshgrid(x1, y1)\n
radial_position = np.abs(y)\n        scalars: dict[str, np.ndarray] = {}\n        vectors: dict[str, tuple[np.ndarray, np.ndarray]] = {}\n
for result in results:\n            velocity_profile = radial_interpolate(result, radial_position)\n            nut_ratio =
interpolate_field(result.radius, result.nut / result.nu, radial_position)\n            k_field = interpolate_field(result.radius, result.k,
radial_position)\n            epsilon_field = interpolate_field(result.radius, result.epsilon, radial_position)\n            omega_field =
interpolate_field(result.radius, result.omega, radial_position)\n            pressure = result.pressure_drop * (1.0 - x / 10.0)\n
scalars[f"U_{result.name}"] = velocity_profile\n            scalars[f"nut_over_nu_{result.name}"] = nut_ratio\n
scalars[f"pressure_{result.name}"] = pressure\n            scalars[f"k_{result.name}"] = k_field\n
scalars[f"epsilon_{result.name}"] = epsilon_field\n            scalars[f"omega_{result.name}"] = omega_field\n
vectors[f"velocity_{result.name}"] = (velocity_profile, np.zeros_like(velocity_profile))\n        tag = f"{reynolds:.0f}"\n
write_structured_grid_vtk(f"pipe2d_compare_Re{tag}.vtk", x, y, scalars, vectors, "2D axisymmetric pipe turbulence-model comparison")\n
with open(f"pipe2d_profiles_Re{tag}.csv", "w", newline="", encoding="utf-8") as handle:\n            writer = csv.writer(handle)\n
header = ["r_over_R"]\n            for result in results:\n                header.extend([f"U_{result.name}", f"nut_over_nu_{result.name}",
f"k_{result.name}", f"epsilon_{result.name}", f"omega_{result.name}"])\n            writer.writerow(header)\n            for i in
range(n_radial):\n                row: list[float] = [results[0].radius[i] / 0.5]\n                for result in results:\n
row.extend([result.velocity[i], result.nut[i] / result.nu, result.k[i], result.epsilon[i], result.omega[i]])\n
writer.writerow(row)\n    with open("pipe2d_rans_summary.csv", "w", newline="", encoding="utf-8") as handle:\n        writer =
csv.writer(handle)\n        writer.writerow(["Re", "model", "friction_factor_Darcy", "Blasius_reference", "Haaland_smooth_reference",
"uTau", "pressure_drop_over_10D", "U_center", "maximum_nut_over_nu", "iterations", "residual"])\n        writer.writerows(summary_rows)\n
write_multiline_vtk("pipe2d_walls.vtk", [(np.array([0.0, 10.0]), np.array([-0.5, -0.5])), (np.array([0.0, 10.0]), np.array([0.5, 0.5]))],
"Pipe wall lines")\n    with open("pipe2d_rans_summary.txt", "w", encoding="utf-8") as handle:\n        handle.write("Teaching-level 2D
axisymmetric fully developed pipe-flow turbulence-model comparison\\n")\n        handle.write("Models laminar mixing_length van_driest
equilibrium_kepsilon equilibrium_komega_sst\\n")\n        handle.write("The model solves radial eddy-viscosity closures and extrudes the
profile along the pipe axis.\\n")\n        handle.write("This is not a full industrial two-equation RANS transport solver.\\n")\n
handle.write("Compare friction factors with Blasius and Haaland columns in pipe2d_rans_summary.csv.\\n")\n    print("Wrote
pipe2d_compare_Re*.vtk, pipe2d_profiles_Re*.csv, pipe2d_rans_summary.csv, pipe2d_rans_summary.txt, pipe2d_walls.vtk")\n    return 0\n\n\nif
__name__ == "__main__":\n    raise SystemExit(main())\n', encoding="utf-8")
SCRIPT = CODE_DIR / 'exp10_pipe_turbulence_models.py'
print("Runtime:", "Google Colab" if IN_COLAB else "local notebook test")
print("Working folder:", ROOT)
print("No package installation or external file upload is required.")


# ===== NOTEBOOK CODE CELL 2 =====
#@title 2. Select run mode and parameters
RUN_MODE = "custom" #@param ["quick", "lab", "custom"]

n_radial = 121 #@param {type:"integer"}
vtk_nx = 160 #@param {type:"integer"}
vtk_ny = 61 #@param {type:"integer"}
Re_min = 10000.0 #@param {type:"number"}
Re_max = 100000.0 #@param {type:"number"}
n_Re = 3 #@param {type:"integer"}
max_iterations = 1500 #@param {type:"integer"}
tolerance = 1e-07 #@param {type:"number"}

QUICK_ARGS = [21, 30, 21, 10000, 12000, 1, 8, 0.0001]
LAB_ARGS = [121, 160, 61, 10000, 100000, 3, 1500, 1e-07]
CUSTOM_ARGS = [n_radial, vtk_nx, vtk_ny, Re_min, Re_max, n_Re, max_iterations, tolerance]
ARGS = QUICK_ARGS if RUN_MODE == "quick" else LAB_ARGS if RUN_MODE == "lab" else CUSTOM_ARGS
print("Run mode:", RUN_MODE)
print("Arguments:", ARGS)
```

```
# ===== NOTEBOOK CODE CELL 4 =====
#@title 3. Run the numerical solver
OUT = run_solver(SCRIPT, ARGS, OUTPUT_DIR)


# ===== NOTEBOOK CODE CELL 6 =====
#@title 4. Read the solver summary
summary_text = show_summary(OUT / 'pipe2d_rans_summary.txt')


# ===== NOTEBOOK CODE CELL 8 =====
#@title 5. Plot the primary numerical results

summary = read_csv(OUT / "pipe2d_rans_summary.csv")

plt.figure(figsize=(8, 4))
for model in np.unique(summary["model"]):
    mask = summary["model"] == model
    plt.loglog(summary["Re"][mask],
               summary["friction_factor_Darcy"][mask],
               marker="o", label=str(model))

reference_re = np.unique(summary["Re"])

blasius = np.array([
    summary["Blasius_reference"][summary["Re"] == r][0]
    for r in reference_re
])
haaland = np.array([
    summary["Haaland_smooth_reference"][summary["Re"] == r][0]
    for r in reference_re
])

plt.loglog(reference_re, blasius, "k--", label="Blasius")
plt.loglog(reference_re, haaland, "k-.", label="Haaland")
plt.xlabel("Reynolds number")
plt.ylabel("Darcy friction factor")
plt.title("Friction-factor comparison")
plt.legend(fontsize=8)
plt.grid(True)
plt.show()


# 2. Velocity and turbulent viscosity profiles
profile_files = sorted(OUT.glob("pipe2d_profiles_Re*.csv"))

print("Number of Reynolds number cases:", len(profile_files))

for file in profile_files:
    profiles = read_csv(file)

    print("\nProcessing:", file.stem)

    # Velocity profiles
    plt.figure(figsize=(8, 5))

    for field in profiles.dtype.names:
        if field.startswith("U_"):
            plt.plot(profiles[field],
                     profiles["r_over_R"],
                     label=field[2:])

    plt.xlabel("Mean velocity")
    plt.ylabel("r/R")
    plt.title(f"Velocity profiles: {file.stem}")
    plt.legend()
    plt.grid(True)
    plt.show()

    # Turbulent viscosity ratio profiles
    plt.figure(figsize=(8, 5))

    for field in profiles.dtype.names:
        if field.startswith("nut_over_nu_"):
            plt.plot(profiles[field],
                     profiles["r_over_R"],
                     label=field.replace("nut_over_nu_", ""))

    plt.xlabel("Turbulent viscosity ratio (nu_t/nu)")
    plt.ylabel("r/R")
    plt.title(f"Turbulent viscosity ratio: {file.stem}")
    plt.legend()
    plt.grid(True)
    plt.show()

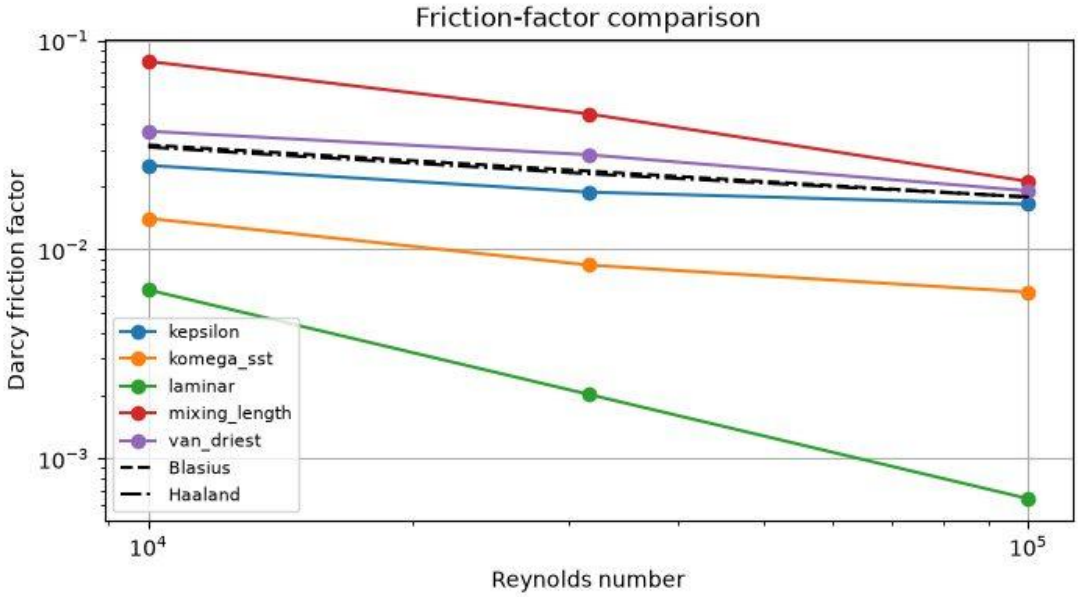

# ===== NOTEBOOK CODE CELL 10 =====
#@title 6. Display the final field and generated files
plot_vtk_scalar(OUT / 'pipe2d_compare_Re10000.vtk', 'U_mixing_length')
files = list_outputs(OUT)


# ===== NOTEBOOK CODE CELL 12 =====
#@title 7. Package and download the results
DOWNLOAD_RESULTS = False #@param {type:"boolean"}
RESULTS_ZIP = make_results_zip(OUT, 10)
if DOWNLOAD_RESULTS:
    if IN_COLAB:
        colab_files.download(str(RESULTS_ZIP))
```

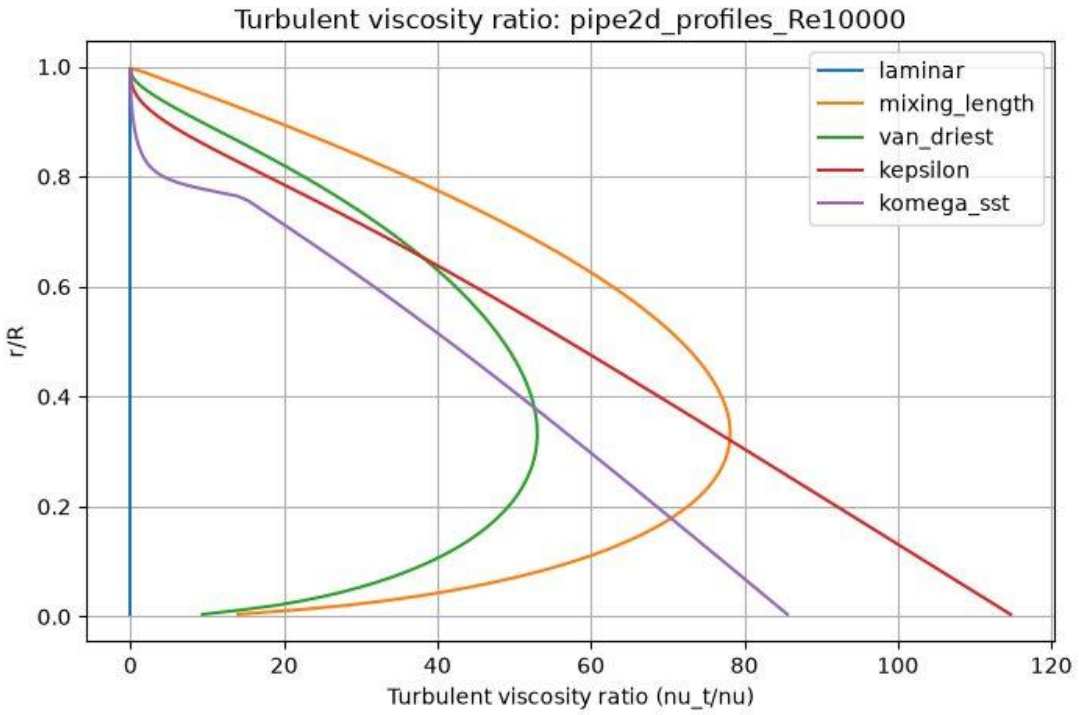
```
    else:
        print("Local test mode. Archive retained at:", RESULTS_ZIP)
else:
    print("Set DOWNLOAD_RESULTS to True and run this cell when ready to download.")
```

Output

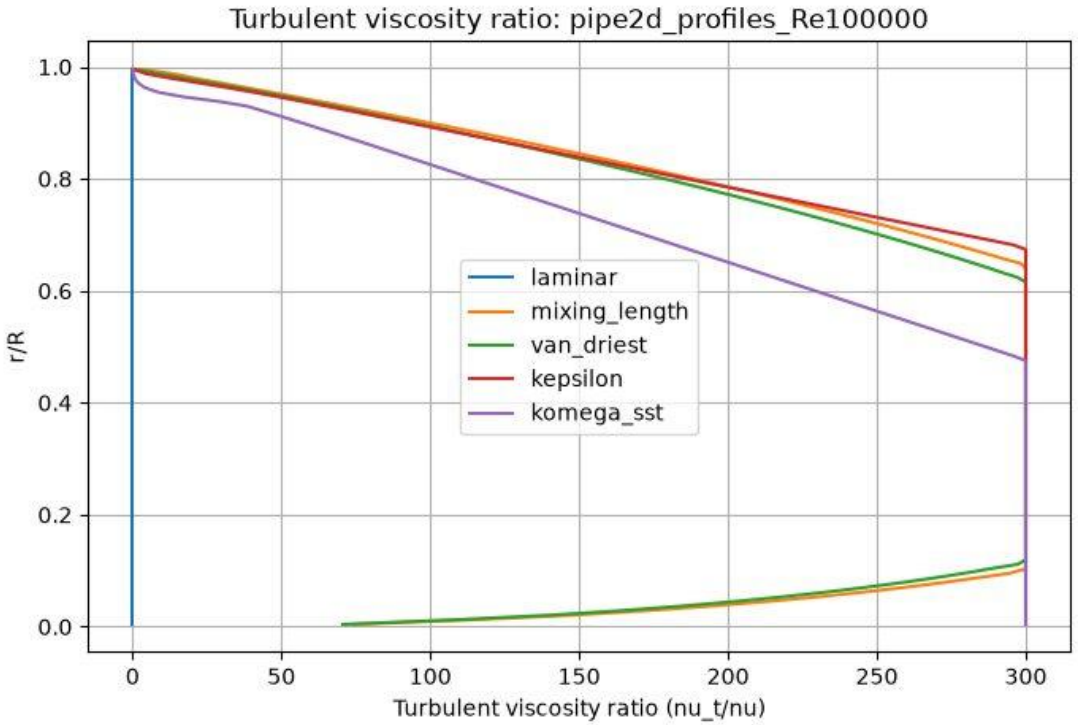
The following figures are reproduced from the output folder supplied for this experiment.



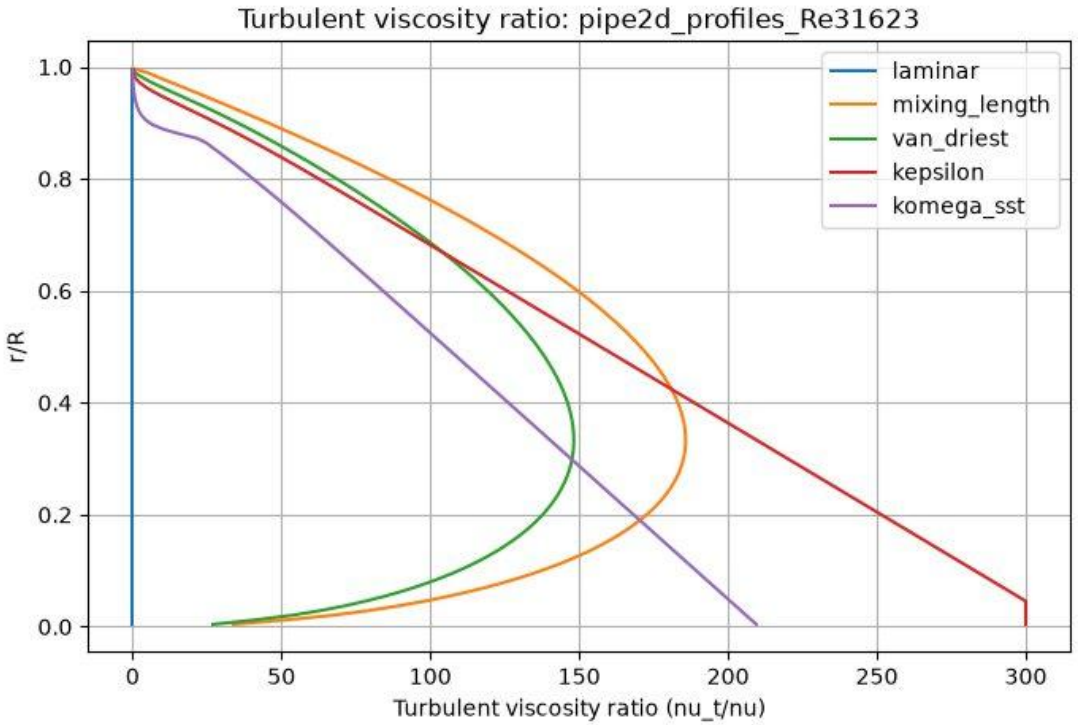
Experiment 10 output: Friction Factor Comparison



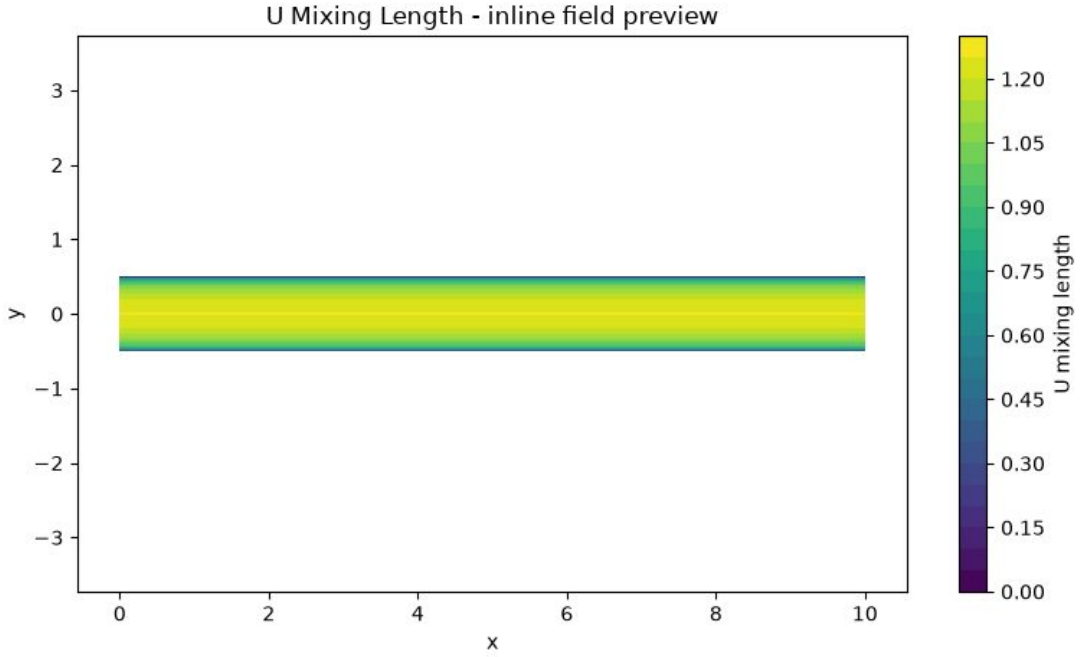
Experiment 10 output: Turbulent Viscosity Ratio Re10^4



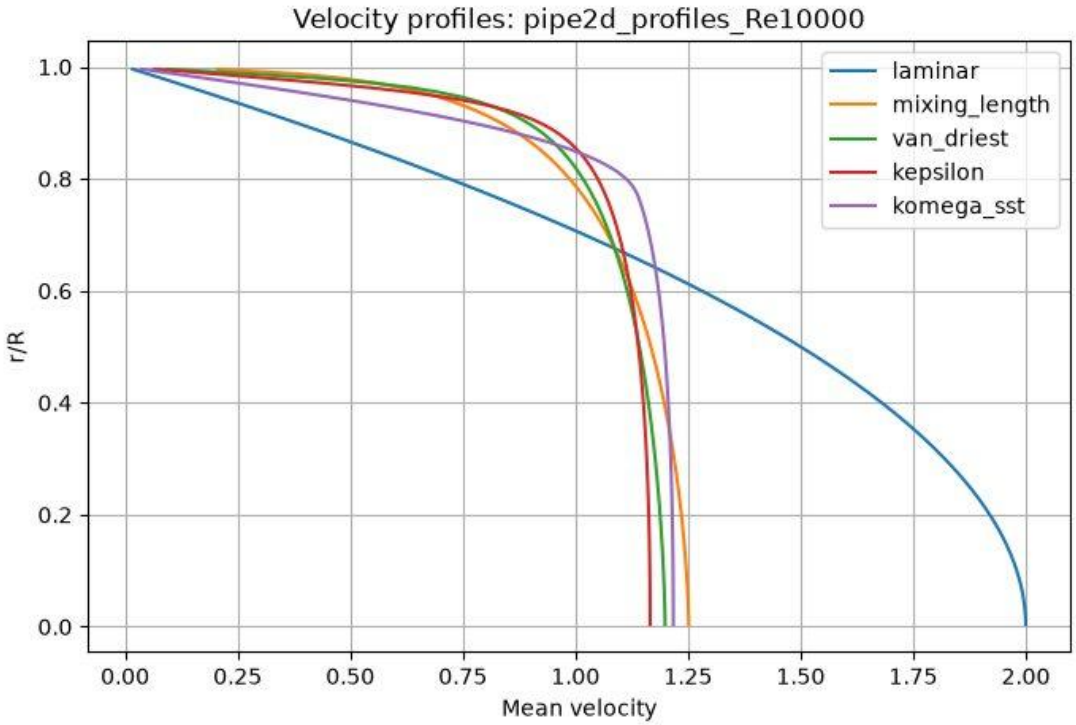
Experiment 10 output: Turbulent Viscosity Ratio Re10^5



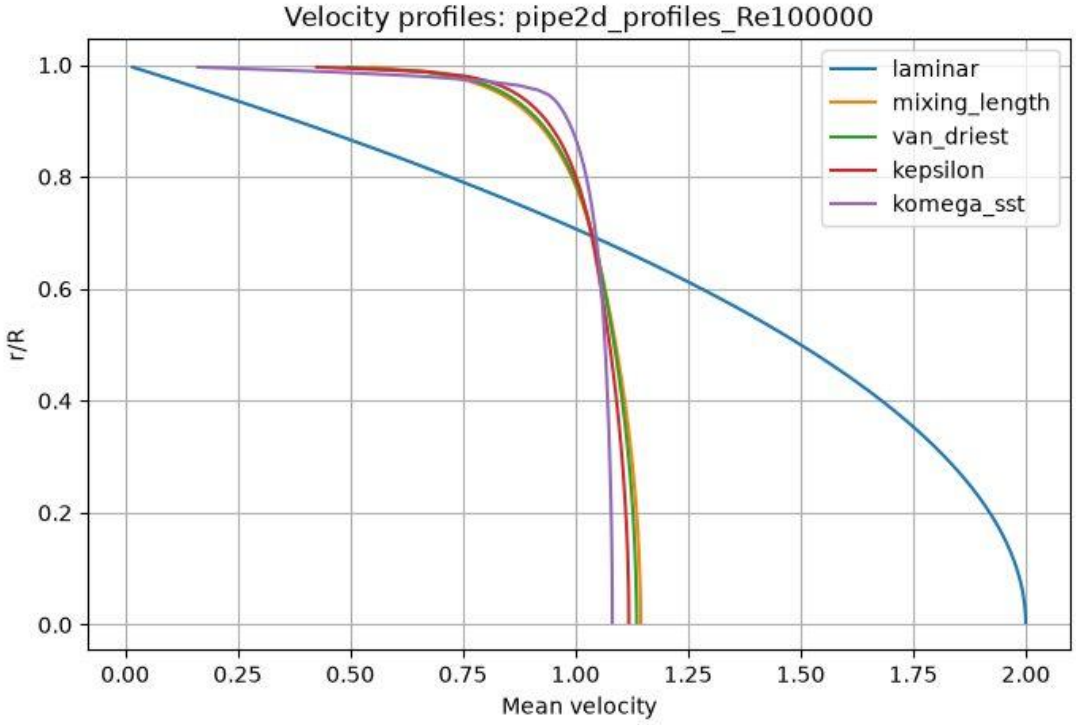
Experiment 10 output: Turbulent Viscosity Ratio Re31623



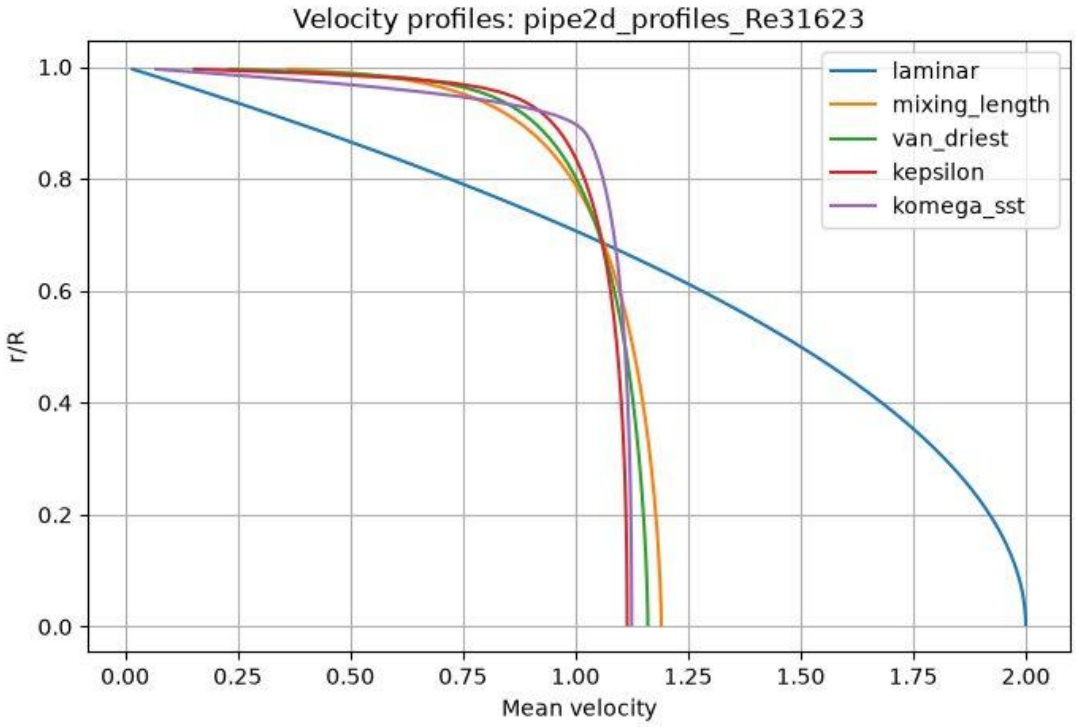
Experiment 10 output: U Mixing Length



Experiment 10 output: Velocity Profile Re10^4



Experiment 10 output: Velocity Profile Re10^5



Experiment 10 output: Velocity Profile Re31623

Results and Discussion

The supplied friction-factor plot compares five model curves with Blasius and Haaland references. The velocity profiles show a parabolic laminar profile and flatter turbulent profiles. The turbulent-viscosity ratio plots show model-dependent radial distributions and larger scales at higher Re. In the supplied profiles, k–ω SST appears among the fullest/most plug-like velocity distributions; this observation is specific to the saved output and model settings.

Conclusion

The experiment demonstrates how turbulence closures affect pipe velocity profiles, eddy viscosity and friction-factor prediction. The three Reynolds-number cases are present in the saved archive. Since the teaching models simplify turbulence closure physics, their outputs should be compared with the reference correlations rather than treated as universal high-fidelity predictions.

EXPERIMENT 11 – Convection–diffusion transport

Aim

To solve a two-dimensional scalar convection–diffusion equation for three prescribed transport configurations and study the effect of Peclet number.

Objective

- Solve the scalar transport equation using prescribed velocity fields and an upwind convection discretisation.
- Compare forced flow over a flat plate, forced flow in a pipe/channel and a natural-convection surrogate case.
- Run Pe=50, 200 and 1000 for all three modes.
- Compare scalar-field shape, thermal boundary-layer thickness, outlet or mean temperature, wall flux and residual convergence.

Governing Equation

Scalar transport equation:

$\partial T/\partial t + u\partial T/\partial x + v\partial T/\partial y = \alpha\nabla^2 T$.

Peclet number: $Pe=UL/\alpha$; the code sets $\alpha=1/Pe$ in its nondimensional cases. Spatial convection derivatives are upwinded and diffusion is represented by a central Laplacian. The explicit time step is selected from convective and diffusive restrictions.

Boundary Condition

- Forced flat plate: inlet T=0; upper wall is heated to T=1 from x≥0.25L and otherwise T=0; lower/far boundary T=0; outlet zero streamwise gradient.
- Forced pipe/channel: inlet T=0; both channel walls T=1; outlet zero streamwise gradient.
- Natural vertical plate surrogate: left boundary T=1, right boundary T=0; top and bottom use zero-normal-gradient conditions.
- The natural-convection case uses a prescribed divergence-free recirculating velocity field; it is not a coupled buoyancy-driven Navier–Stokes solution.

Code

Parameters in the updated notebook: mode='all', nx=121, ny=61, Pe=50/200/1000, iterations=3000 and output_every=300. The code supports forced_plate, forced_pipe and natural_plate modes.

```
# ===== NOTEBOOK CODE CELL 0 =====
from pathlib import Path
import shutil
import subprocess
import sys
import zipfile
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Markdown, display

IN_COLAB = False
try:
    from google.colab import files as colab_files
    IN_COLAB = True
except ImportError:
    colab_files = None


def run_solver(script_path, arguments, output_dir, timeout=1800):
    output_dir = Path(output_dir)
    if output_dir.exists():
        shutil.rmtree(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)
    command = [sys.executable, str(script_path), *[str(value) for value in arguments]]
    print("Running:", " ".join(command))
    completed = subprocess.run(command, cwd=output_dir, capture_output=True, text=True, timeout=timeout)
    (output_dir / "solver_stdout.log").write_text(completed.stdout, encoding="utf-8")
```

```
    (output_dir / "solver_stderr.log").write_text(completed.stderr, encoding="utf-8")
    if completed.stdout.strip():
        print(completed.stdout.strip())
    if completed.returncode != 0:
        raise RuntimeError(completed.stderr.strip() or f"Solver exited with code {completed.returncode}")
    if completed.stderr.strip():
        print(completed.stderr.strip())
    return output_dir


def read_csv(path):
    data = np.genfromtxt(path, delimiter=",", names=True, dtype=None, encoding="utf-8")
    return np.atleast_1d(data)


def show_summary(path):
    text = Path(path).read_text(encoding="utf-8").strip()
    display(Markdown(f"### Solver summary\n```text\n{text}\n```"))
    return text


def list_outputs(output_dir):
    rows = []
    for path in sorted(Path(output_dir).iterdir()):
        if path.is_file():
            size = path.stat().st_size
            if size < 1024:
                size_text = f"{size} B"
            elif size < 1024**2:
                size_text = f"{size / 1024:.1f} KB"
            else:
                size_text = f"{size / 1024**2:.1f} MB"
            rows.append((path.name, size_text))
    table = "\n".join(f"| `{name}` | {size} |" for name, size in rows)
    display(Markdown(f"### Generated files\n| File | Size |\n|---|---:|\n{table}"))
    return rows


def read_structured_vtk(path):
    lines = Path(path).read_text(encoding="utf-8").splitlines()
    dims_index = next(i for i, line in enumerate(lines) if line.startswith("DIMENSIONS "))
    _, nx, ny, nz = lines[dims_index].split()
    nx, ny, nz = int(nx), int(ny), int(nz)
    if nz != 1:
        raise ValueError("Only 2D structured VTK files are supported by this notebook preview")
    points_index = next(i for i, line in enumerate(lines) if line.startswith("POINTS "))
    npoints = int(lines[points_index].split()[1])
    points = np.array([[float(v) for v in lines[points_index + 1 + k].split()[:3]] for k in range(npoints)])
    x = points[:, 0].reshape(ny, nx)
    y = points[:, 1].reshape(ny, nx)
    data_index = next(i for i, line in enumerate(lines) if line.startswith("POINT_DATA ")) + 1
    scalars = {}
    vectors = {}
    i = data_index
    while i < len(lines):
        if lines[i].startswith("SCALARS "):
            name = lines[i].split()[1]
            i += 2
            values = np.array([float(lines[i + k].split()[0]) for k in range(npoints)])
            scalars[name] = values.reshape(ny, nx)
            i += npoints
        elif lines[i].startswith("VECTORS "):
            name = lines[i].split()[1]
            i += 1
            values = np.array([[float(v) for v in lines[i + k].split()[:3]] for k in range(npoints)])
            vectors[name] = (values[:, 0].reshape(ny, nx), values[:, 1].reshape(ny, nx))
            i += npoints
        else:
            i += 1
    return x, y, scalars, vectors


def plot_vtk_scalar(path, preferred_field=None):
    path = Path(path)
    if not path.is_file():
        print(f"VTK preview skipped: {path.name} was not generated")
        return
    x, y, scalars, _ = read_structured_vtk(path)
    if not scalars:
        print("VTK preview skipped: no scalar field found")
        return
    field = preferred_field if preferred_field in scalars else next(iter(scalars))
    plt.figure(figsize=(8, 4.8))
    image = plt.contourf(x, y, scalars[field], levels=30)
    plt.colorbar(image, label=field.replace("_", " "))
    plt.xlabel("x")
    plt.ylabel("y")
    plt.title(f"{field.replace('_', ' ').title()} - inline field preview")
    plt.axis("equal")
    plt.tight_layout()
    plt.show()


def make_results_zip(output_dir, experiment_number):
    output_dir = Path(output_dir)
    archive = output_dir.parent / f"experiment_{experiment_number:02d}_results.zip"
    if archive.exists():
        archive.unlink()
    with zipfile.ZipFile(archive, "w", compression=zipfile.ZIP_DEFLATED) as handle:
```

```
        for path in sorted(output_dir.rglob("*")):
            if path.is_file():
                handle.write(path, path.relative_to(output_dir))
    print(f"Results archive: {archive} ({archive.stat().st_size / 1024:.1f} KB)")
    return archive


EXPERIMENT = 11
ROOT = Path("/content/cfd_lab_exp11") if IN_COLAB else Path.home() / "CFD_Runs" / "exp11"
CODE_DIR = ROOT / "code"
OUTPUT_DIR = ROOT / "outputs"
CODE_DIR.mkdir(parents=True, exist_ok=True)
(CODE_DIR / "common.py").write_text('from __future__ import annotations\n\nimport csv\nimport math\nfrom pathlib import Path\nfrom typing
import Iterable, Mapping\n\nimport numpy as np\n\n\ndef trapezoidal_integral(values: np.ndarray, coordinates: np.ndarray) -> float:\n    y
= np.asarray(values, dtype=float).ravel()\n    x = np.asarray(coordinates, dtype=float).ravel()\n    if y.size != x.size or y.size < 2:\n
raise ValueError("trapezoidal integration requires equally sized arrays with at least two points")\n    return float(np.sum(0.5 * (y[1:] +
y[:-1]) * np.diff(x)))\n\n\ndef write_csv(path: str | Path, header: Iterable[str], rows: Iterable[Iterable[object]]) -> None:\n    with
Path(path).open("w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(list(header))\n
writer.writerows(rows)\n\n\ndef mesh_xy(nx: int, ny: int, lx: float = 1.0, ly: float = 1.0, x0: float = 0.0, y0: float = 0.0) ->
tuple[np.ndarray, np.ndarray]:\n    x = np.linspace(x0, x0 + lx, nx)\n    y = np.linspace(y0, y0 + ly, ny)\n    return np.meshgrid(x,
y)\n\n\ndef write_structured_grid_vtk(\n    path: str | Path,\n    x: np.ndarray,\n    y: np.ndarray,\n    scalars: Mapping[str,
np.ndarray] | None = None,\n    vectors: Mapping[str, tuple[np.ndarray, np.ndarray]] | None = None,\n    title: str = "CFD lab output",\n)
-> None:\n    x = np.asarray(x, dtype=float)\n    y = np.asarray(y, dtype=float)\n    if x.shape != y.shape or x.ndim != 2:\n        raise
ValueError("x and y must be equally shaped 2D arrays")\n    ny, nx = x.shape\n    scalar_map = dict(scalars or {})\n    vector_map =
dict(vectors or {})\n    for name, values in scalar_map.items():\n        if np.asarray(values).shape != x.shape:\n            raise
ValueError(f"scalar {name} has shape {np.asarray(values).shape}, expected {x.shape}")\n    for name, values in vector_map.items():\n
if len(values) != 2 or np.asarray(values[0]).shape != x.shape or np.asarray(values[1]).shape != x.shape:\n            raise
ValueError(f"vector {name} has incompatible shape")\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("#
vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET
STRUCTURED_GRID\\n")\n        handle.write(f"DIMENSIONS {nx} {ny} 1\\n")\n        handle.write(f"POINTS {nx * ny} float\\n")\n        for j
in range(ny):\n            for i in range(nx):\n                handle.write(f"{x[j, i]:.12e} {y[j, i]:.12e} 0\\n")\n
handle.write(f"POINT_DATA {nx * ny}\\n")\n        for name, values in scalar_map.items():\n            data = np.asarray(values,
dtype=float)\n            handle.write(f"SCALARS {name} float 1\\n")\n            handle.write("LOOKUP_TABLE default\\n")\n            for
value in data.ravel(order="C"):\n                handle.write(f"{value:.12e}\\n")\n        for name, values in vector_map.items():\n
vx = np.asarray(values[0], dtype=float)\n            vy = np.asarray(values[1], dtype=float)\n            handle.write(f"VECTORS {name}
float\\n")\n            for ax, ay in zip(vx.ravel(order="C"), vy.ravel(order="C")):\n                handle.write(f"{ax:.12e} {ay:.12e}
0\\n")\n\n\ndef write_polyline_vtk(path: str | Path, x: np.ndarray, y: np.ndarray, title: str = "polyline", close: bool = False) -> None:\n
x = np.asarray(x, dtype=float).ravel()\n    y = np.asarray(y, dtype=float).ravel()\n    if x.size != y.size or x.size < 2:\n        raise
ValueError("polyline requires equally sized x and y arrays")\n    ids = list(range(x.size))\n    if close:\n        ids.append(0)\n    with
Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n
handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET POLYDATA\\n")\n        handle.write(f"POINTS
{x.size} float\\n")\n        for xi, yi in zip(x, y):\n            handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n        handle.write(f"LINES
1 {len(ids) + 1}\\n")\n        handle.write(f"{len(ids)} {\' \'.join(str(i) for i in ids)}\\n")\n\n\ndef write_multiline_vtk(path: str |
Path, lines: list[tuple[np.ndarray, np.ndarray]], title: str = "lines") -> None:\n    points: list[tuple[float, float]] = []\n
connectivity: list[list[int]] = []\n    for x, y in lines:\n        xa = np.asarray(x, dtype=float).ravel()\n        ya = np.asarray(y,
dtype=float).ravel()\n        if xa.size != ya.size or xa.size < 2:\n            raise ValueError("each line needs equally sized x and y
arrays")\n        ids = list(range(len(points), len(points) + xa.size))\n        points.extend(zip(xa.tolist(), ya.tolist()))\n
connectivity.append(ids)\n    size = sum(len(ids) + 1 for ids in connectivity)\n    with Path(path).open("w", encoding="utf-8") as
handle:\n        handle.write("# vtk DataFile Version 3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n
handle.write("DATASET POLYDATA\\n")\n        handle.write(f"POINTS {len(points)} float\\n")\n        for xi, yi in points:\n
handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n        handle.write(f"LINES {len(connectivity)} {size}\\n")\n        for ids in connectivity:\n
handle.write(f"{len(ids)} {\' \'.join(str(i) for i in ids)}\\n")\n\n\ndef write_points_vtk(path: str | Path, x: np.ndarray, y: np.ndarray,
scalars: Mapping[str, np.ndarray] | None = None, title: str = "points") -> None:\n    xa = np.asarray(x, dtype=float).ravel()\n    ya =
np.asarray(y, dtype=float).ravel()\n    if xa.size != ya.size:\n        raise ValueError("point arrays must have equal size")\n
scalar_map = dict(scalars or {})\n    with Path(path).open("w", encoding="utf-8") as handle:\n        handle.write("# vtk DataFile Version
3.0\\n")\n        handle.write(f"{title}\\n")\n        handle.write("ASCII\\n")\n        handle.write("DATASET POLYDATA\\n")\n
handle.write(f"POINTS {xa.size} float\\n")\n        for xi, yi in zip(xa, ya):\n            handle.write(f"{xi:.12e} {yi:.12e} 0\\n")\n
handle.write(f"VERTICES {xa.size} {2 * xa.size}\\n")\n        for i in range(xa.size):\n            handle.write(f"1 {i}\\n")\n        if
scalar_map:\n            handle.write(f"POINT_DATA {xa.size}\\n")\n            for name, values in scalar_map.items():\n
data = np.asarray(values, dtype=float).ravel()\n                if data.size != xa.size:\n                    raise ValueError(f"scalar
{name} has wrong size")\n                handle.write(f"SCALARS {name} float 1\\n")\n                handle.write("LOOKUP_TABLE
default\\n")\n                for value in data:\n                    handle.write(f"{value:.12e}\\n")\n\n\ndef gradient(field: np.ndarray,
dx: float, dy: float) -> tuple[np.ndarray, np.ndarray]:\n    gy, gx = np.gradient(field, dy, dx, edge_order=2)\n    return gx, gy\n\n\ndef
laplacian(field: np.ndarray, dx: float, dy: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    out[1:-1, 1:-1] = (\n
(field[1:-1, 2:] - 2.0 * field[1:-1, 1:-1] + field[1:-1, :-2]) / (dx * dx)\n        + (field[2:, 1:-1] - 2.0 * field[1:-1, 1:-1] + field[:-
2, 1:-1]) / (dy * dy)\n    )\n    return out\n\n\ndef upwind_derivative_x(field: np.ndarray, velocity: np.ndarray, dx: float) ->
np.ndarray:\n    out = np.zeros_like(field)\n    backward = (field[1:-1, 1:-1] - field[1:-1, :-2]) / dx\n    forward = (field[1:-1, 2:] -
field[1:-1, 1:-1]) / dx\n    out[1:-1, 1:-1] = np.where(velocity[1:-1, 1:-1] >= 0.0, backward, forward)\n    return out\n\n\ndef
upwind_derivative_y(field: np.ndarray, velocity: np.ndarray, dy: float) -> np.ndarray:\n    out = np.zeros_like(field)\n    backward =
(field[1:-1, 1:-1] - field[:-2, 1:-1]) / dy\n    forward = (field[2:, 1:-1] - field[1:-1, 1:-1]) / dy\n    out[1:-1, 1:-1] =
np.where(velocity[1:-1, 1:-1] >= 0.0, backward, forward)\n    return out\n\n\ndef poisson_jacobi(phi: np.ndarray, rhs: np.ndarray, dx:
float, dy: float, iterations: int, boundary: str = "neumann", mask: np.ndarray | None = None) -> np.ndarray:\n    dx2 = dx * dx\n    dy2 =
dy * dy\n    denom = 2.0 * (dx2 + dy2)\n    for _ in range(iterations):\n        old = phi.copy()\n        phi[1:-1, 1:-1] = (\n
(old[1:-1, 2:] + old[1:-1, :-2]) * dy2\n            + (old[2:, 1:-1] + old[:-2, 1:-1]) * dx2\n            - rhs[1:-1, 1:-1] * dx2 * dy2\n
) / denom\n        if mask is not None:\n            phi[mask > 0.5] = 0.0\n        if boundary == "neumann":\n            phi[:, 0] =
phi[:, 1]\n            phi[:, -1] = 0.0\n            phi[0, :] = phi[1, :]\n            phi[-1, :] = phi[-2, :]\n        elif boundary ==
"closed":\n            phi[:, 0] = phi[:, 1]\n            phi[:, -1] = phi[:, -2]\n            phi[0, :] = phi[1, :]\n            phi[-1,
:] = phi[-2, :]\n            phi -= np.mean(phi)\n        else:\n            raise ValueError("unsupported pressure boundary")\n    return
phi\n\n\ndef tdma(a: np.ndarray, b: np.ndarray, c: np.ndarray, d: np.ndarray) -> np.ndarray:\n    a = np.asarray(a, dtype=float).copy()\n
b = np.asarray(b, dtype=float).copy()\n    c = np.asarray(c, dtype=float).copy()\n    d = np.asarray(d, dtype=float).copy()\n    n =
d.size\n    if n == 0 or a.size != n or b.size != n or c.size != n:\n        raise ValueError("TDMA arrays must have equal nonzero size")\n
for i in range(1, n):\n        if abs(b[i - 1]) < 1e-30:\n            raise FloatingPointError("zero TDMA pivot")\n        factor = a[i] /
b[i - 1]\n        b[i] -= factor * c[i - 1]\n        d[i] -= factor * d[i - 1]\n    x = np.empty(n, dtype=float)\n    if abs(b[-1]) < 1e-
30:\n        raise FloatingPointError("zero TDMA pivot")\n    x[-1] = d[-1] / b[-1]\n    for i in range(n - 2, -1, -1):\n        if
abs(b[i]) < 1e-30:\n            raise FloatingPointError("zero TDMA pivot")\n        x[i] = (d[i] - c[i] * x[i + 1]) / b[i]\n    return
x\n\n\ndef relative_l2(numerical: np.ndarray, reference: np.ndarray) -> float:\n    a = np.asarray(numerical, dtype=float)\n    b =
np.asarray(reference, dtype=float)\n    denominator = float(np.linalg.norm(b.ravel()))\n    numerator = float(np.linalg.norm((a -
b).ravel()))\n    return numerator / max(denominator, 1e-30)\n\n\ndef require_finite(*arrays: np.ndarray) -> None:\n    for array in
arrays:\n        if not np.all(np.isfinite(array)):\n            raise FloatingPointError("non-finite numerical field detected")\n\n\ndef
positive_float(value: str, name: str) -> float:\n    result = float(value)\n    if not math.isfinite(result) or result <= 0.0:\n
raise ValueError(f"{name} must be positive and finite")\n    return result\n\n\ndef positive_int(value: str, name: str, minimum: int = 1) -
> int:\n    result = int(value)\n    if result < minimum:\n        raise ValueError(f"{name} must be at least {minimum}")\n    return
result\n', encoding="utf-8")
(CODE_DIR / 'exp11_convection_diffusion.py').write_text('from __future__ import annotations\n\nimport csv\nimport math\nimport sys\nfrom
dataclasses import dataclass\n\nimport numpy as np\n\nfrom common import laplacian, require_finite, upwind_derivative_x,
upwind_derivative_y, write_multiline_vtk, write_structured_grid_vtk\n\n\n@dataclass\nclass CaseData:\n    name: str\n    x: np.ndarray\n
y: np.ndarray\n    temperature: np.ndarray\n    u: np.ndarray\n    v: np.ndarray\n    marker: np.ndarray\n    pe: float\n    alpha: float\n
dx: float\n    dy: float\n\n\ndef initialize_case(mode: str, nx: int, ny: int, pe: float) -> CaseData:\n    if mode == "forced_plate":\n
length, height = 5.0, 1.0\n    elif mode == "forced_pipe":\n        length, height = 5.0, 1.0\n    elif mode == "natural_plate":\n
```

```
length, height = 1.0, 1.0\n    else:\n        raise ValueError("unknown case")\n    x1 = np.linspace(0.0, length, nx)\n    y1 = np.linspace(0.0, height, ny)\n    x, y = np.meshgrid(x1, y1)\n    dx = x1[1] - x1[0]\n    dy = y1[1] - y1[0]\n    temperature = np.zeros((ny, nx), dtype=float)\n    marker = np.zeros_like(temperature)\n    if mode == "forced_plate":\n        delta = 0.035 + 0.22 * np.sqrt(np.maximum(x, dx) / length)\n        u = np.tanh(y / np.maximum(delta, 1e-12))\n        v = np.zeros_like(u)\n        marker[0, x1 >= 0.25 * length] = 1.0\n        name = "forced_flat_plate"\n    elif mode == "forced_pipe":\n        eta = y / height\n        u = 6.0 * eta * (1.0 - eta)\n        v = np.zeros_like(u)\n        marker[0, :] = 1.0\n        marker[-1, :] = 1.0\n        name = "forced_pipe_channel"\n    else:\n        amplitude = 1.0 / math.pi\n        u = -amplitude * math.pi / height * np.sin(math.pi * x / length) * np.cos(math.pi * y / height)\n        v = amplitude * math.pi / length * np.cos(math.pi * x / length) * np.sin(math.pi * y / height)\n        temperature = 1.0 - x / length\n        marker[:, 0] = 1.0\n        marker[:, -1] = 2.0\n        name = "natural_vertical_plate_surrogate"\n    return CaseData(name, x, y, temperature, u, v, marker, pe, 1.0 / pe, dx, dy)\n\n\ndef apply_bc(case: CaseData) -> None:\n    t = case.temperature\n    ny, nx = t.shape\n    if case.name == "forced_flat_plate":\n        t[:, 0] = 0.0\n        t[-1, :] = 0.0\n        heated = case.x[0, :] >= 0.25 * case.x[0, -1]\n        t[0, :] = np.where(heated, 1.0, 0.0)\n        t[1:-1, -1] = t[1:-1, -2]\n    elif case.name == "forced_pipe_channel":\n        t[:, 0] = 0.0\n        t[0, :] = 1.0\n        t[-1, :] = 1.0\n        t[1:-1, -1] = t[1:-1, -2]\n    else:\n        t[:, 0] = 1.0\n        t[:, -1] = 0.0\n        t[0, 1:-1] = t[1, 1:-1]\n        t[-1, 1:-1] = t[-2, 1:-1]\n\n\ndef outlet_or_mean(case: CaseData) -> float:\n    if case.name == "natural_vertical_plate_surrogate":\n        return float(np.mean(case.temperature[1:-1, 1:-1]))\n    weights = np.maximum(case.u[1:-1, -1], 0.0)\n    if float(np.sum(weights)) < 1e-14:\n        return float(np.mean(case.temperature[1:-1, -1]))\n    return float(np.average(case.temperature[1:-1, -1], weights=weights))\n\n\ndef average_wall_flux(case: CaseData) -> float:\n    t = case.temperature\n    if case.name == "forced_flat_plate":\n        heated = case.x[0, 1:-1] >= 0.25 * case.x[0, -1]\n        flux = -(t[1, 1:-1] - t[0, 1:-1]) / case.dy\n        return float(np.mean(flux[heated]))\n    if case.name == "forced_pipe_channel":\n        bottom = np.abs((t[0, 1:-1] - t[1, 1:-1]) / case.dy)\n        top = np.abs((t[-1, 1:-1] - t[-2, 1:-1]) / case.dy)\n        return float(np.mean(np.concatenate((bottom, top))))\n    flux = -(t[1:-1, 1] - t[1:-1, 0]) / case.dx\n    return float(np.mean(flux))\n\n\ndef run_case(case: CaseData, iterations: int, output_every: int) -> list[object]:\n    apply_bc(case)\n    max_u = max(float(np.max(np.abs(case.u))), 1e-12)\n    max_v = max(float(np.max(np.abs(case.v))), 1e-12)\n    dt_convective = 0.35 * min(case.dx / max_u, case.dy / max_v)\n    dt_diffusive = 0.20 * min(case.dx * case.dx, case.dy * case.dy) / case.alpha\n    dt = min(dt_convective, dt_diffusive)\n    history: list[list[float]] = []\n    residual = float("inf")\n    for iteration in range(1, iterations + 1):\n        old = case.temperature.copy()\n        convection = case.u * upwind_derivative_x(old, case.u, case.dx) + case.v * upwind_derivative_y(old, case.v, case.dy)\n        updated = old + dt * (-convection + case.alpha * laplacian(old, case.dx, case.dy))\n        case.temperature[1:-1, 1:-1] = np.clip(updated[1:-1, 1:-1], -0.05, 1.05)\n        apply_bc(case)\n        residual = float(np.sqrt(np.mean((case.temperature[1:-1, 1:-1] - old[1:-1, 1:-1]) ** 2)))\n        if iteration == 1 or iteration == iterations or iteration % output_every == 0:\n            outlet = outlet_or_mean(case)\n            flux = average_wall_flux(case)\n            history.append([iteration, residual, outlet, flux, dt])\n            print(f"case={case.name} iteration={iteration} residual={residual:.6e} outlet_or_mean_temperature={outlet:.6e} wall_flux={flux:.6e}")\n        require_finite(case.temperature)\n        if residual < 1e-9 and iteration > 100:\n            break\n    speed = np.sqrt(case.u * case.u + case.v * case.v)\n    local_pe = speed / max(case.alpha, 1e-30)\n    write_structured_grid_vtk(f"{case.name}.vtk", case.x, case.y, {"temperature": case.temperature, "speed": speed, "wall_or_boundary_marker": case.marker, "local_peclet_speed_based": local_pe}, {"velocity": (case.u, case.v)}, case.name)\n    length = case.x[0, -1]\n    height = case.y[-1, 0]\n    if case.name in {"forced_flat_plate", "forced_pipe_channel"}:\n        lines = [(np.array([0.0, length]), np.array([0.0, 0.0])), (np.array([0.0, length]), np.array([height, height]))]\n    else:\n        lines = [(np.array([0.0, 0.0]), np.array([0.0, height])), (np.array([length, length]), np.array([0.0, height]))]\n    write_multiline_vtk(f"{case.name}_boundaries.vtk", lines, f"{case.name} boundaries")\n    with open(f"{case.name}_history.csv", "w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(["iteration", "residual_l2", "outlet_or_mean_temperature", "average_wall_flux", "dt"])\n        writer.writerows(history)\n    i_mid = case.temperature.shape[1] // 2\n    j_mid = case.temperature.shape[0] // 2\n    with open(f"{case.name}_profiles.csv", "w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(["x", "y", "temperature", "u", "v", "speed"])\n        for j in range(case.temperature.shape[0]):\n            writer.writerow([case.x[j, i_mid], case.y[j, i_mid], case.temperature[j, i_mid], case.u[j, i_mid], case.v[j, i_mid], speed[j, i_mid]])\n        for i in range(case.temperature.shape[1]):\n            writer.writerow([case.x[j_mid, i], case.y[j_mid, i], case.temperature[j_mid, i], case.u[j_mid, i], case.v[j_mid, i], speed[j_mid, i]])\n    return [case.name, case.temperature.shape[1], case.temperature.shape[0], case.pe, dt, residual, outlet_or_mean(case), average_wall_flux(case)]\n\n\ndef main() -> int:\n    mode = sys.argv[1] if len(sys.argv) > 1 else "all"\n    nx = int(sys.argv[2]) if len(sys.argv) > 2 else 121\n    ny = int(sys.argv[3]) if len(sys.argv) > 3 else 61\n    pe = float(sys.argv[4]) if len(sys.argv) > 4 else 200.0\n    iterations = int(sys.argv[5]) if len(sys.argv) > 5 else 3000\n    output_every = int(sys.argv[6]) if len(sys.argv) > 6 else 300\n    if nx < 21 or ny < 21 or pe < 1.0 or iterations < 1 or output_every < 1:\n        raise ValueError("usage: python exp11_convection_diffusion.py mode nx ny Pe iterations outputEvery")\n    aliases = {\n        "forced_plate": "forced_plate",\n        "flat": "forced_plate",\n        "forced_flat_plate": "forced_plate",\n        "forced_pipe": "forced_pipe",\n        "pipe": "forced_pipe",\n        "forced_pipe_channel": "forced_pipe",\n        "natural_plate": "natural_plate",\n        "natural": "natural_plate",\n        "natural_vertical_plate_surrogate": "natural_plate",\n    }\n    modes = ["forced_plate", "forced_pipe", "natural_plate"] if mode == "all" else [aliases.get(mode, "")]\n    if not all(modes):\n        raise ValueError("mode must be all, forced_plate, forced_pipe, or natural_plate")\n    rows = []\n    for selected in modes:\n        rows.append(run_case(initialize_case(selected, nx, ny, pe), iterations, output_every))\n    with open("convective_transport_summary.csv", "w", newline="", encoding="utf-8") as handle:\n        writer = csv.writer(handle)\n        writer.writerow(["case", "nx", "ny", "Pe", "dt", "final_residual", "outlet_or_mean_temperature", "average_wall_flux"])\n        writer.writerows(rows)\n    with open("convective_transport_summary.txt", "w", encoding="utf-8") as handle:\n        handle.write("Generic 2D convection-diffusion transport equation\\n")\n        handle.write("forced_flat_plate uses prescribed external flow over a partially heated plate.\\n")\n        handle.write("forced_pipe_channel uses prescribed parabolic internal flow between heated walls.\\n")\n        handle.write("natural_vertical_plate_surrogate uses a prescribed divergence-free recirculating field.\\n")\n        handle.write("The natural-convection case is not a coupled Navier-Stokes and energy solution.\\n")\n    print("Wrote convection-diffusion VTK, CSV, boundary, profile, and summary files")\n    return 0\n\n\nif __name__ == "__main__":\n    raise SystemExit(main())\n', encoding="utf-8")
SCRIPT = CODE_DIR / 'exp11_convection_diffusion.py'
print("Runtime:", "Google Colab" if IN_COLAB else "local notebook test")
print("Working folder:", ROOT)
print("No package installation or external file upload is required.")


# ===== NOTEBOOK CODE CELL 2 =====
#@title 2. Select run mode and parameters
RUN_MODE = "custom" #@param ["quick", "lab", "custom"]

mode = "all" #@param ["all", "forced_plate", "forced_pipe", "natural_plate"]
nx = 121 #@param {type:"integer"}
ny = 61 #@param {type:"integer"}
Pe = 1000.0 #@param {type:"number"}
iterations = 3000 #@param {type:"integer"}
output_every = 300 #@param {type:"integer"}

QUICK_ARGS = ['all', 25, 21, 100, 8, 8]
LAB_ARGS = ['all', 121, 61, 200, 3000, 300]
CUSTOM_ARGS = [mode, nx, ny, Pe, iterations, output_every]
ARGS = QUICK_ARGS if RUN_MODE == "quick" else LAB_ARGS if RUN_MODE == "lab" else CUSTOM_ARGS
print("Run mode:", RUN_MODE)
print("Arguments:", ARGS)


# ===== NOTEBOOK CODE CELL 4 =====
#@title 3. Run the numerical solver
OUT = run_solver(SCRIPT, ARGS, OUTPUT_DIR)


# ===== NOTEBOOK CODE CELL 6 =====
#@title 4. Read the solver summary
summary_text = show_summary(OUT / 'convective_transport_summary.txt')
```

```
# ===== NOTEBOOK CODE CELL 8 =====
#@title 5. Plot the primary numerical results
summary = read_csv(OUT / "convective_transport_summary.csv")
labels = [str(value) for value in summary["case"]]
plt.figure(figsize=(8, 4))
plt.bar(labels, summary["outlet_or_mean_temperature"])
plt.ylabel("Outlet or mean temperature")
plt.title("Case comparison")
plt.xticks(rotation=20)
plt.grid(True, axis="y")
plt.show()

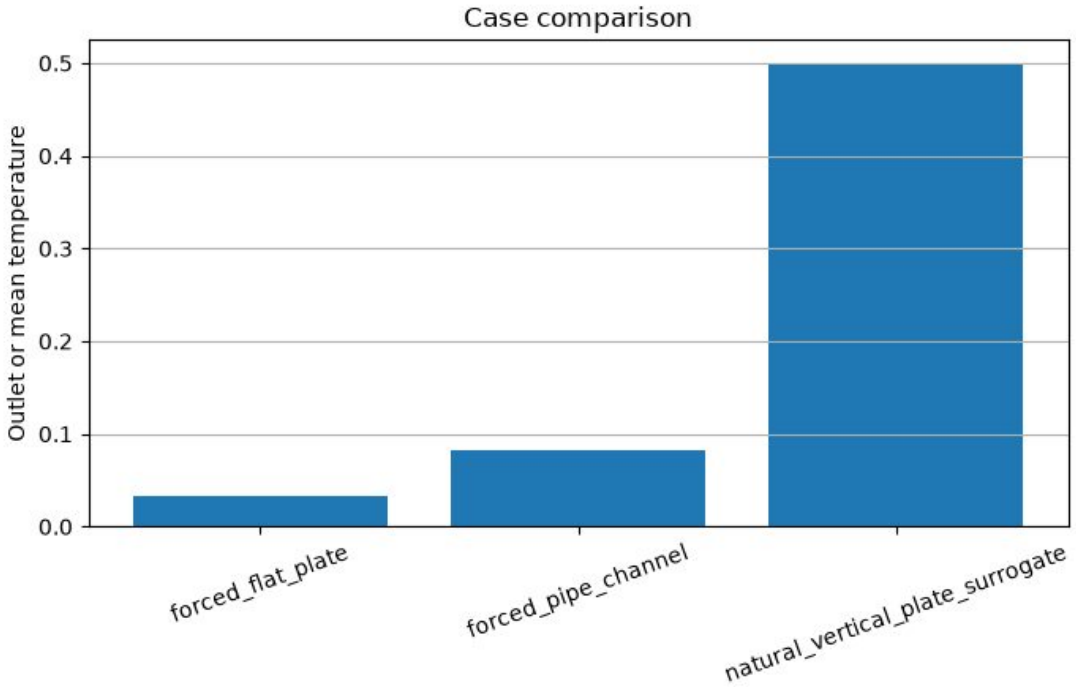
plt.figure(figsize=(8, 4))
for profile_file in sorted(OUT.glob("*_profiles.csv")):
    profiles = read_csv(profile_file)
    vertical_mask = np.isclose(profiles["x"], profiles["x"][0])
    plt.plot(profiles["temperature"][vertical_mask], profiles["y"][vertical_mask], label=profile_file.stem.replace("_profiles", ""))
plt.xlabel("Temperature")
plt.ylabel("Profile coordinate y")
plt.title("Extracted temperature profiles")
plt.legend(fontsize=8)
plt.grid(True)
plt.show()

# ===== NOTEBOOK CODE CELL 10 =====
#@title 6. Display the final field and generated files
plot_vtk_scalar(OUT / 'forced_flat_plate.vtk', 'temperature')
files = list_outputs(OUT)

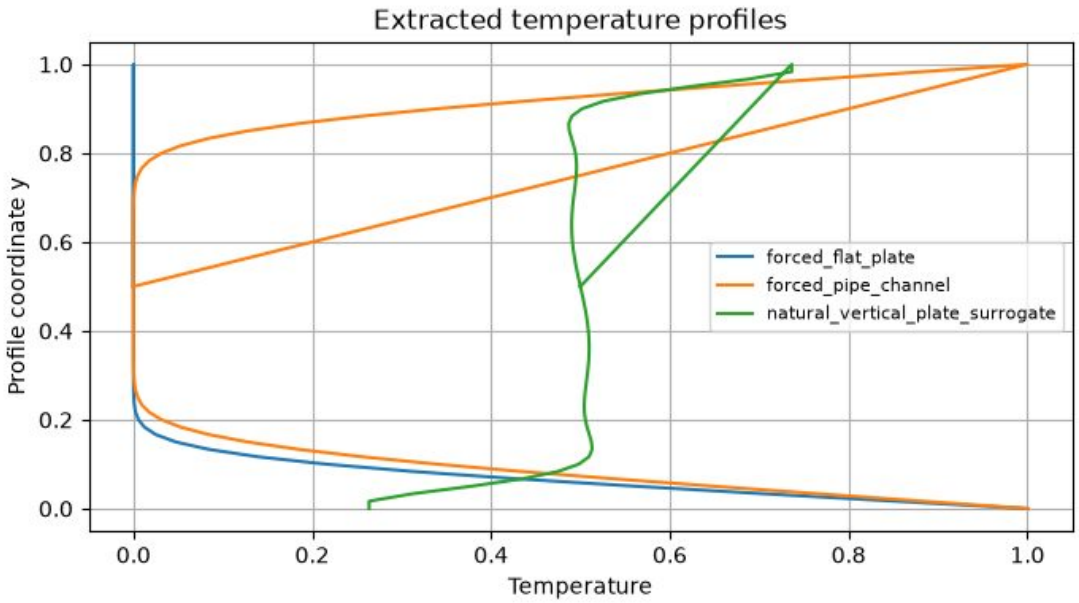

# ===== NOTEBOOK CODE CELL 12 =====
#@title 7. Package and download the results
DOWNLOAD_RESULTS = False #@param {type:"boolean"}
RESULTS_ZIP = make_results_zip(OUT, 11)
if DOWNLOAD_RESULTS:
    if IN_COLAB:
        colab_files.download(str(RESULTS_ZIP))
    else:
        print("Local test mode. Archive retained at:", RESULTS_ZIP)
else:
    print("Set DOWNLOAD_RESULTS to True and run this cell when ready to download.")
```

Output

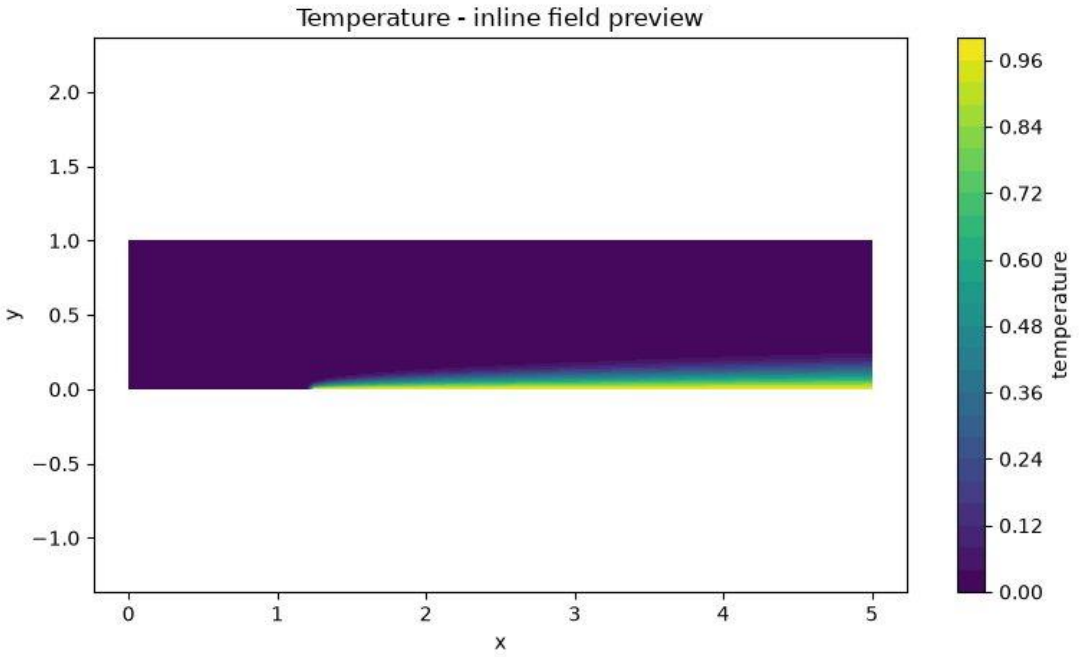
The following figures are reproduced from the output folder supplied for this experiment.



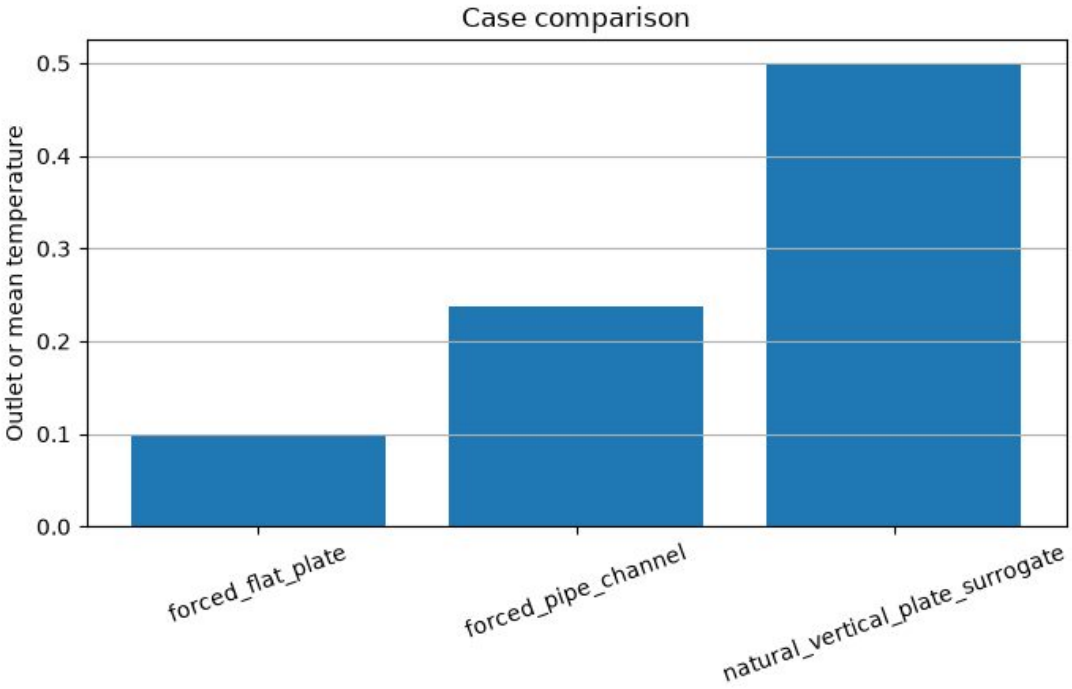
Experiment 11 output: Pe 1000/Case Comparison



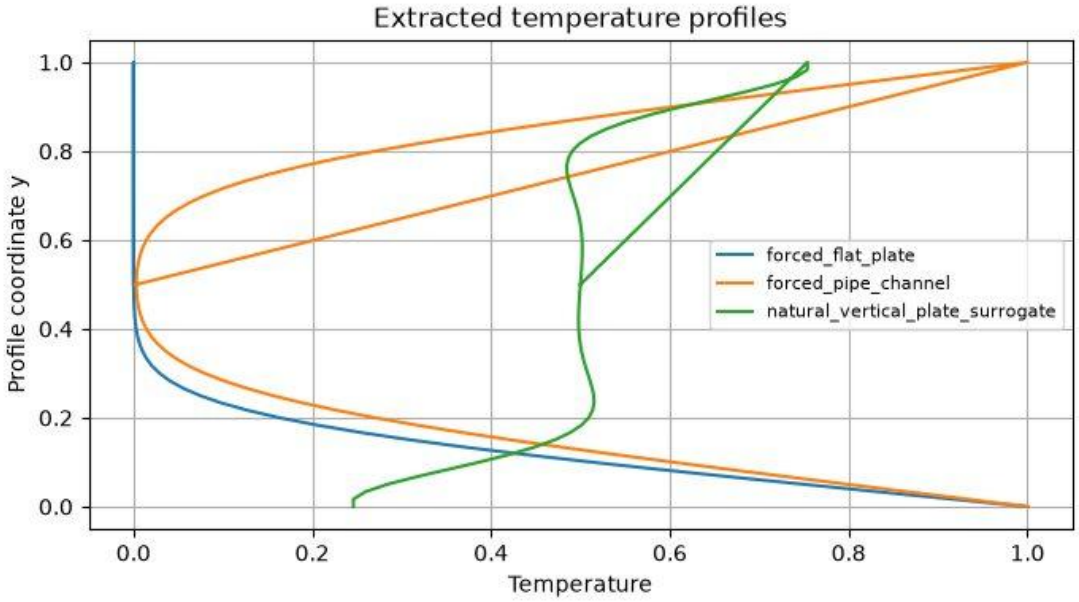
Experiment 11 output: Pe 1000/Extracted Temperature



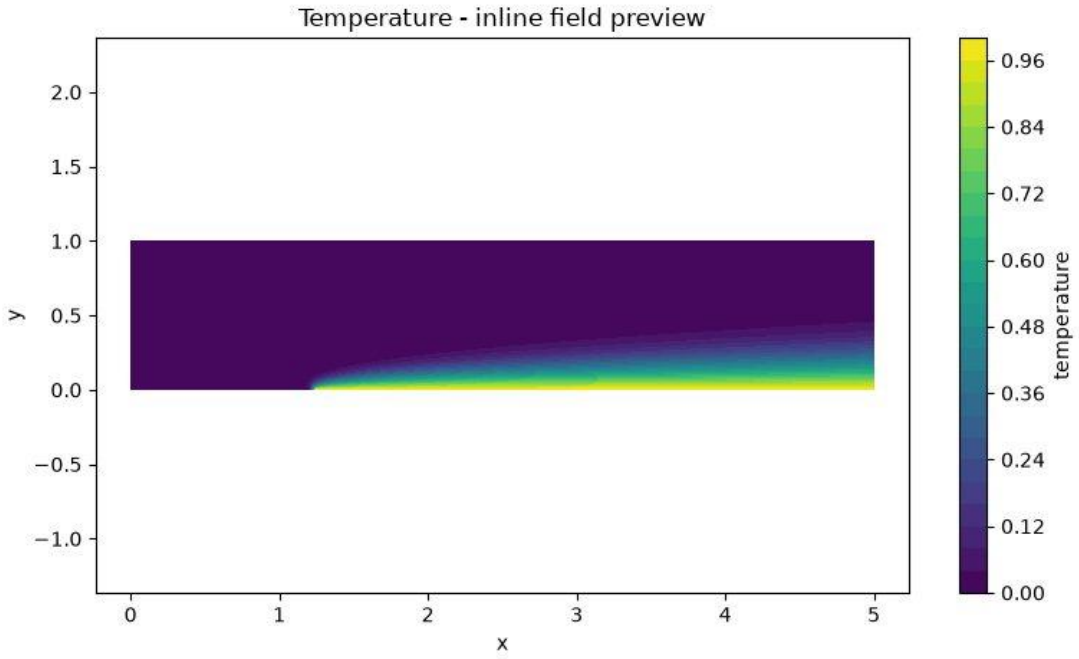
Experiment 11 output: Pe 1000/Temperature



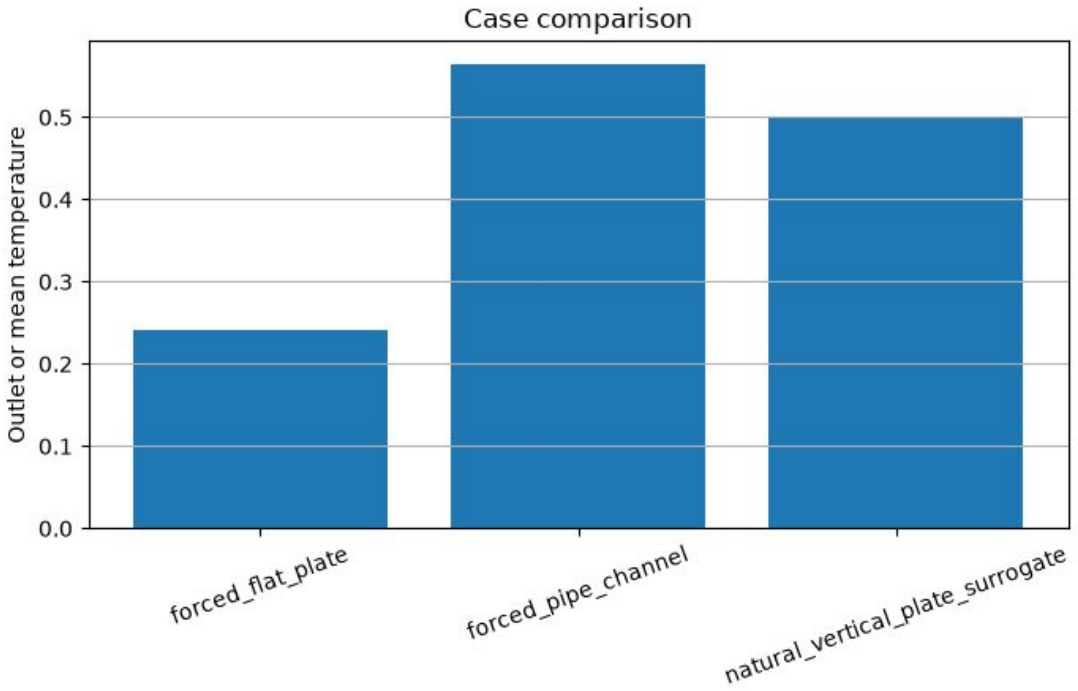
Experiment 11 output: Pe 200/Case Comparison



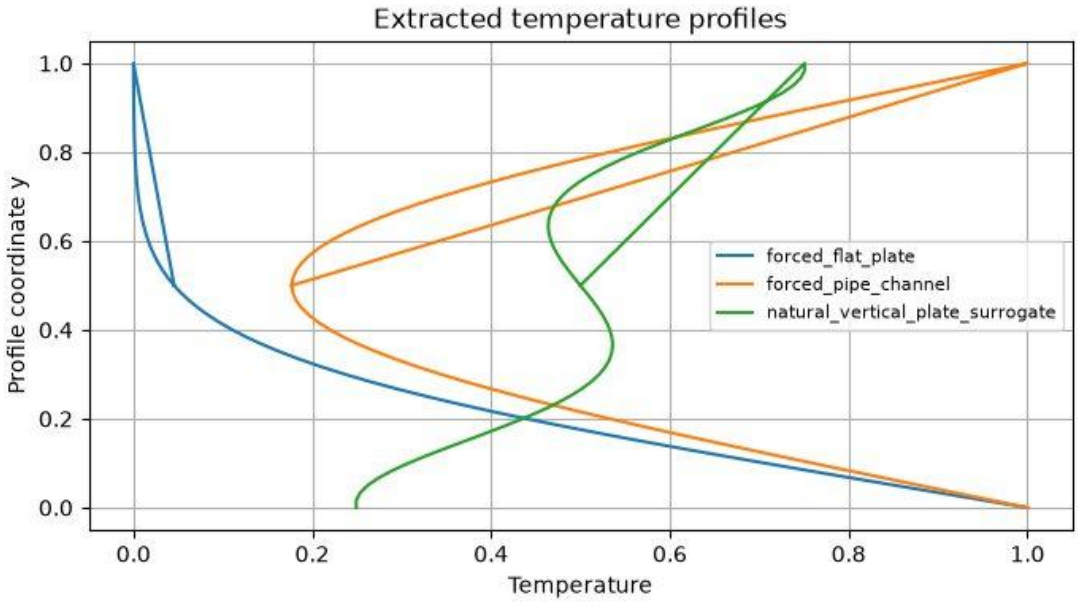
Experiment 11 output: Pe 200/Extracted Temperature Profile



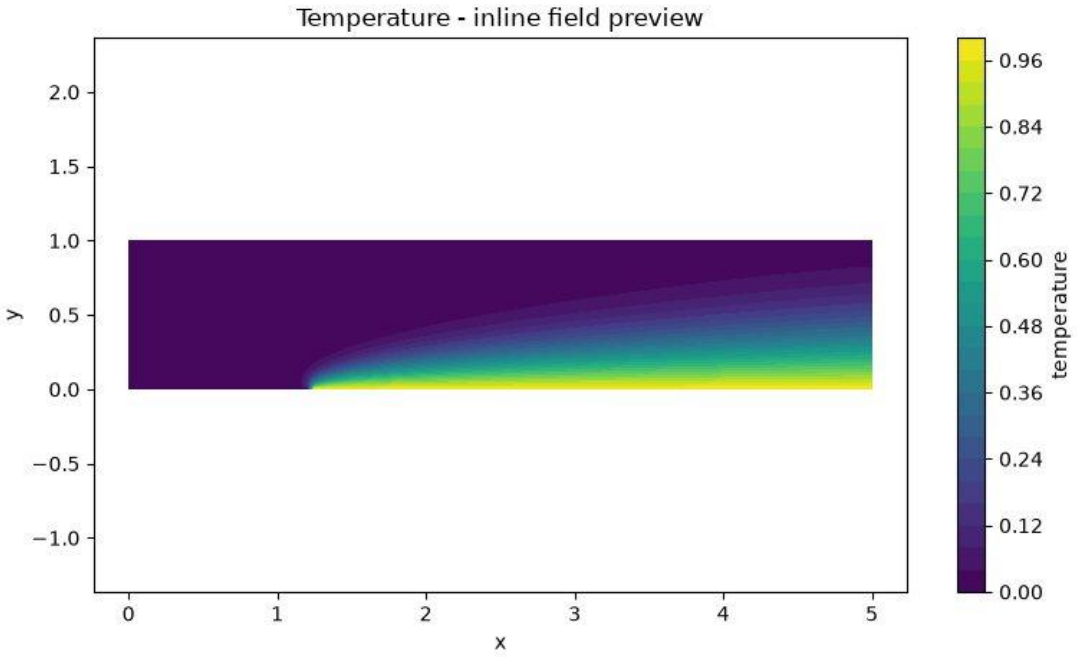
Experiment 11 output: Pe 200/Temperature



Experiment 11 output: Pe 50/Case Comparison



Experiment 11 output: Pe 50/Extracted Temperature Profile



Experiment 11 output: Pe 50/Temperature

Results and Discussion

The archive contains case-comparison, temperature-field and extracted-temperature-profile figures for Pe=50, 200 and 1000. Increasing Pe reduces α=1/Pe, so convection becomes more dominant relative to diffusion; the expected visual consequence is a sharper scalar gradient and thinner diffusion layer. Report actual outlet/mean temperatures, wall fluxes and residuals from the solver summary/CSV if those numerical files are available; the ZIP supplied here primarily stores the PNG outputs.

Conclusion

The three prescribed Peclet-number cases and three transport modes are represented in the saved outputs. The experiment illustrates convection–diffusion competition and the formation of sharper scalar gradients at higher Pe. The natural-convection case is explicitly a surrogate with prescribed flow, so it should not be described as a fully coupled natural-convection CFD solution.